



DobotStudio Pro 用户手册

(MG400 & M1 Pro)



深圳市越疆科技有限公司

目录

前言

1 从这里开始

2 连接机器人

3 主界面说明

3.1 主界面概览

3.2 顶部工具栏

3.3 控制面板

4 设置

4.1 基础设置

4.2 通讯设置

4.3 坐标系管理

4.3.1 用户坐标系

4.3.2 工具坐标系

4.4 负载参数设置

4.5 运动参数设置

4.6 安全设置

4.7 远程控制

4.8 左右手标定 (M1 Pro)

4.9 固件更新

4.10 零点标定

5 I/O监控

6 Modbus

7 全局变量

8 编程

8.1 图形编程

8.2 脚本编程

9 最佳实践

附录A Modbus寄存器定义

附录B 图形编程积木说明

B.1 快速体验

B.1.1 控制机械臂运动

B.1.2 读写Modbus寄存器数据

B.1.3 通过TCP通信传递数据

B.1.4 使用托盘积木进行码垛

B.2 积木说明

B.2.1 事件积木组

B.2.2 控制积木组

B.2.3 运算积木组

B.2.4 字符积木组

B.2.5 自定义积木组

B.2.6 IO积木组

B.2.7 运动积木组

B.2.8 运动高级配置

B.2.9 位姿积木组

B.2.10 Modbus积木组

B.2.11 TCP积木组

附录C 脚本编程函数说明

C.1 Lua基础语法

C.1.1 变量与数据类型

C.1.2 运算符

C.1.3 流程控制

C.2 函数说明

C.2.1 运动指令

C.2.2 运动参数

C.2.3 相对运动指令

C.2.4 IO

C.2.5 TCP/UDP

C.2.6 Modbus

C.2.7 程序控制

C.2.8 视觉

前言

目的

本手册介绍了DobotStudio Pro机械臂控制软件针对四轴机械臂（MG400和M1Pro）的功能、操作等，方便用户了解和使用DobotStudio Pro。

读者对象

本手册适用于：

- 客户
- 销售工程师
- 安装调试工程师
- 技术支持工程师

修订记录

时间	修订记录
2024/01/31	对应控制器1.6.0.0版本
2024/01/05	对应软件2.8.0版本
2023/05/16	更新Modbus寄存器定义，优化部分描述
2023/04/21	优化积木编程快速体验DEMO
2023/01/12	对应软件2.6.0版本
2022/11/30	对应软件2.5.0版本
2022/10/30	对应软件2.4.0版本； 根据最新软件界面调整文档目录，并整体优化内容； 增加三个附录； 将6轴机械臂和4轴机械臂的操作说明拆分为两本手册。
2022/03/25	软件更名为DobotStudio Pro； 根据最新软件界面更新MG400功能说明，增加报警说明、运动参数设置、WiFi设置等； 新增第三章关于CR系列机械臂的操作说明 删除M1操作说明
2021/08/31	增加标定块的操作说明

2021/03/15	增加第四章关于MG400的操作说明
2020/05/20	第一次发布

符号约定

在本手册中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示有高度潜在危险，如果不能避免，会导致人员死亡或严重伤害
 警告	表示有中度或低度潜在危害，如果不能避免，可能导致人员轻微伤害、机械臂毁坏等情况
 注意	表示有潜在风险，如果忽视这些文本，可能导致机械臂损坏、数据丢失或不可预知的结果
 说明	表示是正文的附加信息，是对正文的强调和补充

1 从这里开始

欢迎使用DobotStudio Pro。DobotStudio Pro是越疆针对Dobot机械臂开发的PC端控制软件，具有功能简单易用，实用性强，界面简洁易懂等特点，可以帮助用户快速掌握Dobot机械臂的用法。

本手册主要介绍如何使用DobotStudio Pro控制四轴机械臂（MG400和M1Pro）机械臂。控制MG400和M1 Pro的方法大致相同，本手册主要以MG400为例进行说明。

DobotStudio Pro所支持的操作系统如下所示：

- Win7
- Win10
- Win11

安装软件

请从[官网](#)下载最新的DobotStudio Pro安装包并进行安装，安装步骤如下：

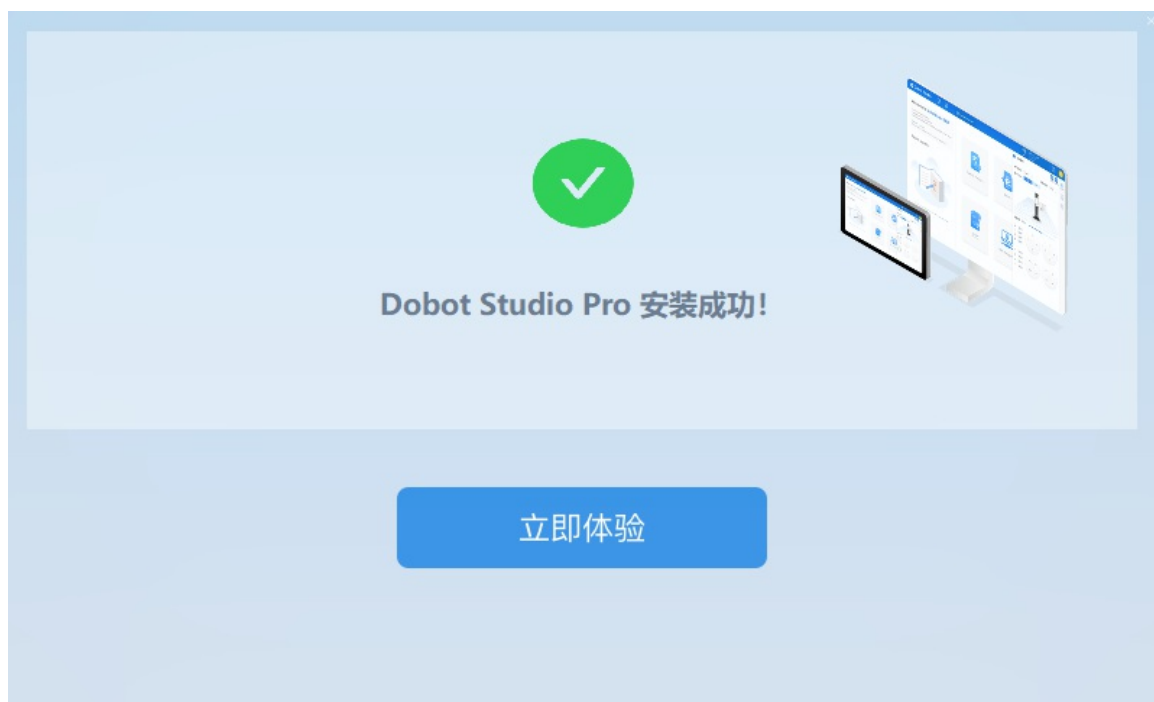
1. 双击打开安装包后，选择安装语言，并单击"下一步"。



2. 在安装界面中单击“一键安装”，或在自定义选项中设置安装路径后开始安装。



3. 软件安装成功后在提示界面单击“立即体验”可以直接打开软件。



手册导读

如果您是初次使用DobotStudio Pro，我们建议您按照如下顺序阅读本手册。

1. **连接机器人**：将DobotStudio Pro连接至机械臂。
2. **主界面说明**：了解DobotStudio Pro的主界面，大致了解DobotStudio Pro有哪些功能。
3. **设置**：根据实际需求对机械臂进行配置。
4. **监控**：了解DobotStudio Pro提供的监控等能力。

5. [编程](#)：了解DobotStudio Pro提供的编程能力，尝试创建自己的工程吧。
6. [远程控制](#)：工程开发完后，尝试通过远程控制来运行工程吧。

2 连接机器人

DobotStudio Pro支持有线（LAN）和无线（WiFi）两种连接方式。

说明

如果您的机器人控制器固件版本低于1.5.6.0，您需要在PC上打开SMB1协议才可正常连接机器人，详见本章的 [（可选）打开SMB协议](#)。

有线连接

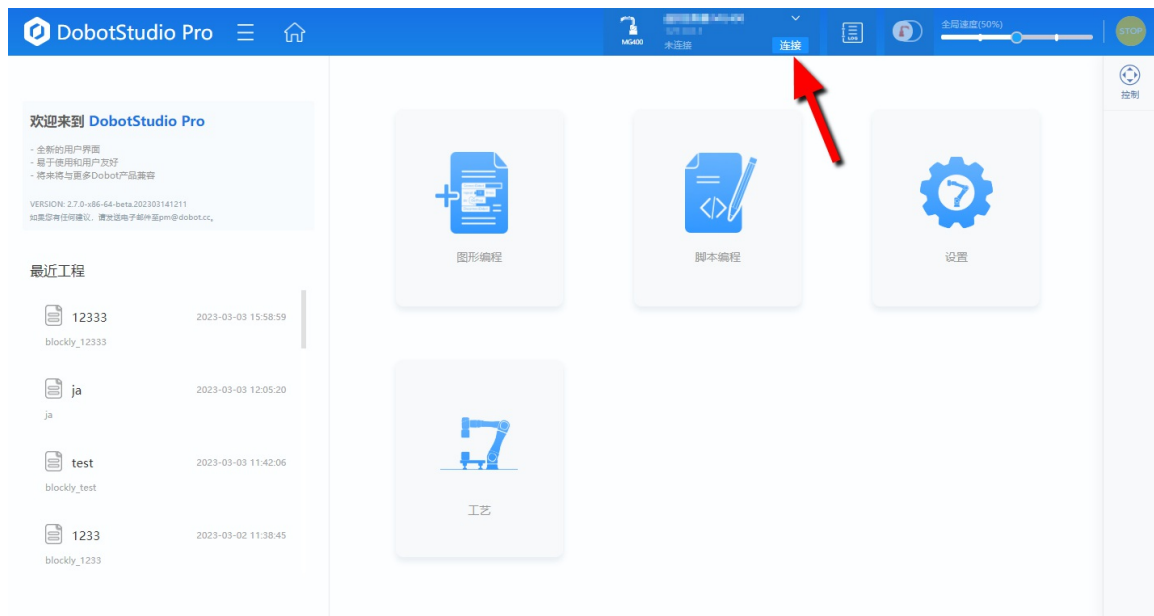
请将网线一端连接至控制柜的LAN接口，另一端连接至PC，然后修改PC的IP地址，使其与控制柜的IP地址在同一网段。控制柜LAN1的IP地址为192.168.1.6；LAN2的IP地址默认为192.168.2.6，可在[通讯设置](#)中修改。本文以使用LAN1为例。

不同的Windows版本中修改PC的IP地址的方法略有差异，本文以Windows 10为例进行介绍。

1. 在任务栏中搜索“网络”，选择“查看网络连接”。
2. 在当前网络连接（例如“以太网”）的图标上右键并选择“属性”，然后在弹出的窗口中找到并双击“Internet协议版本4（TCP/IPv4）”。
3. 在“Internet协议版本4（TCP/IPv4）属性”界面选择“使用下面的IP地址”，修改PC机的IP地址、子网掩码以及默认网关。可将PC机的IP地址修改为与控制柜同一网段未被占用的任意IP地址，其子网掩码和默认网关与控制柜的一致。如设置电脑IP地址为192.168.1.40，子网掩码为255.255.255.0。



4. 在DobotStudio Pro界面上方选择你要连接的机器人，然后单击“连接”。



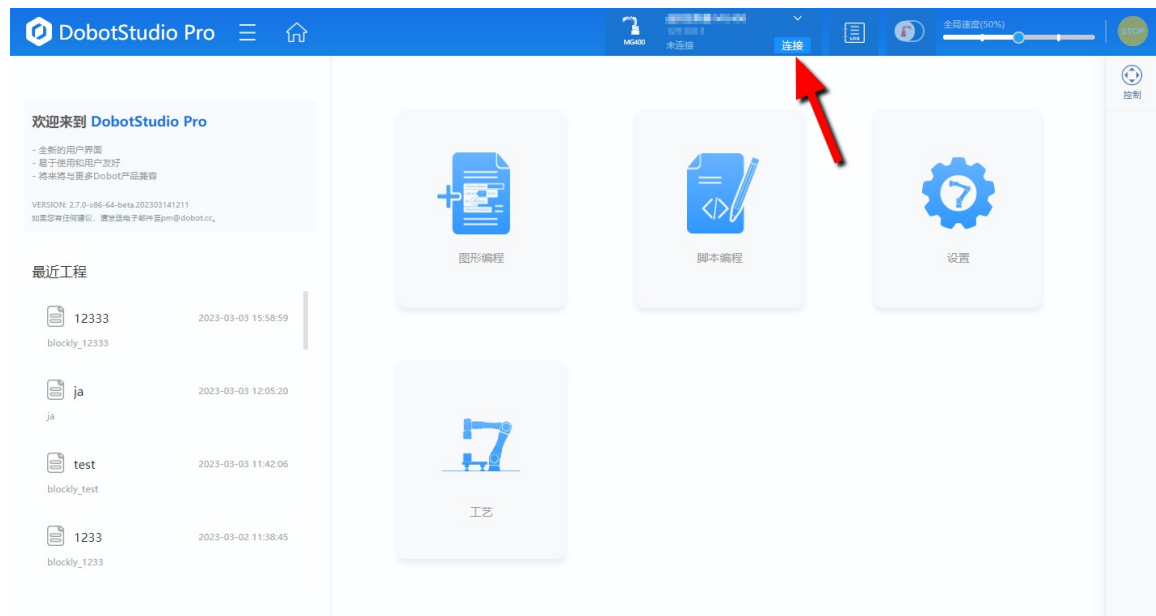
5. 连接成功后DobotStudio Pro会弹出提示信息，并且连接状态变为“已连接”。单击“已连接”可以断开和机器人的连接。



无线连接

在连接机器人前，需确保控制柜已安装WiFi模块。

1. 在PC上搜索并连接机器人的WiFi，WiFi的默认SSID为“MagicianPro”，初始WiFi密码为：1234567890。WiFi的SSID和密码可在[通讯设置](#)中修改。
2. 在DobotStudio Pro界面上方选择你要连接的机器人，然后单击“连接”。



3. 连接成功后DobotStudio Pro会弹出提示信息，并且连接状态变为“已连接”。单击“已连接”可以断开和机器人的连接。

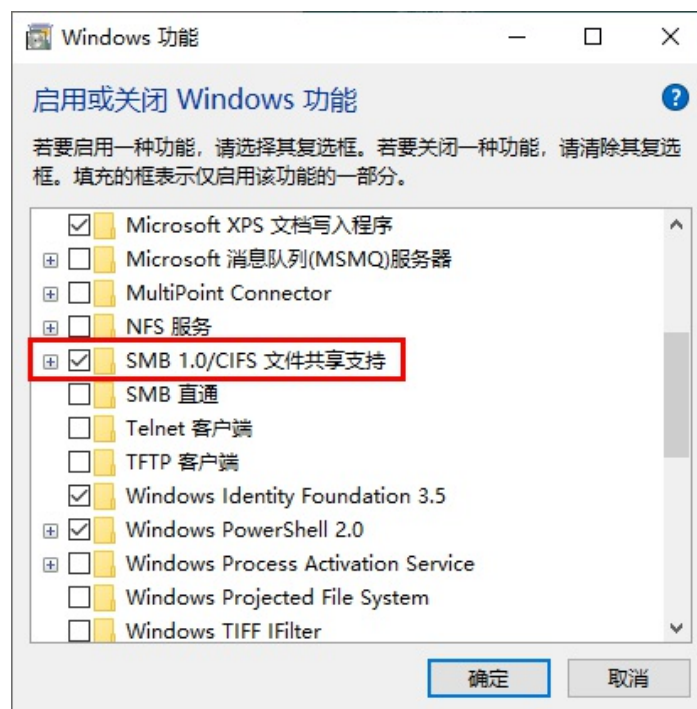


(可选) 打开SMB协议

1. 以Windows 10为例，在任务栏搜索“Windows 功能”，然后单击打开“启用或关闭Windows 功能”。



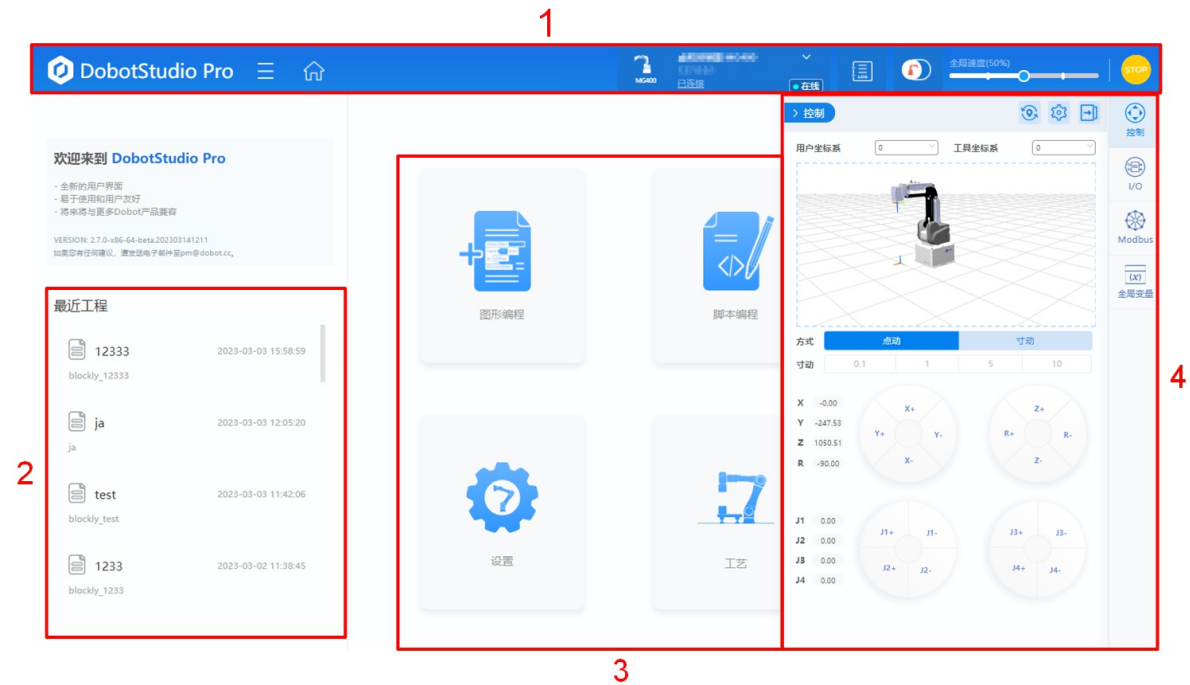
2. 勾选 “SMB 1.0/CIFS 文件共享支持” ，然后单击 “确定” 。



3 主界面说明

- [3.1 主界面概览](#)
- [3.2 顶部工具栏](#)
- [3.3 控制面板](#)

3.1 主界面概览



序号	说明
1	顶部工具栏
2	显示最新打开的工程，单击可快速打开工程。
3	主要功能的入口，包括图形编程，脚本编程，设置和工艺。工艺的使用指南请参考各工艺的独立说明文档，不在本手册中详述。
4	点击右侧工具栏上的按钮可显示或隐藏对应的工具面板，从上到下分别为控制面板，I/O监控面板，Modbus面板和全局变量面板。其中控制面板在连接成功后默认显示。

3.2 顶部工具栏



序号	说明
1	单击后弹出下述的菜单。 • 设置：单击打开设置界面。 • 语言：设置控制软件的显示语言。 • 帮助：提供包括帮助文档、调试工具在内的一系列帮助功能。 • 检查更新：查看软件版本信息和是否有新版本发布。 • 关于：查看软件的构成组件说明。
2	单击可从其他界面返回主界面。
3	机器人连接面板，详见 连接机器人 。
4	报警日志按钮，详见下文的 报警日志 。
5	使能按钮，详见下文的 使能状态说明 。
6	拖动蓝点或单击速度条的任意位置调整全局速率。全局速率为机械臂实际运行速度的计算因子，速度计算方法详见 运动参数设置 。
7	急停按钮，机械臂运行过程中出现突发情况时按下该按钮，机械臂紧急停止并断电，详见下文的 急停按钮说明 。

报警日志

当点动或存点的方法不正确或机械臂使用不当时，例如使机械臂限位或处于奇异点，会触发报警。

如果机械臂运行过程中触发报警，DobotStudio Pro界面的报警图标会变为，此时单击报警图标，打开报警界面查看报警信息。



此时用户可单击报警信息查看报警原因及解决方案，并单击“清除报警”，如下所示。



使能状态说明

机械臂需要进入使能状态后才可以工作。

- 使能按钮在左侧，机械臂图标为红色时，表示机械臂处于下使能状态。单击按钮会弹出负载设置窗口（末端负载的偏心坐标需在J4轴为0°时设置，负载重量（含末端治具重量）不得超过机械臂允许的最大负载重量），确认后机械臂会开始上使能。上使能后，使能按钮移到右侧，图标也会变为绿色.



注意：

负载设置不正确可能导致出现碰撞检测异常报警或者拖拽时本体不受控制。

设置负载参数

为了保证机械臂的平稳运行以及避免出现碰撞检测误触发现象，需在J4轴角度为0°时设置末端负载的偏心坐标(x1,y1)



末端负载	0	g
x方向偏心坐标	0	mm
y方向偏心坐标	0	mm

确定

- 使能按钮图标为绿色时，表示机械臂处于使能状态。单击按钮会弹出下使能的确认框，确认后机械臂会开始下使能。下使能后，使能按钮图标也会变为蓝色。
- 使能按钮为蓝色闪烁状态时，表示机械臂处于拖拽模式，此时不能下使能和通过控制软件控制机械臂运动（运行工程、点动、RunTo指定姿态等）。

急停按钮说明

急停按钮被按下后，机械臂会急停并断电，按钮图标变为红色。

如需重新使能机械臂，请先再次单击急停按钮将其复位，然后再上电和使能。

说明

如果是实体急停按钮被按下，软件上急停按钮的图标不会变，清除报警前需要先复位实体急停按钮（一般是通过顺时针旋转按钮复位）。

3.3 控制面板



序号	说明
1	单击可隐藏面板，隐藏后可通过单击右侧工具栏的“控制”按钮恢复显示。
2	长按可使机械臂运动至初始位姿。初始位姿支持 设置 。
3	单击可打开 设置页面 。
4	单击可折叠显示控制面板，折叠状态下再次单击可取消折叠。
5	在单击用户坐标系或工具坐标系右侧的下拉菜单可选择对应的坐标系序号（单击下

5	拉菜单中“...”可打开对应的坐标系设置页面）。
6	用户点动或运行机械臂时，可通过虚拟仿真界面实时查看机械臂运动状况。
7	选择机械臂运动方式。 • 点动：用户按住点动按钮时机械臂持续运动，松开时停止运动。 • 寸动：单击点动按钮时移动寸动的值，长按可持续运动。在笛卡尔坐标系下，寸动值的单位是mm，0.1表示每次单击位移0.1mm。在关节坐标系下，寸动值的单位是°，0.1表示每次单击位移0.1°。
8	点动操作板。上半部分为笛卡尔坐标系点动控制，下半部分为关节点动控制。 • 在笛卡尔坐标系下，以“X+”、“X-”为例，点击“X+”、“X-”按钮，控制机械臂沿X轴进行正向或负向移动，以此类推； • 在关节坐标系下，以“J1+”、“J1-”为例，点击“J1+”、“J1-”按钮，控制机械臂沿J1轴进行正向或负向旋转，以此类推。

4 设置

- 4.1 基础设置
- 4.2 通讯设置
- 4.3 坐标系管理
 - 4.3.1 用户坐标系
 - 4.3.2 工具坐标系
- 4.4 负载参数设置
- 4.5 运动参数设置
- 4.6 安全设置
- 4.7 远程控制
- 4.8 左右手标定 (M1 Pro)
- 4.9 固件更新
- 4.10 零点标定

4.1 基础设置

基础设置页面用于查看设备规格和设置机械臂姿态。

设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

固件更新

零点标定

☐ 下次软件启动时默认自动连接!

设备规格

设备名

MG400

重置设备名称

软件版本号:

ChokerStudio Pro 2.7.0-2024-04-16-10240000

控制器固件:

0.0.0

伺服

0.0.0

配置文件:

0.0.0

设备序列号

0.0.0

控制器硬件版本

0.0.0

伺服硬件版本

0.0.0

功率板硬件版本

0.0.0

初始姿态

重新设置初始点位

InitialPose

移动至初始点位

恢复为默认点位

X	350.000	Z	0.000	User	0
Y	0.000	R	0.000	Tool	0

TrueMotion功能

☐

- 勾选“下次软件启动时默认自动连接”，则下次软件启动时会尝试自动连接当前连接的机械臂。
- 修改设备名后单击“重置设备名称”可修改设备名。

初始姿态

初始姿态是一个可自定义的姿态，其默认姿态为零点位姿，即各个关节角度均为0。

初始姿态

重新设置初始点位

InitialPose

 移动至初始点位

 恢复为默认点位

X	350.000	Z	0.000	User	0
Y	0.000	R	0.000	Tool	0

单击“重新设置初始点位”可修改初始姿态。

初始姿态

重新设置初始点位

InitialPose

获取当前点位

X

350

Z

0

User

0

Y

0

R

0

Tool

0

取消

确定

您可以手动输入各个关节的角度，或者运动机械臂到指定姿态后单击“获取当前点位”，读取机械臂当前各个关节的角度。确定各关节角度后，单击“确定”即可更新初始姿态。

长按“移动至初始点位”可以让机械臂运动至初始姿态。单击“恢复为默认点位”可将初始姿态恢复为默认姿态。

TrueMotion功能

TrueMotion功能可以使复现轨迹更加精准，运动速度更加平稳，在对运动轨迹精确度和速度稳定性较高的场景下（如涂胶），可以开启该功能。

4.2 通讯设置

IP设置

MG400/M1 Pro有两个LAN接口，其中LAN2接口用于与外部设备通信，支持TCP、UDP或Modbus协议。用户可修改LAN2接口的IP地址，子网掩码和网关。连接外部设备时，IP地址必须与外部设备的IP地址在同一网段，且不冲突。LAN2接口的默认IP地址为192.168.2.6。

- 如果机械臂与外部设备直接连接或通过交换机连接，则需勾选“手动设置IP”，修改IP地址、子网掩码以及默认网关后单击“应用”。
- 如果机械臂与外部设备通过路由器，则勾选“自动获得IP”（由路由器自动分配IP地址），然后单击“应用”。

IP设置

 仅支持修改LAN2接口的IP地址，用于连接外部设备

☐ 自动获取IP ☒ 手动设置IP

IP地址 - - -

子网掩码 - - -

网关 - - -

WiFi设置

机械臂可通过WiFi与外部设备通信。用户可修改WiFi名称和密码（长度8~63位），重新启动设备后生效。WiFi初始密码为1234567890。

WIFI设置

 用于WIFI连接的上位机软件

SSID 0/20

密 码

4.3 坐标系管理

- 4.3.1 用户坐标系
- 4.3.2 工具坐标系

4.3.1 用户坐标系

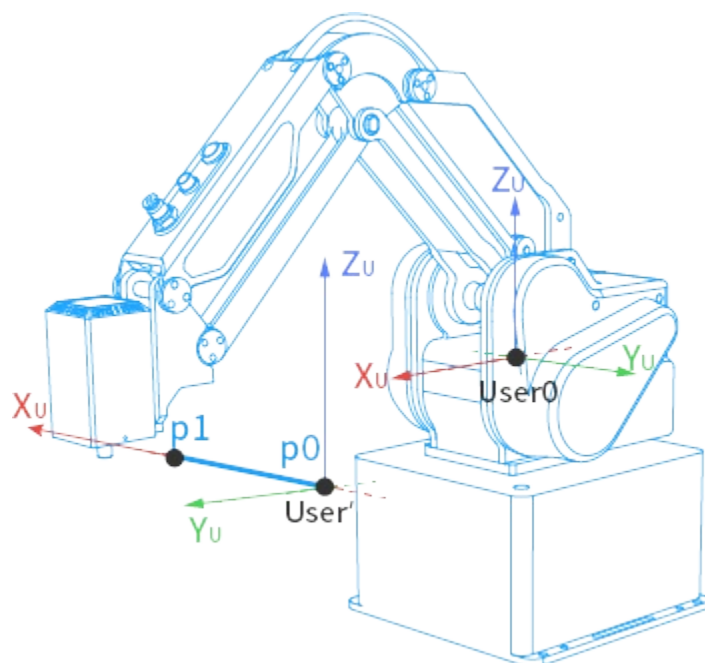
当工件的位置发生变化或机械臂的运行程序需要在多个同类型的加工系统中重复使用时，此时需要设置用户坐标系，使所有路径都跟随用户坐标同步更新，极大的简化了示教编程。

当前系统支持最多10个用户坐标系，用户坐标系**0**为基坐标系，不可更改。

说明

建立用户坐标系时，请确保参考坐标系为基坐标系。

用户坐标系采用两点示教法生成：将MG400移动至任意两点 $P0(x0,y0,z0)$ 和 $P1(x1,y1,z1)$ ，其中，将 $P0$ 点作为原点， $P0$ 与 $P1$ 两点之间的连线确定为用户坐标系X轴正方向，Z轴正方向默认垂直向上，然后根据右手定则确定Y轴方向，如下图所示：



创建用户坐标系

1. 单击“新增”，如下图所示。



2. 在新增用户坐标系页面中选择“两点设置”。

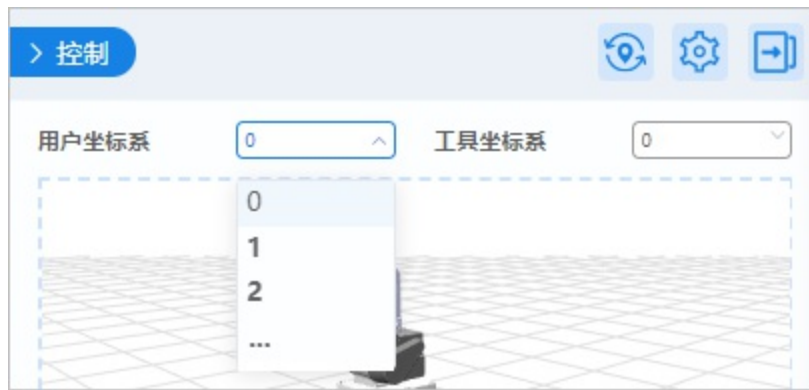


说明

- 建立用户坐标系时，请确保参考坐标系为基坐标系，即点动机械臂时用户坐标系为“0”。
- 长按“RunTo”可以让机械臂运动至已设置的点位。

- 点动机械臂移动至P1点，在P1坐标面板单击“获取坐标”。
- 点动机械臂移动至P2点，在P2坐标面板单击“获取坐标”。
- 单击“确定”，用户坐标系创建完成。

创建或修改用户坐标系后，可在控制面板选择用户坐标系进行点动，如下图所示。。



说明

创建或修改用户坐标系时，用户也可选择“输入设置”，直接修改X、Y、Z、R的值，并点击“确定”。

其他操作

- 勾选坐标系后单击“修改”，可以修改已创建的坐标系，修改方法与创建方法类似，此处不再赘述。
- 勾选坐标系后单击“复制”，可以创建一个与勾选坐标系相同的新坐标系。

4.3.2 工具坐标系

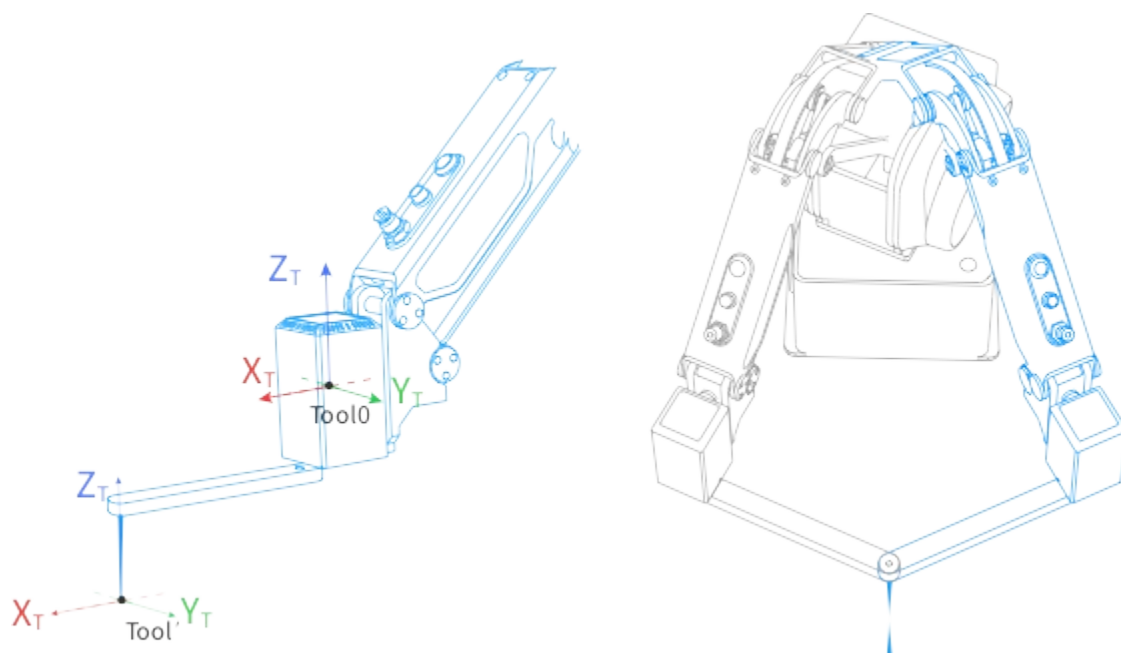
当机械臂末端安装了工具（如焊枪、喷嘴、夹具等）后，为了编程和机械臂运行的需要，此时需要设置工具坐标系。例如，利用多个夹具同时搬运多个工件，可将每个夹具设置为独立的工具坐标系以提高搬运效率。

当前系统支持最多10个工具坐标，工具坐标系0为默认工具坐标系，即不使用工具，不可更改。

说明

建立用户坐标系时，请确保参考坐标系为基坐标系。

工具坐标系采用两点示教法生成：MG400末端安装工具后，调整工具的位姿，使TCP（Tool Center Point）以两种不同的姿态对准空间中同一点（参考点），获取工具的位置偏移，生成工具坐标系，如下图所示：



创建工具坐标系

1. 在机械臂末端安装工具，本节不做详细说明。
2. 单击“新增”，如下图所示。

设置

通用

MG400

基础设置

通讯设置

坐标系管理

用户坐标系

工具坐标系

复制 修改 新增

index	别名	X	Y	Z	R
☐ 0		0.000	0.000	0.000	0.000
☐ 1		0.000	0.000	0.000	0.000
☐ 2		0.000	0.000	0.000	0.000

3. 在新增工具坐标系页面中选择“两点设置”。

设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

固件更新

零点标定

用户坐标系

工具坐标系

新增工具坐标系: index3 别名

☐ 输入设置
 ☒ 两点设置

MG400两点设置工具坐标系示意图

P1坐标

X 0

Y 0

Z 0

R 0

获取坐标

RunTo

P2坐标

X 0

Y 0

Z 0

R 0

获取坐标

RunTo

取消

确定

说明

- 建立工具坐标系时，请确保参考坐标系为基坐标系，即点动机械臂时工具坐标系为“0”
- 长按“RunTo”可以让机械臂运动至已设置的点位。

4. 点动机械臂，使TCP以第一种姿态对准参考点（空间中的一点），在P1坐标面板单击“获取坐标”。
5. 点动机械臂，使TCP以第二种姿态对准参考点，在P2坐标面板单击“获取坐标”。
6. 单击“确定”，工具坐标系创建完成。

创建或修改工具坐标系后，可在控制面板选择工具坐标系进行点动，如下图所示。。



说明

新增或修改工具坐标系时，用户也可选择“输入设置”，直接修改X、Y、Z、R的值，并点击“确定”。

其他操作

- 勾选坐标系后单击“修改”，可以修改已创建的坐标系，修改方法与创建方法类似，此处不再赘述。
- 勾选坐标系后单击“复制”，可以创建一个与勾选坐标系相同的新坐标系。

4.4 负载参数设置

为充分发挥机械臂自身具备的性能，需将末端负载质量与偏心坐标设定在允许的范围内，确保机器运行时J4轴不会产生偏心。通过合理的设定，可优化机械臂的运动，抑制振动，缩短作业时间。

说明

- 机械臂每次软件启动后第一次上使能时都会弹窗要求设置负载参数，设置的参数也会同步到此界面。
- 伺服参数设置为高级功能，请在技术支持的指导下使用。



单击“修改”按钮可修改负载参数。

- 末端负载的偏心坐标需在J4轴为0°时设置。
- 末端负载重量即夹具末端重量与工件重量之和，不能超过机械臂允许的最大负载。

注意：

负载设置不正确可能导致出现碰撞检测异常报警或者拖拽时本体不受控制。

修改设置后，单击“确定”按钮才会生效。

请务必设定合适的负载质量与偏心坐标，否则可能会导致发生错误或冲击，缩短机器使用寿命。

4.5 运动参数设置

机械臂出厂的时候已经设置了最优的运动参数，如无特殊需求，不建议修改。如果工况要求更高的运动速度，建议小幅度更改，否则过高的运动速度可能会损害关节寿命并造成安全隐患。

点动设置

请根据实际需要，设置点动时关节或坐标系下各轴的最大速度、最大加速度，设置完成后请单击“保存”。

设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

固件更新

零点标定

点动设置

再现设置

Jump设置

关节速度

J1 12.000 °/s J2 12.000 °/s

J3 12.000 °/s J4 12.000 °/s

坐标系速度

X 50.000 mm/s Z 50.000 mm/s

Y 50.000 mm/s R 12.000 °/s

关节加速度

J1 100.000 °/s² J2 100.000 °/s²

J3 100.000 °/s² J4 100.000 °/s²

坐标系加速度

X 300.00 mm/s² Z 300.00 mm/s²

Y 300.00 mm/s² R 100.000 °/s²

取消

保存

机器人实际运动速度/加速度 = 此处设置的速度/加速度 x 全局速率。

例：关节速度为12°/s，全局速率为50%，则实际点动速度为12°/s x 50% = 6°/s。

单击 会将对应板块的设置全部恢复为默认值。

再现设置

请根据实际需要，设置关节或坐标系下各轴的运动速度、加速度、加加速度，设置完成后请单击“保存”。



设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

固件更新

零点标定

点动设置

再现设置

Jump设置

关节

坐标系

关节速度

J1

180.000 °/s

J2

180.000 °/s

J3

180.000 °/s

J4

180.000 °/s

关节加速度

J1

200.000 °/s²

J2

200.000 °/s²

J3

200.000 °/s²

J4

500.000 °/s²

关节加加速度

J1

2000.000 °/s³

J2

2000.000 °/s³

J3

2000.000 °/s³

J4

5000.000 °/s³

取消

保存



设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

固件更新

零点标定

点动设置

再现设置

Jump设置

关节

坐标系

坐标系速度

XYZ

2000.000 mm/s

R

180.000 °/s

坐标系加速度

XYZ

10000.000 mm/s²

R

900.000 °/s²

坐标系加加速度

XYZ

18000.000 mm/s³

R

9000.000 °/s³

取消

保存

机器人实际运动速度/加速度 = 此处设置的速度/加速度 × 全局速率 × 编程时速度指令设置的百分比。

例：坐标系速度为2000mm/s，全局速率为50%，编程时运动指令设置的速率为80%，则实际运动速度为2000mm/s x 50% x 80%= 800mm/s。

单击 会将对应版块的设置全部恢复为默认值。

Jump设置

再现时，若机械运动模式为“Jump”，需设置起始点抬升高度、结束点下降高度以及抬升最大高度。最大支持设置10组Jump参数，选中后单击修改（或双击选中）可以修改Jump参数。

设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

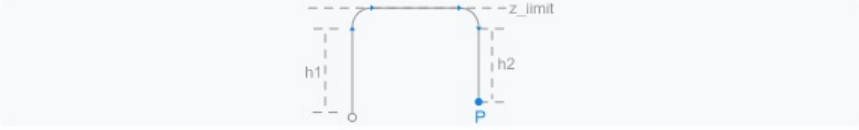
固件更新

零点标定

点动设置

再现设置

Jump设置



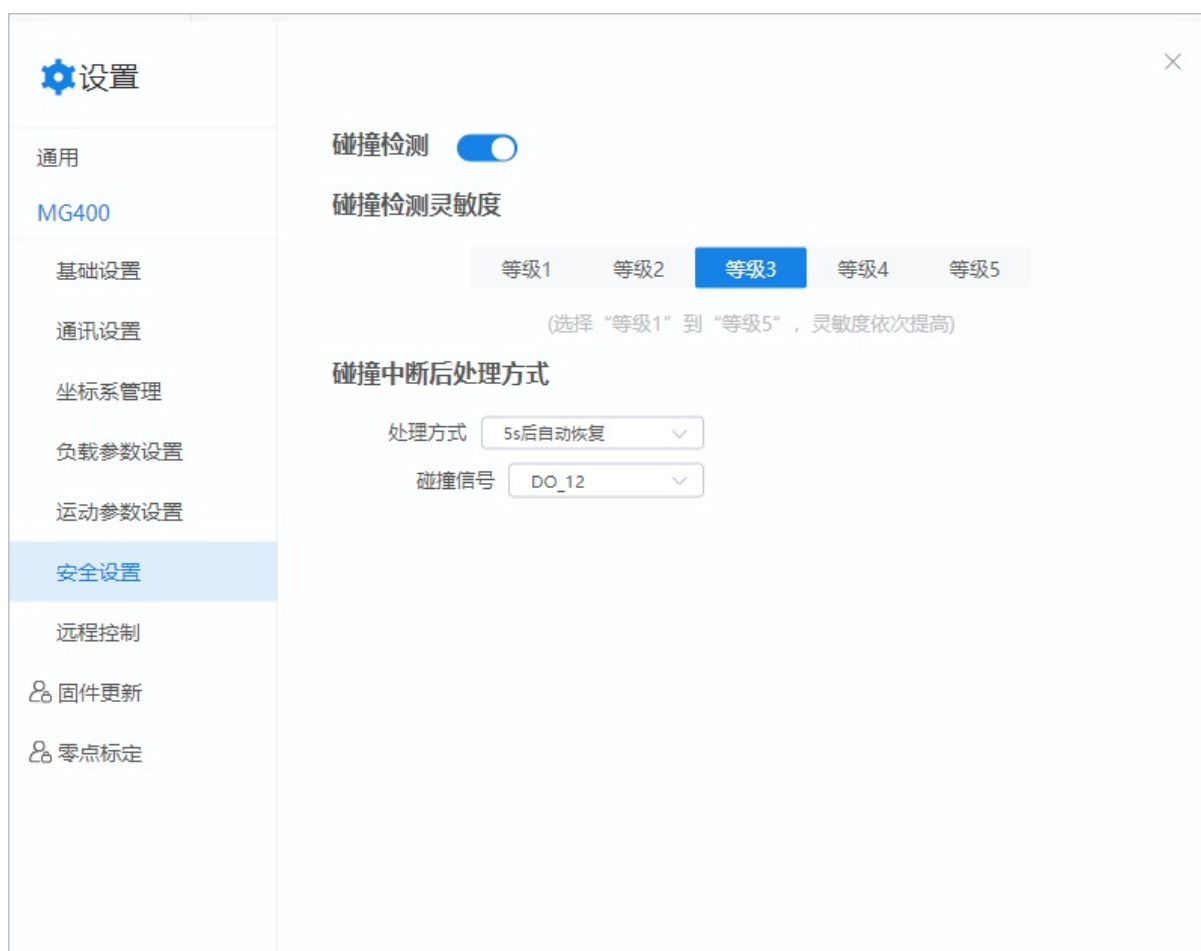
序号	抬升高度h1(mm)	下降高度h2(mm)	最大高度zLimit(mm)
0	100	100	680
1	100	100	680
2	100	100	680
3	100	100	680
4	100	100	680
5	100	100	680
6	100	100	680
7	100	100	680
8	100	100	680
9	100	100	680

修改

用户在编程中调用Jump指令时可以利用Arch参数指定索引序号，调用Jump参数。

4.6 安全设置

碰撞安全功能主要用于减少碰撞力对机器人本体的影响，避免机器人本体或者外围设备损坏。开启碰撞安全功能后，当机械臂碰撞到工件或其他障碍物时，机械臂会自动停止运行，以防机器或操作人员碰撞受伤。



如果配置了“碰撞信号”，则机械臂因碰撞中断后会触发对应的DO端口。已在[远程IO](#)中配置的DO不可以再被配置为碰撞信号。

开启了碰撞检测功能后，顶部工具栏机器人连接面板上会显示当前的安全等级。



DobotStudio Pro支持三种再现时碰撞中断后的处理方式。

- 5s后自动恢复：检测到碰撞后弹窗提示，机械臂暂停运行；5秒后弹窗自动消失，机械臂恢复运行。

- 暂停：碰撞中断后弹窗提示，机械臂暂停运行，需要通过软件界面或远程IO继续信号恢复运行或停止工程。



选择处理方式为暂停后，可配置继续信号，默认与[远程IO](#)中的继续信号相同，修改后会自动同步至远程IO配置，反之亦然。



- 停止：检测到碰撞后弹窗提示，机械臂停止运行。此时您需要解决碰撞的原因，并单击“复位”。如果解决碰撞原因需要操作控制软件，可单击“一分钟后提醒我”暂时关闭弹窗（一分钟后会再次弹窗提示）。



4.7 远程控制

外部设备可以通过不同的远程控制模式如远程I/O模式、远程Modbus模式下发指令控制机械臂（即对示教好的程序文件进行控制运行）。

说明

- 远程控制模式切换不需重新启动机器人控制系统。
- 无论机器人控制系统处于哪种模式，急停开关始终有效。
- 若机械臂在远程控制模式下运行，此时切换为其他工作模式工程会自动停止。
- 进入远程控制模式后，顶部工具栏会显示当前模式，点击右侧的“运行日志”可查看远程模式的运行日志，下图以远程I/O模式为例。

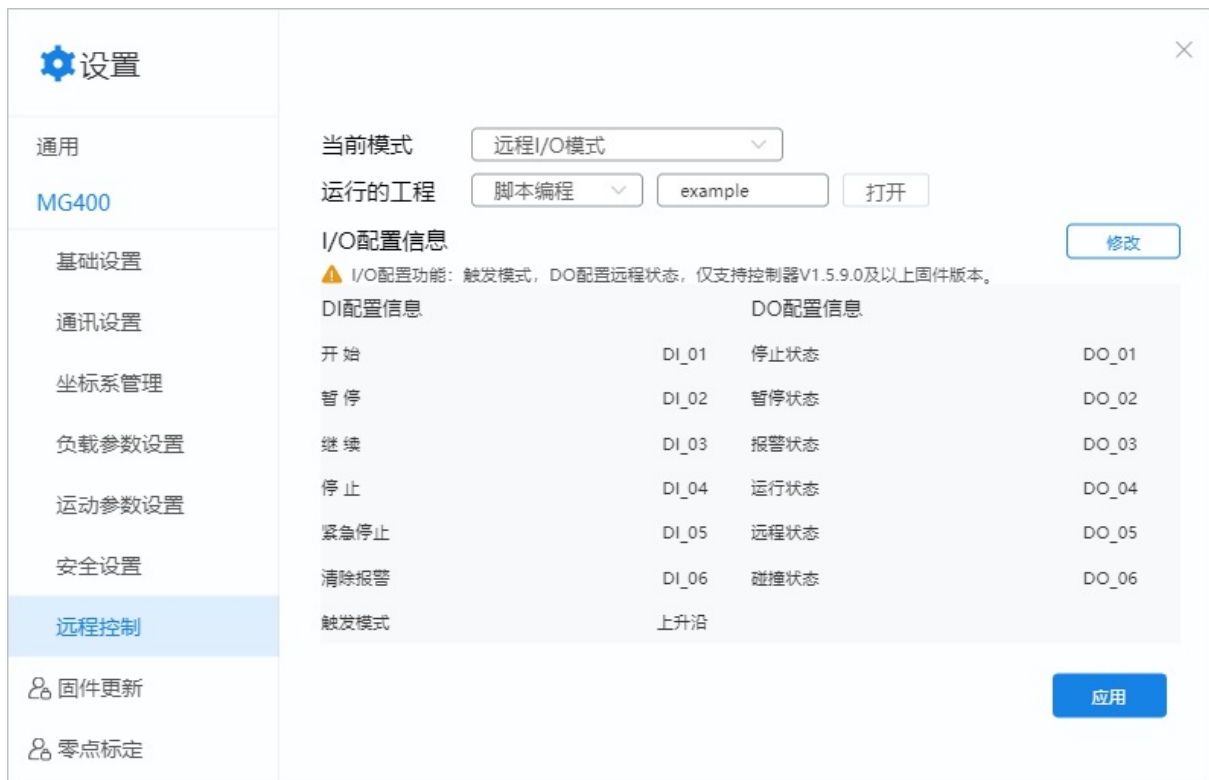


在线模式

默认的控制模式，使用DobotStudio Pro对机械臂进行控制。

远程IO

当远程控制模式为远程I/O时，外部设备可通过远程I/O来控制机械臂。



控制系统特定的I/O接口定义如上图所示，可单击“修改”进行编辑，修改IO的分配和触发模式（上升沿/下降沿）。

通过远程IO模式运行工程的步骤如下。

前提条件

- 已准备好远程运行的工程文件，且能正确运行。
- 已通过数字I/O接口将外部设备与机械臂连接。
- 机械臂已上电。

操作步骤

1. 选择“当前模式”为“远程I/O模式”，并选择待远程运行的工程，支持图形编程或脚本编程的工程。
2. 单击“应用”按钮，此时机械臂进入远程I/O控制模式，不再接受控制软件急停以外的指令。
3. 在外部设备上触发启动信号，此时机械臂会按照选择的工程文件进行运行。
4. 若触发停止信号，则停止运动。

远程Modbus

当远程控制模式为远程Modbus时，外部设备可通过远程Modbus来控制机械臂。



Modbus寄存器特定功能如上图所示。

通过远程Modbus模式运行工程的步骤如下。

前提条件

- 已准备好远程运行的工程文件，且能正确运行。
- 已通过LAN2接口将外部设备与机械臂连接。可直接连接或通过路由器连接，请根据实际情况选择。其中，机械臂和外部设备的IP地址需在同一网段。LAN2接口的默认IP地址为192.168.2.6，可在[通讯设置](#)中配置。
- 机械臂已上电。

操作步骤

1. 选择“当前模式”为“远程modbus模式”，并选择待远程运行的工程，支持图形编程或脚本编程的工程。
2. 如果需要通过Modbus启动多个（最大99个）不同的工程，可单击“高级设置”。在高级设置中，可以设置备选工程的保存寄存器地址，以及配置备选工程列表，如下图所示。

高级设置

保存寄存器配置 ! 用于远程Modbus模式可设置多个工程启动选项!

保持寄存器地址: 3095 备选工程: — +

寄存器数值	备选工程
1	图形编程 打开
2	图形编程 打开
3	图形编程 打开

取消 保存

- 单击“应用”按钮，此时机械臂进入远程Modbus控制模式，不再接受控制软件急停以外的指令。
- 在外部设备上触发启动信号，此时机械臂会按照选择的工程文件进行运行。
- 若触发停止信号，则停止运动。

TCP/IP二次开发

此模式仅用于客户基于TCP自行开发控制软件的场合。如果您需要自行开发控制软件，请参考<https://github.com/Dobot-Arm/TCP-IP-Protocol>。

4.8 左右手标定 (M1 Pro)

在M1 Pro使用过程中，如对机械臂精度要求较高，需进行左右手标定，以提高绝对定位精度。

标定时机械臂以左右手方向切换至同一点，切换前后J2轴关节坐标为轴对称。若切换前后J2轴关节坐标变为非对称，则导致机械臂无法以左右手方向到达同一点，绝对定位精度降低。此时需通过补充J2轴关节角度使左右手切换前后关节坐标变为对称，以提高绝对定位精度。

说明

左右手标定功能需要输入管理员密码 (默认密码：888888)后才可使用。

设置

通用

M1Pro

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

左右手标定

固件更新

零点标定

左右手标定 ! 左右手标定功能需要处于使能状态!



图1 左手姿态



图2 右手姿态

P10.0000获取 P1

第一步，如图1所示，使用左手姿态，移动机械臂至固定位置。得到P1点；

P20.0000获取 P2

第二步，如图2所示，使用右手姿态，移动机械臂至固定位置。得到P2点；

标定

1. 运动机械臂，使机械臂以左手方向移动至固定位置，单击“获取P1”。
2. 运动机械臂，使机械臂以右手方向移动至上一步中相同的位置，单击“获取P2”。
3. 单击“标定”。

4.9 固件更新

当控制器固件需要更新时，可在此页面导入最新的控制器固件进行更新。



危险

在升级过程中，请不要对机械臂进行任何其他操作或下电，避免机械臂处于异常状态。否则，很容易对设备或人体造成伤害。

⚙️ 设置

通用

MG400

基础设置

通讯设置

坐标系管理

负载参数设置

运动参数设置

安全设置

远程控制

🔧 固件更新

📍 零点标定

控制器固件

当前版本: 0

控制器固件:

打开

上传并更新

伺服固件

! 伺服固件更新需在高级用户权限下使用。

单击“打开”导入最新的控制固件，并单击“上传并更新”，固件将会自动更新，固件更新完后，会弹窗提示要重启控制器，确定后控制器会重启，请等待重启完成后软件自动连接。

DobotStudio Pro也支持伺服固件更新功能，请在技术支持的指导下使用。

4.10 零点标定

当更换机械臂的电机、减速机等传动部件或者与工件发生碰撞等情况下，机械臂的零点位置发生变化，此时需对机械臂进行零点标定。

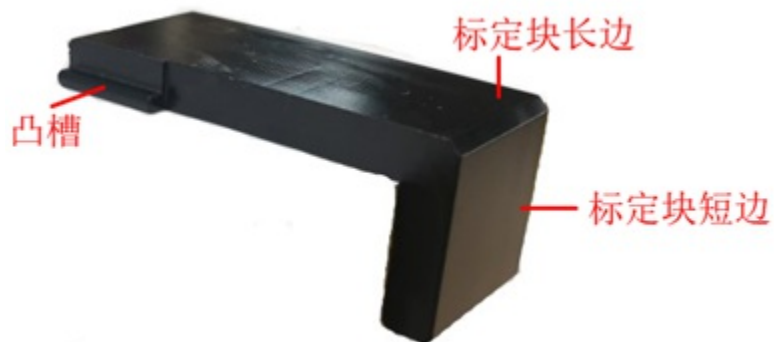
说明

- 零点标定仅在零点位置发生改变时使用，请谨慎操作。
- 零点标定功能需要输入管理员密码 (默认密码：888888)后才可使用。



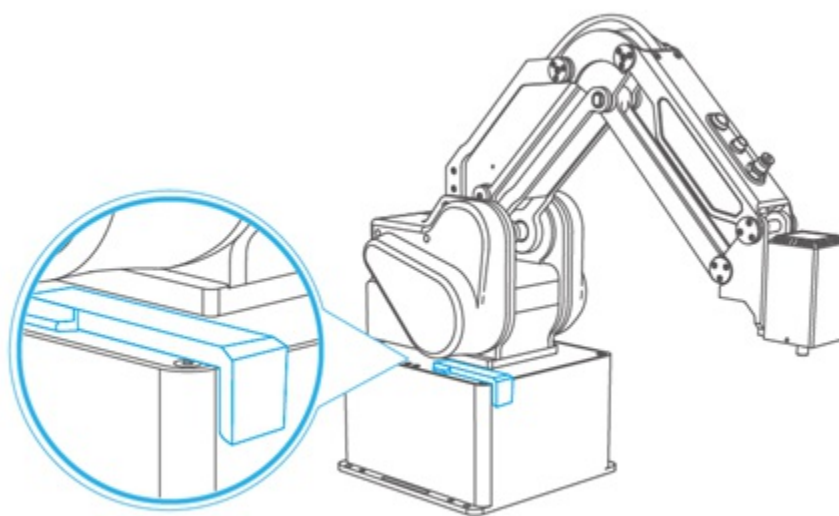
用户可以分别对各个轴进行零点标定，也可以通过右上角的“零点标定”按钮对机械臂整体进行零点标定。

进行零点标定时，可以利用如下图所示的标定块。

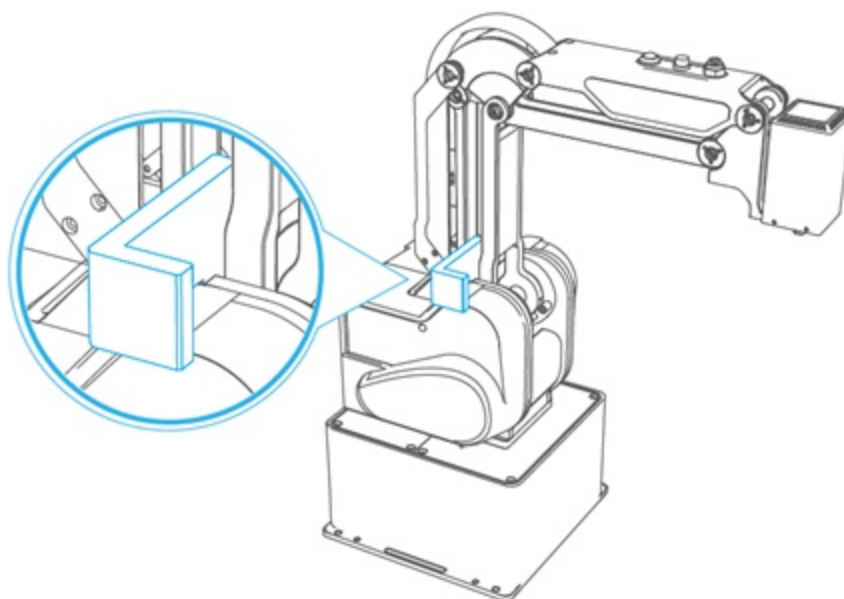


下面以整体零点标定为例介绍如何进行零点标定。

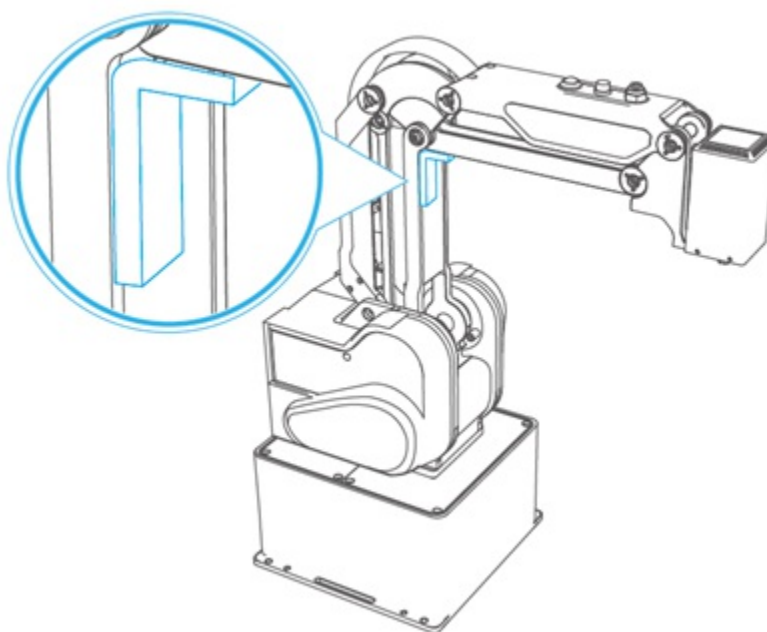
1. 将标定块放在下图中的位置并靠近转动盘，旋转J1轴，使转动盘与标定块平行并贴紧。



2. 将标定块底部的凸槽卡在下图中的缺口处，并使标定块的短边朝向大臂；按下拖拽示教按钮，拖动J2轴和J3轴，使大臂与标定块平行并贴紧，同时使大臂与小臂的夹角大于90°。



3. 将标定块放在下图中的位置，即大臂与小臂的夹角位置，使标定块的长边与大臂平行并贴紧；通过在控制面板上点动J3轴，使小臂与标定块的短边平行并贴紧。

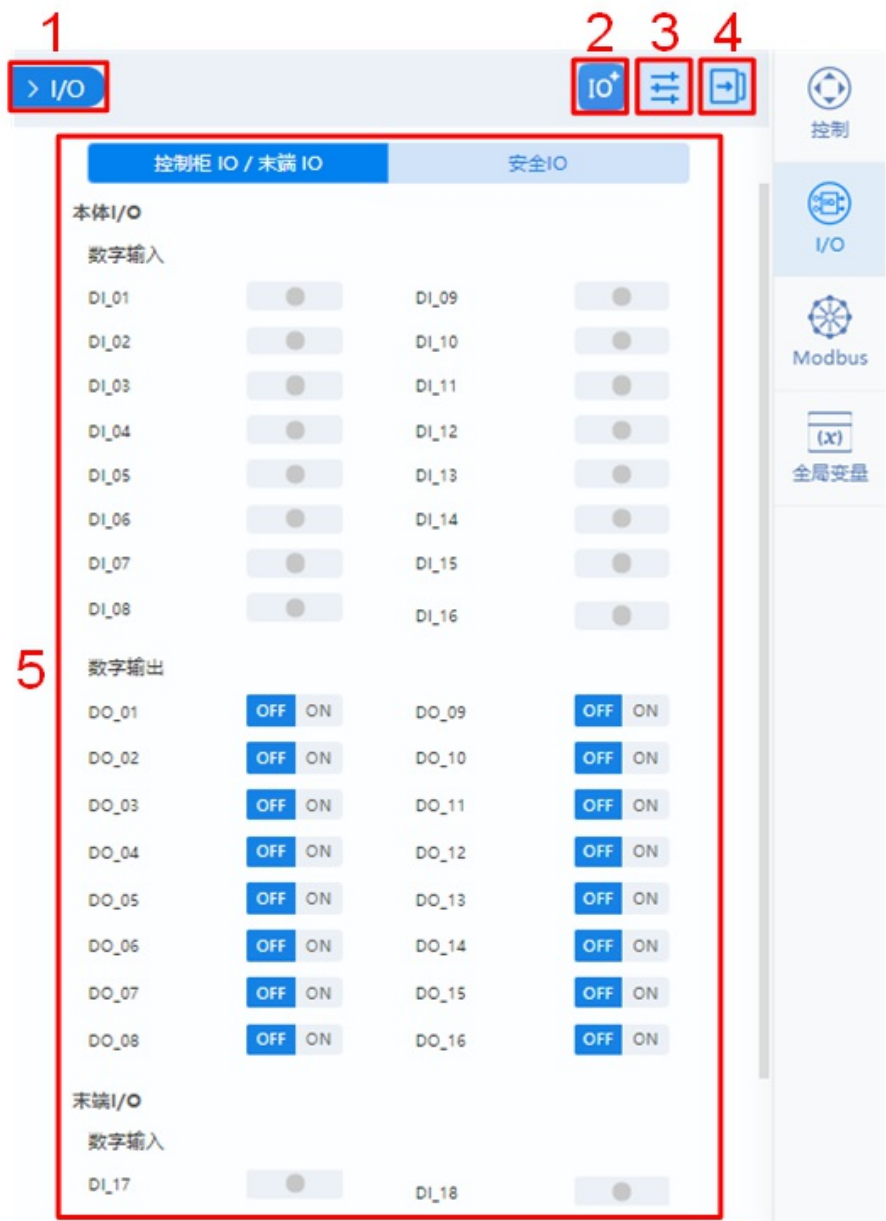


4. 单击“零点标定”按钮，确认进行标定。

标定成功后，点动板上各个关节的角度都应当显示为0。

5 I/O监控

I/O监控页面可以监控和设置控制柜和本体末端各个I/O状态。各个IO的具体定义请参考对应型号机械臂硬件手册中的IO接口说明，不同控制柜的IO数量不同，本文中的截图仅供参考。



序号	说明
1	单击可隐藏面板，隐藏后可通过单击右侧工具栏的“I/O”按钮恢复显示。
2	单击可添加拓展I/O，可用于监控Modbus通信的IO。详见下文的 I/O拓展 。
3	单击可设置I/O的别名以及是否显示等。详见下文的 I/O配置 。

4	单击可折叠显示控制面板，折叠状态下再次单击可取消折叠。
5	IO监控区域。详见下文的 监控区域 。

I/O拓展



I/O 扩展

ID: 2 名称: Modbus_IO2

IP地址: 127.0.0.1 端口号: 502

☒ DI: 地址(0~1000) 0

数量(1~96) 1

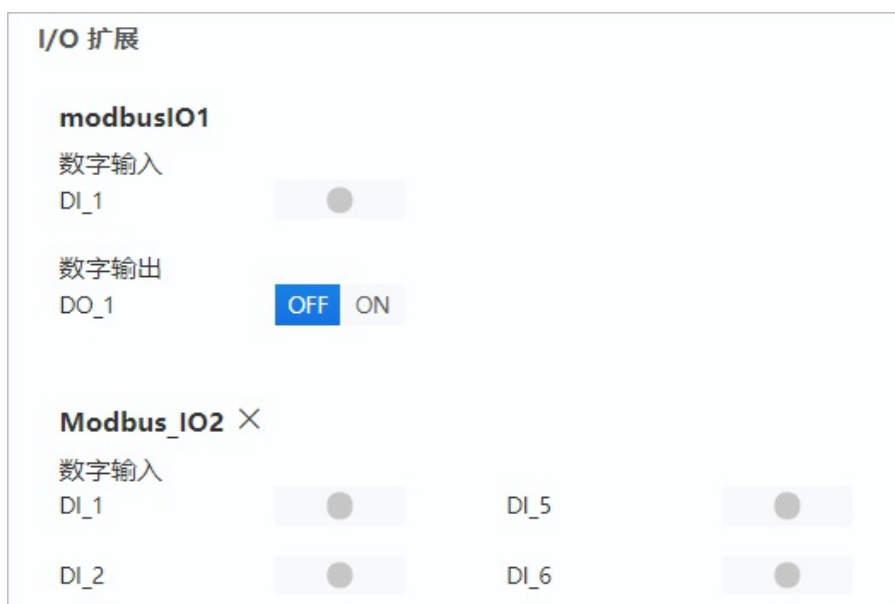
☒ DO: 地址(0~1000) 0

数量(1~96) 1

取消 确定

- ID: 从站设备的ID。
- 名称: 拓展I/O的名称。
- IP地址: Modbus设备的地址。不可与控制柜LAN口的IP（例如默认的192.168.1.6）相同，否则可能导致功能异常。
- 端口号: Modbus通讯的端口号。
- DI/DO: 勾选后配置DI/DO的寄存器地址和数量。

单击“确定”后会在监控页面的最下方出现新的I/O，但监控功能需要重启控制柜后才会生效。



I/O 扩展

modbusIO1

数字输入
DI_1 ☐

数字输出
DO_1 OFF ON

Modbus_IO2 ×

数字输入

DI_1 ☐ DI_5 ☐

DI_2 ☐ DI_6 ☐

单击对应IO右侧的X可以删除该拓展I/O。

I/O配置

> I/O

取消 保存

! 重命名I/O, 仅支持字母、数字、_和-, 不能超过30个字符。
! 勾选I/O, 使其在I/O页面展示

本体I/O

24V 1 2 3 4 5 6 7 8 24V

DO

DI

GND 1 2 3 4 5 6 7 8 GND

24V 9 10 11 12 13 14 15 16 24V

DO

DI

GND 9 10 11 12 13 14 15 16 GND

数字输入

☒	DI_01		☒	DI_09	
☒	DI_02		☒	DI_10	
☒	DI_03		☒	DI_11	

- 勾选IO，使其在监控页面展示。
- 在IO右侧可以输入该IO的别名，别名会在监控页面显示，同时用户也可在图形编程和脚本编程中通过别名调用对应IO。

监控区域

控制柜IO和末端IO页签支持以下功能。

- 输出：设置数字输出的值。直接单击右侧对应的开关状态即可切换。
- 监控：查看输入、输出的真实状态。数字输入右侧的圆点表示对应DI的状态，灰色表示DI未触发，绿色表示DI已触发。
- 模拟：模拟数字输入状态，方便调试运行程序。单击对应DI的状态显示区域会弹出设置窗口，选择“虚拟”并勾选“DI转换”，该DI会变为虚拟触发状态（蓝色圆点），逻辑上视为ON。若不勾选“DI转换”。则该DI保持真实状态。

索引: 1

真实

虚拟

☒ DI转换

确定

安全页面页签用于设置用户安全功能对应的DI，单击“修改”可进行修改，编辑完成后需要保存。请根据 实际需求进行配置。

> I/O

IO⁺

控制柜 IO / 末端 IO

安全IO

数字输入

修改

用户急停1

不做配置

用户急停2

不做配置

防护停止1

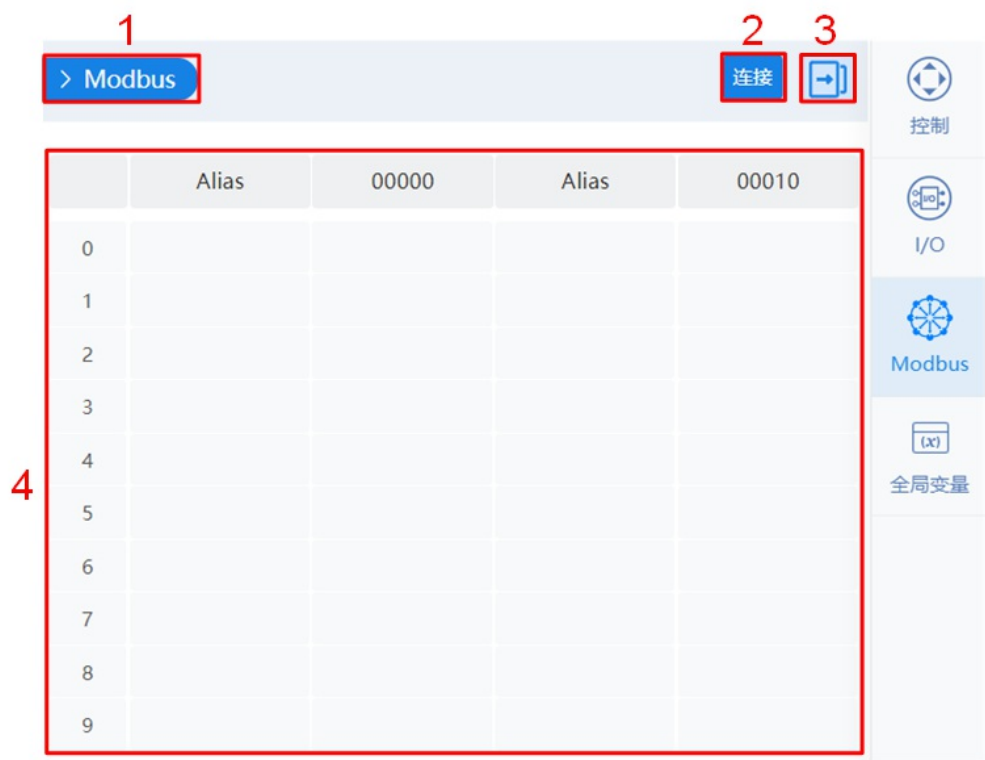
不做配置

防护停止2

不做配置

6 Modbus

Modbus功能用于作为Modbus主站连接Modbus从站。



序号	说明
1	单击可隐藏面板，隐藏后可通过单击右侧工具栏的“Modbus”按钮恢复显示。
2	单击可连接Modbus从站。详见下文的 连接从站 。
3	单击可折叠显示控制面板，折叠状态下再次单击可取消折叠。
4	显示已连接的从站的寄存器信息。

连接从站

设置

×

连接设置

从站IP:127.0.0.1

端口:502

功能码定义

从站ID:1

功能码:01: 线圈寄存器

地址:0

数量:10

扫描频率:1000ms

取消

确定

- 从站IP：Modbus设备的地址。连接机械臂自带的Modbus从站时输入控制柜的IP地址，例如192.168.5.1。
- 端口：Modbus通讯的端口号。连接机械臂自带的Modbus从站时输入502。
- 从站ID：从站设备的ID。
- 功能码：选择从站设备的功能类型。
- 地址/数量：寄存器的地址和数量。连接机械臂自带的Modbus从站时，请参考[附录A Modbus寄存器定义](#)。
- 扫描频率：机械臂扫描从站的时间间隔。

7 全局变量

该界面用于配置与查看全局变量。

设置全局变量后，在图形编程中可使用全局变量相关的积木调用，在脚本编程中可直接通过变量名进行调用。



支持以下类型的全局变量：

- bool：布尔值
- String：字符串
- int：整型数
- float：双精度浮点数
- point：机械臂点位，可通过将机械臂运动到指定点位后获取，如下图所示。

添加变量

变量名

var_1

变量类型

int

值

☒ 全局保持

取消

添加

全局保持：

- 勾选该选项时，该变量的任何修改都可以保存下来，退出脚本或者断电重启都会保存修改后的值。
- 不勾选时，对此变量的修改仅在脚本运行时有效，退出脚本运行则恢复为初始设置值。

8 编程

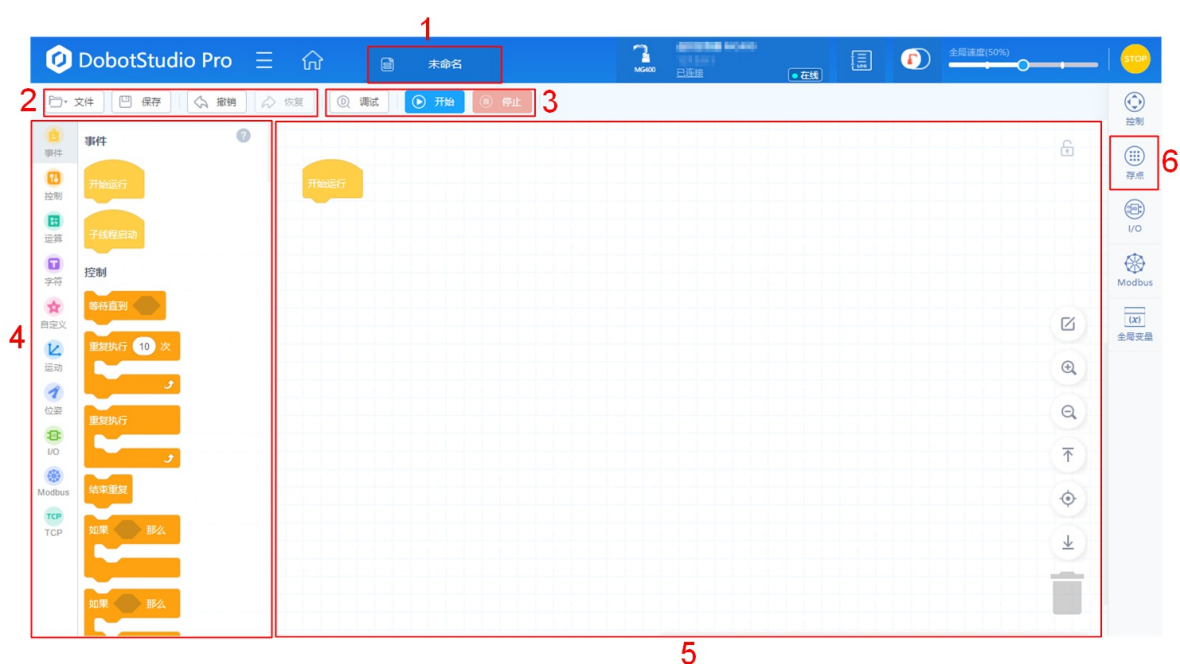
- 8.1 图形编程
- 8.2 脚本编程

8.1 图形编程

DobotStudio Pro提供图形化编程的能力，用户可通过拖拽积木块的方式进行编程，从而控制机械臂的运行，直观易懂。

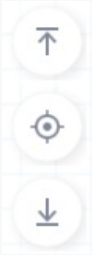
说明

本章主要介绍如何使用图形编程界面，关于积木的具体说明请参考[附录B](#)。



序号	说明
1	显示当前打开的工程名称。
2	用于管理工程文件和对编程操作进行撤销或恢复。 在“文件”下拉菜单中支持将图形编程的工程转换为脚本编程的工程，转换成功后可在脚本编程界面打开转换后的工程。
3	用于控制当前工程的调试、运行、暂停和停止。详见 调试与运行工程 。
4	提供编程所需的积木，可以按照分类及颜色查找所需的积木。单击模块右上角的“?”可查看积木说明文档。
5	程序的编写区域，可以将积木拖放至该区域来编写程序。 在已放进该区域的积木上右键可打开菜单，支持复制积木、删除积木以及将不包含事件积木的一组积木变为子程序。 已修改且未保存的积木左端会出现  图标，提示用户该积木已修改。
6	单击可打开存点界面，如果有未保存的修改时图标右下角会显示红点。详见 存点界面 。

编程区右侧的图标说明如下。

图标	说明
	用于锁定/解锁编程区域。
	用于进入编辑状态。 编辑状态下，用户可多选或全选积木进行复制或删除。 单击“取消编辑”或者在编程区进行其他操作都会退出编辑状态。 
	分别用于放大/缩小编程区域。
	分别用于回到积木的顶部/居中显示积木/回到积木的底部。
	将编程区中的积木拖到此处进行删除。用户也可右键积木选择删除。

存点界面

存点界面用于管理编程过程中要用到的机械臂点位，如下图所示。



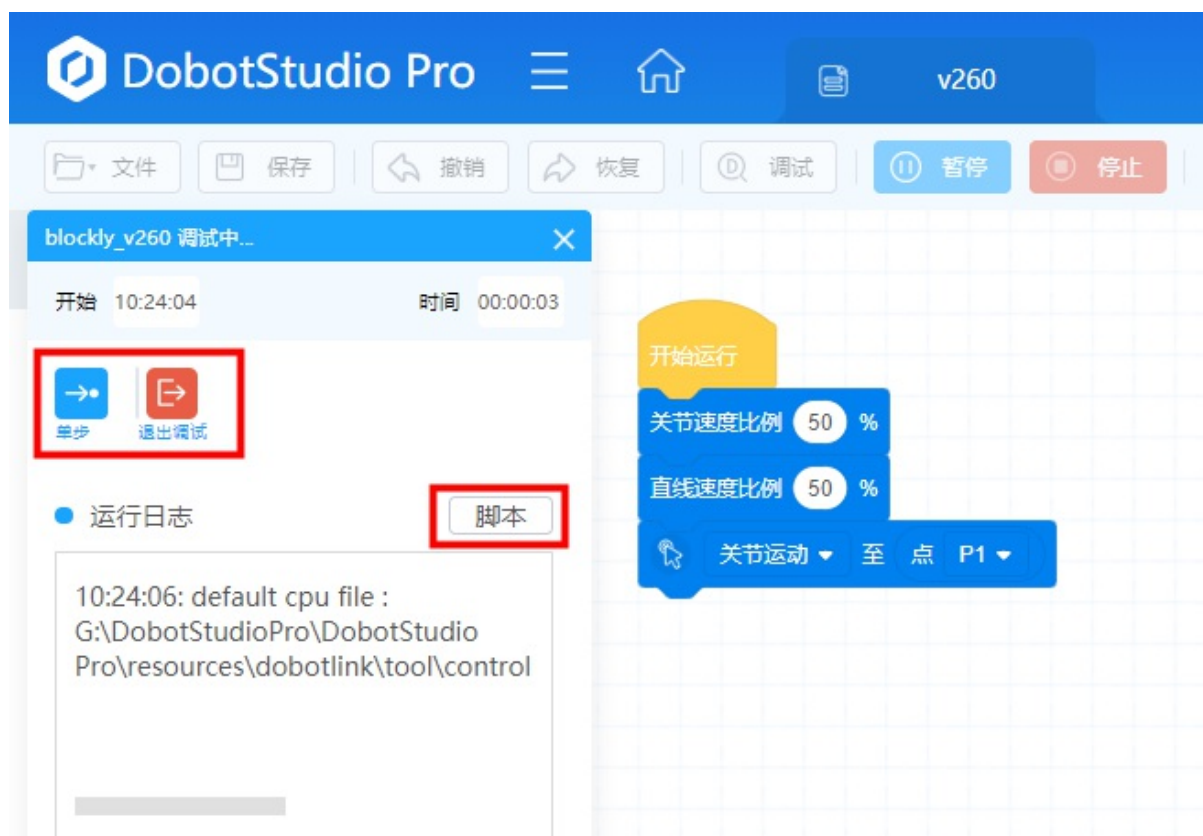
序号	说明
1	单击可隐藏存点面板，隐藏后可通过单击右侧工具栏的“存点”按钮恢复显示。如果有未保存的修改时右上角会显示红色*。
2	设置RunTo功能的运动模式。
3	导入示教点列表文件或者将当前示教点列表导出为文件。
4	单击可折叠显示控制面板，折叠状态下再次单击可取消折叠。

5	存点管理区域。 • 将机械臂运动至指定点位后，单击“新增”可以保存机械臂当前点位为一个新示教点。 • 选中一个示教点后双击这个示教点“Name”以外的任意值，可以直接修改该值。 • 选中一个示教点后单击“覆盖”，可以使用机械臂当前点位覆盖该存点。 • 选中一个示教点后长按“RunTo”，可以让机械臂运动至该点位。 • 选中一个示教点后单击“删除”，可以删除该示教点。
---	---

调试与运行工程

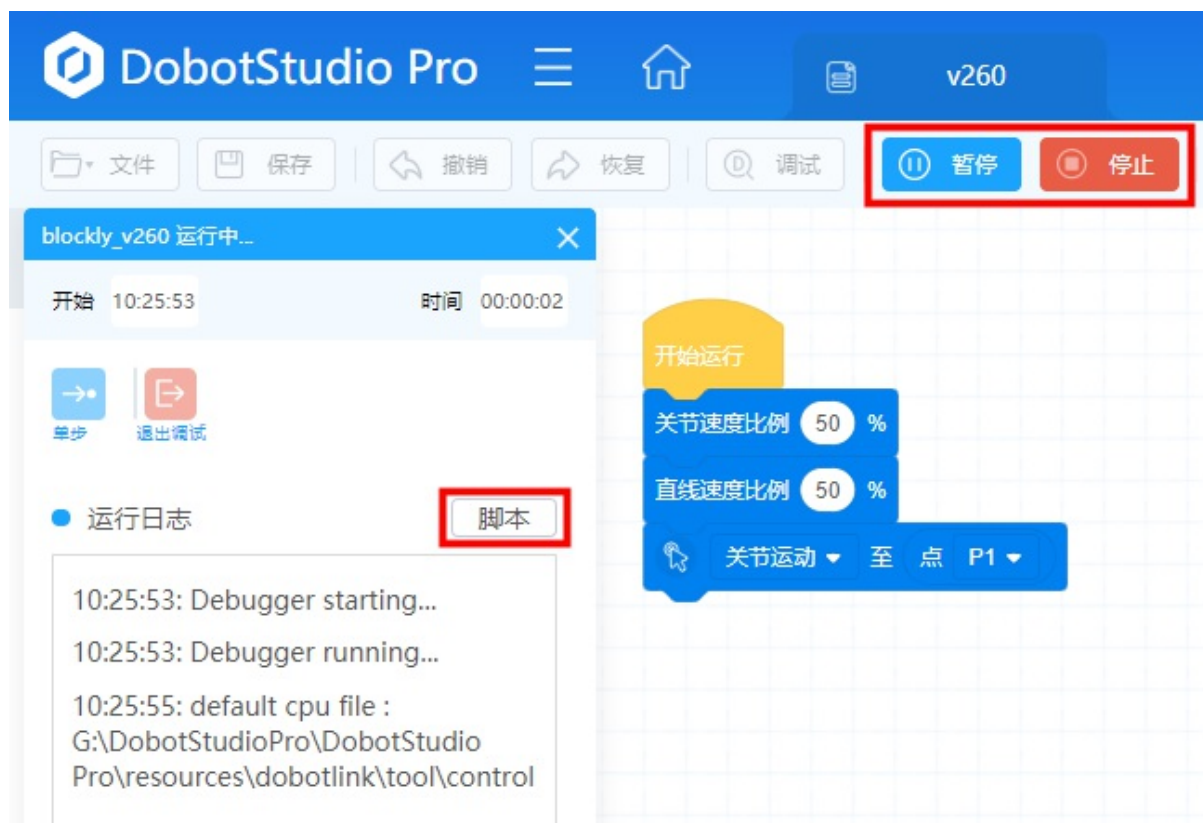
保存工程后单击“调试”，可以单步运行工程，并显示运行过程中的日志。

- 单击“单步”可单步运行工程，单击一次运行一条指令。
- 单击“退出调试”可以退出调试模式。
- 单击“脚本”可以显示该工程对应的运行脚本。



保存工程后单击“开始”，工程会开始运行，并显示运行过程中的日志。

- 单击“暂停”可以暂停运行工程，且该按钮会变为“继续”，单击会继续运行工程。
- 单击“停止”可以停止运行工程。
- 单击“脚本”可以显示该工程对应的运行脚本。



操作步骤

下文以编写一个程序控制机械臂在两个点之间循环运动为例，介绍如何使用图形编程的主要功能。

1. 打开存点界面，将机械臂运动至任意点P1后，单击"新增"保存点P1的位置信息。
2. 将机械臂运动至另一任意点P2后，单击"新增"保存点P2的位置信息。
3. 从左侧积木区拖动“重复执行”积木到“开始运行”积木的下方。
4. 拖动点运动积木到“重复执行”积木中，目标点选择P1。
5. 再拖动一个点运动积木到上一个点运动积木下，目标点选择P2。



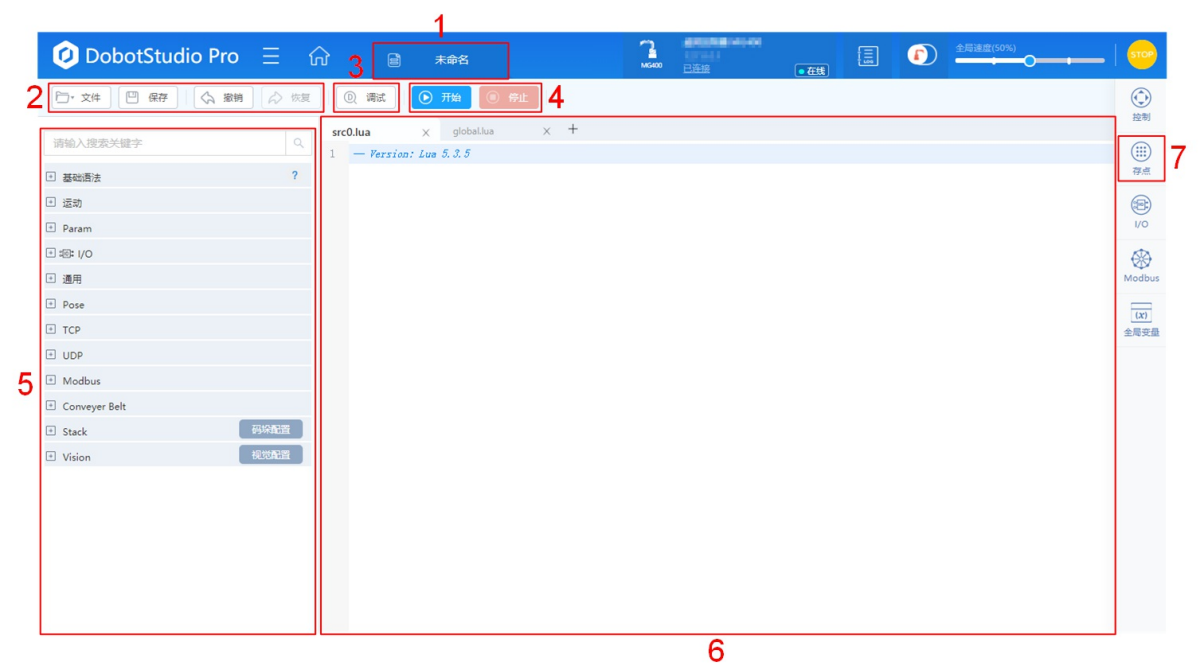
6. 单击“保存”，自定义工程名称并单击“确定”。
7. 单击“开始”，机械臂开始运动。

8.2 脚本编程

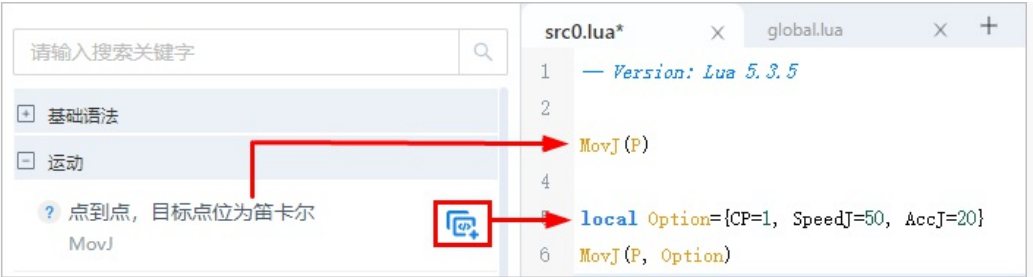
Dobot机械臂提供丰富的Dobot API接口，例如运动指令、TCP/UDP指令等，采用Lua语言，方便用户进行二次开发时调用。DobotStudio Pro提供了Lua脚本的编程环境，用户可以编写自己的Lua脚本控制机器人运行。

 **说明**

本章主要介绍如何使用脚本编程界面，具体的函数说明请参考[附录C](#)。



序号	说明
1	显示当前打开的工程名称。
2	用于管理工程文件和对编程操作进行撤销或恢复。
3	用于打开调试界面。详见 调试工程 。
4	用于控制当前工程的运行和停止。详见 运行工程 。
5	用于查找和使用编程函数。 - 单击函数左边的“?”可查看函数说明文档。 - 双击函数可以快速添加Lua指令到右侧编程区； - 如果函数右侧有蓝色按钮，双击蓝色按钮可快速添加带详细参数的Lua指令到右侧编程区。

	
6	程序的编写区域。 • "src0.lua"文件为主线程，可调用任何指令。 • "global.lua"文件仅用于定义变量和子函数。 • 单击 "+" 可以添加子线程，子线程是随主程序一起运行的并行程序，可设置 I/O、变量等，不能调用运动指令。
7	单击可打开存点界面，如果有未保存的修改时图标右下角会显示红点。详见存点界面。

存点界面

存点界面用于管理编程过程中要用到的机械臂点位，如下图所示。



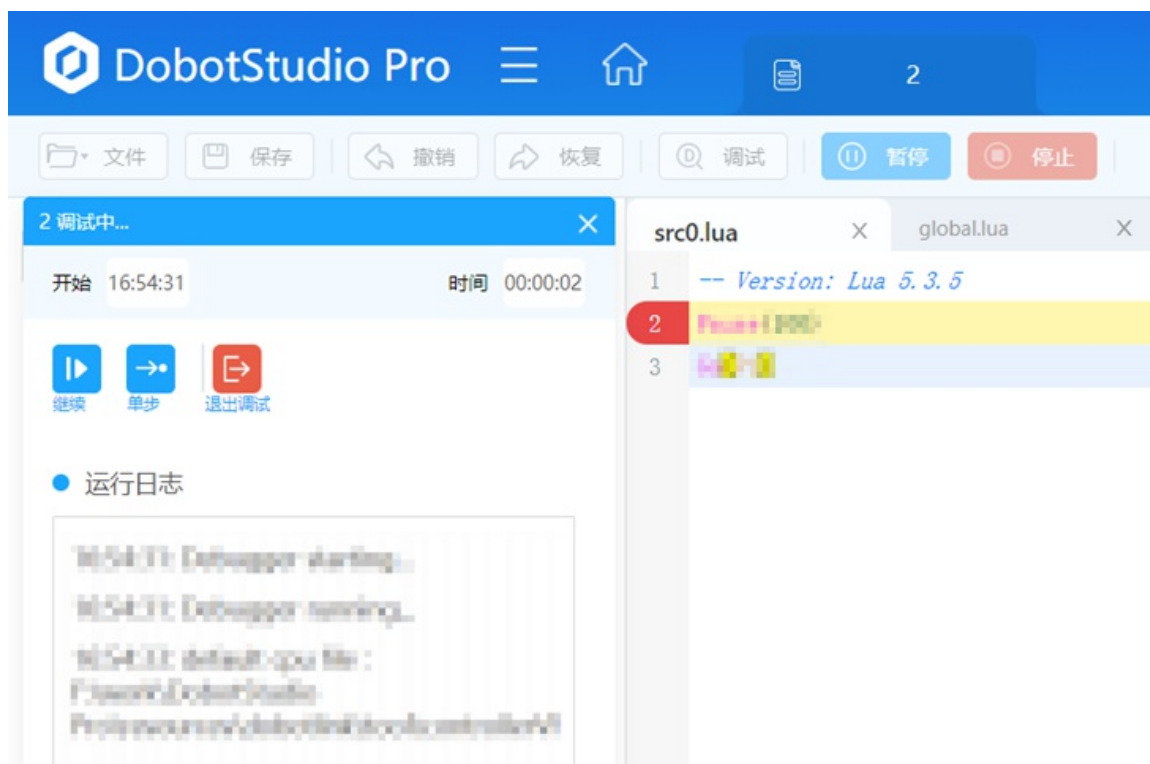
序号	说明
1	单击可隐藏存点面板，隐藏后可通过单击右侧工具栏的“存点”按钮恢复显示。如果有未保存的修改时右上角会显示红色*。
2	设置RunTo功能的运动模式。
3	导入示教点列表文件或者将当前示教点列表导出为文件。
4	单击可折叠显示控制面板，折叠状态下再次单击可取消折叠。

5	存点管理区域。 • 将机械臂运动至指定点位后，单击“新增”可以保存机械臂当前点位为一个新示教点。 • 选中一个示教点后单击“覆盖”，可以使用机械臂当前点位覆盖该存点。 • 选中一个示教点后长按“RunTo”，可以让机械臂运动至该点位。 • 选中一个示教点后单击“删除”，可以删除该示教点。
---	--

调试工程

保存工程后单击“调试”，工程会进入调试模式。

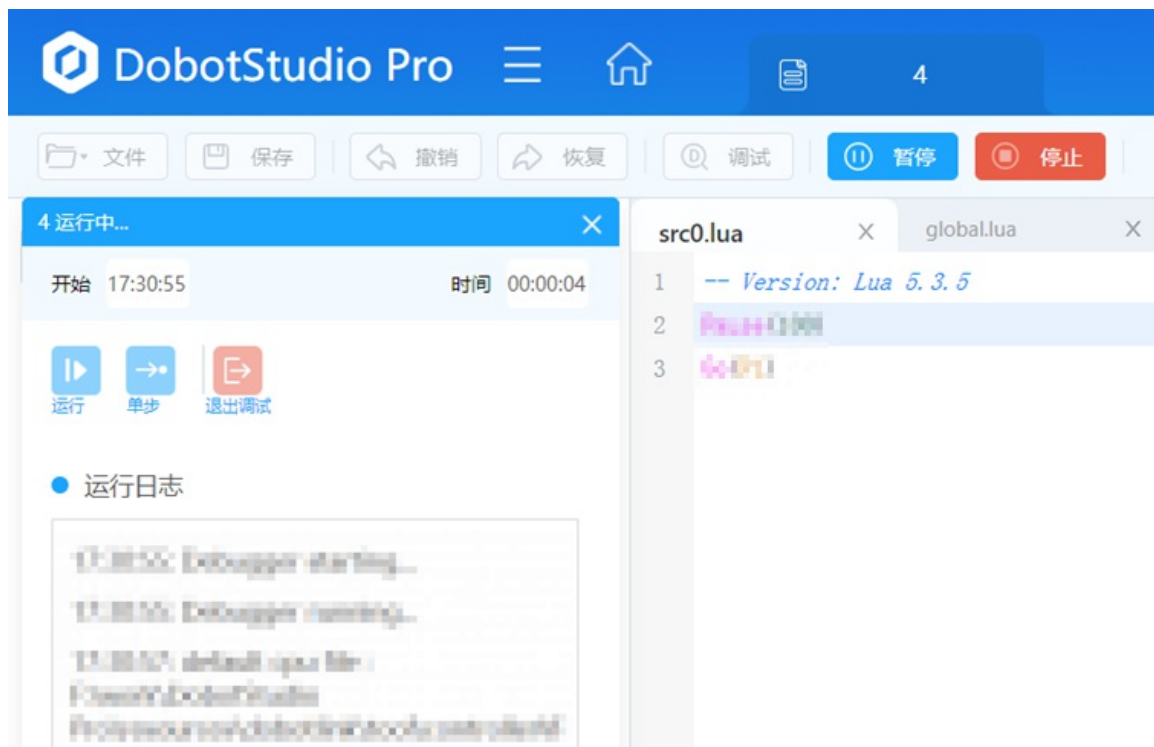
- 单击代码行左边的行号可以打断点，程序在调试模式下运行至断点会自动暂停。
- 程序在断点暂停后，单击“继续”可以让程序继续往下运行；单击“单步”程序会单步运行，单击一次运行一条指令。
- 单击“退出调试”可以退出调试模式。



运行工程

保存工程后单击“开始”，工程会开始运行，并显示运行过程中的日志。

- 单击“暂停”可以暂停运行工程，且该按钮会变为“继续”，单击会继续运行工程。
- 单击“停止”可以停止运行工程。



操作步骤

下文以编写一个程序控制机械臂在两个点之间循环运动为例，介绍如何使用脚本编程的主要功能。

1. 打开存点界面，将机械臂运动至任意点P1后，单击"新增"保存点P1的位置信息。
2. 将机械臂运动至另一任意点P2后，单击"新增"保存点P2的位置信息。
3. 在编程区域中添加循环指令。
4. 在循环代码的"end"前添加一条移动指令，目标点为P1。
5. 再添加一条移动指令，目标点选择P2。

```
while(true)
do
    MovJ(P1)
    MovJ(P2)
end
```

6. 单击“保存”，自定义工程名称并单击“确定”。
7. 单击“开始”，机械臂开始运动。

9 最佳实践

本章将会介绍通过远程I/O控制机械臂的完整流程，以帮助您理解DobotStudio Pro的各个功能是如何配合使用的。

为方便您理解流程，我们先预设一个示例场景：按下开始按钮后，运行指示灯亮起，然后机械臂通过末端的夹爪从取料点抓取物料，运动至下料点放下物料，然后再回到取料点抓取物料，重复循环。

为达成上述场景，需要给机械臂末端安装夹爪（假设安装的夹爪通过末端DI1控制，末端DI1为ON时张开，为OFF时闭合），并将按钮和指示灯连接至控制柜I/O接口（假设开始按钮连接至DI11，停止按钮连接至DI12；运行指示灯连接至DO11，报警指示灯连接至DO12；连线方法请参考机器人对应的硬件使用手册）。

整体流程

硬件安装完成且机械臂上电后，软件操作的整体流程如下：

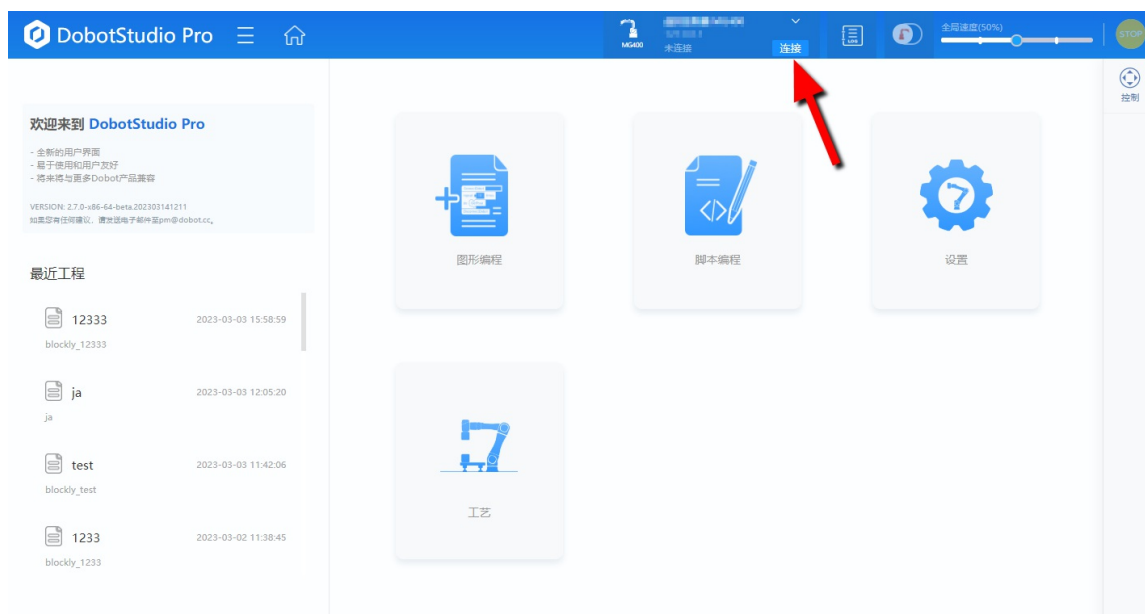
1. 连接机器人
2. 设置并选择工具坐标系
3. 编写工程
4. 配置并进入远程I/O模式

操作步骤

连接并使能机器人

连接机械臂的详细说明请参考章节[连接机器人](#)，此处以无线连接为例简单说明。

1. 在PC上搜索并连接机器人的WiFi，WiFi的默认SSID为“MagicianPro”，初始WiFi密码为：1234567890。
2. 在DobotStudio Pro界面上方选择你要连接的机器人，然后单击“连接”。



3. 单击使能按钮，设置负载参数后使能机械臂。

设置并选择工具坐标系

工具坐标系的详细说明请参考章节[工具坐标系](#)，此处以输入设置为例简单说明。

1. 打开“设置 > 坐标系管理 > 工具坐标系”页面。
2. 新增或修改坐标系，输入工具中心点相对法兰中心点的偏移量，并单击确定。

3. 在[控制面板](#)选择上一步设置的工具坐标系。

编写工程

编程的详细说明请参考章节[图形编程](#)和[脚本编程](#)，此处以图形编程为例简单说明。

为达成本章开头描述的示例场景，我们需要先示教四个点，分别是取料点P1，取料点上方过渡点P2，下料点上方过渡点P3，以及下料点P4。



1. 打开存点页面，将机械臂运动至P1，然后单击“新增”。
2. 用同样方法新增示教P2，P3，P4。
3. 拖动积木编程实现取料和下料。一个简单的程序示例如下图。



4. 保存工程。

配置并进入远程I/O模式

远程控制的详细说明请参考章节[远程控制](#)，此处仅基于示例场景做简单说明。

1. 打开“设置 > 远程控制”页面。
2. 修改当前模式为“远程I/O模式”。
3. 单击“打开”，选择上一节保存的图形编程工程。
4. 单击“修改”，根据本章开头所述场景修改I/O配置。
5. 单击“应用”，进入远程IO模式。

当前模式

远程I/O模式

运行的工程

图形编程

proj1

打开

I/O配置信息

修改

DI配置信息		DO配置信息	
开始	DI_11	准备状态	保留
暂停	保留	暂停状态	保留
继续	保留	报警状态	DO_12
停止	DI_12	运行状态	DO_11
紧急停止	保留		
清除报警	保留		

应用

进入远程I/O模式后，按下已连接到机械臂控制柜的开始按钮，机械臂就会开始运行工程。

Modbus寄存器定义

Modbus协议的数据主要分为四类：线圈状态、离散量输入、输入寄存器、保持寄存器，基于机器人的内存空间，我们定义了线圈、触点（离散输入）、输入、保持寄存器，用于外部设备与机器人控制系统进行数据交互。其中，每个寄存器地址为4096个。详细说明如下所示。**其中线圈寄存器和离散输入的定义可修改，请以远程控制界面显示的实际值为准。**

1 线圈寄存器（控制机器人）

PLC地址	脚本地址(Get/SetCoils)	数据类型	功能
00001	0	Bit	启动
00002	1	Bit	暂停
00003	2	Bit	继续
00004	3	Bit	停止
00005	4	Bit	急停
00006	5	Bit	清除报警
00007	6	Bit	复位
00051~00066	50~65	Bit	底座IO: DO1~DO16
00067~00070	66~69	Bit	末端IO: DO17~DO20
03096~04096	3095~4095	Bit	用户自定义

2 离散输入定义（机器人状态）

PLC地址	脚本地址(GetInBits)	数据类型	功能
10002	1	Bit	停止状态
10003	2	Bit	暂停中
10004	3	Bit	运行状态
10005	4	Bit	报警状态
10006	5	Bit	碰撞状态
10007	6	Bit	手动/自动模式
10008	7	Bit	系统预留

10051~10066	50~65	Bit	底座IO: DI1~DI16
10067~10070	66~69	Bit	末端IO: DI17~DI20

3 输入寄存器定义

PLC地址	脚本地址(GetInRegs)	数据类型	功能
30203	202	F32	机器人运行位置 (关节角1)
30205	204	F32	机器人运行位置 (关节角2)
30207	206	F32	机器人运行位置 (关节角3)
30209	208	F32	机器人运行位置 (关节角4)
30211	210	F32	机器人运行位置 (关节角5)
30213	212	F32	机器人运行位置 (关节角6)
30243	242	F32	机器人运行位置 (x)
30245	244	F32	机器人运行位置 (y)
30247	246	F32	机器人运行位置 (z)
30249	248	F32	机器人运行位置 (a)
30251	250	F32	机器人运行位置 (b)
30253	252	F32	机器人运行位置 (c)

4 保持寄存器定义 (机器人PLC交互)

PLC地址	脚本地址 (Get/SetHoldRegs)	数据类型	功能
40001~41281	0~1280	U16	码垛
41301	1300	U16	切换进入HMI点动模式
41302	1301	U16	切换HMI点动模式就绪
41303	1302	U16	点寸动类型: 关节/笛卡尔
41304	1303	U16	点动/寸动选择
41305	1304	U16	全局速度: 百分比
41306	1305	F32	寸动距离: 毫米

41308	1307	F32	寸动角度：度
41310	1309	U16	工具坐标系选择：索引
41311	1310	U16	用户坐标系选择：索引
41312	1311	U16	手系
41313	1312	U16	通知修改参数
41314	1313	U16	启动点动
41315	1314	U16	J1+/X+
41316	1315	U16	J1-/X-
41317	1316	U16	J2+/Y+
41318	1317	U16	J2-/Y-
41319	1318	U16	J3+/Z+
41320	1319	U16	J3-/Z-
41321	1320	U16	J4+/A+
41322	1321	U16	J4-/A-
41323	1322	U16	J5+/B+
41324	1323	U16	J5-/B-
41325	1324	U16	J6+/C+
41326	1325	U16	J6-/C-
41327	1326	F32	P1(X)
41329	1328	F32	P1(Y)
41331	1330	F32	P1(Z)
41333	1332	F32	P1(R/A)
41335	1334	F32	P1(B)
41337	1336	F32	P1(C)
41339	1338	U16	P1(ARM)
41340	1339	U16	P1(User)
41341	1340	U16	P1(Tool)
41342~41551	1341~1550	F32&U16	P2~P15
41552	1551	F32	P16(X)
41554	1553	F32	P16(Y)

41556	1555	F32	P16(Z)
41558	1557	F32	P16(R/A)
41560	1559	F32	P16(B)
41562	1561	F32	P16(C)
41564	1563	U16	P16(ARM)
41565	1564	U16	P16(User)
41566	1565	U16	P16(Tool)
41567	1566	U16	点位保存
41568	1567	U16	RUNTO: 关节/直线
41569	1568	U16	RUNTO: 点位序号
41570	1569	U16	RUNTO: 开始
41571	1570	U16	清除报警
42010	2009	F32	多机1 (主机) x
42012	2011	F32	多机1 (主机) y
42014	2013	F32	多机1 (主机) r
42016	2015	F32	多机1 (主机) encCount
42018	2017	U16	多机1 (主机) type
42019	2018	U16	多机1 (主机) available
42020~42029	2019~2028	U16	预留
42030	2029	F32	多机2 (从机) x
42032	2031	F32	多机2 (从机) y
42034	2033	F32	多机2 (从机) r
42036	2035	F32	多机2 (从机) encCount
42038	2037	U16	多机2 (从机) type
42039	2038	U16	多机2 (从机) available
42040~42049	2039~2048	U16	预留
42050	2049	F32	多机3 (从机) x
42052	2051	F32	多机3 (从机) y
42054	2053	F32	多机3 (从机) r

42056	2055	F32	多机3（从机） encCount
42058	2057	U16	多机3（从机） type
42059	2058	U16	多机3（从机） available
43095~44095	3095~4095	U16	用户自定义

图形编程积木说明

- **B.1 快速体验**
 - B.1.1 控制机械臂运动
 - B.1.2 读写Modbus寄存器数据
 - B.1.3 通过TCP通信传递数据
 - B.1.4 使用托盘积木进行码垛
- **B.2 积木说明**
 - B.2.1 事件积木组
 - B.2.2 控制积木组
 - B.2.3 运算积木组
 - B.2.4 字符积木组
 - B.2.5 自定义积木组
 - B.2.6 IO积木组
 - B.2.7 运动积木组
 - B.2.8 运动高级配置
 - B.2.9 位姿积木组
 - B.2.10 Modbus积木组
 - B.2.11 TCP积木组

快速体验

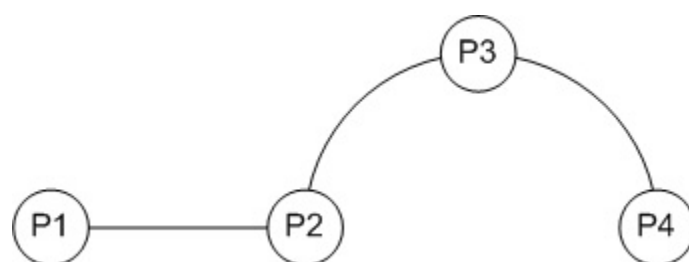
- 控制机械臂运动
- 读写Modbus寄存器数据
- 通过TCP通信传递数据
- 使用托盘积木进行码垛

控制机器人运动

场景说明

为了方便体验如何用图形编程控制机器人运动，我们先假设要实现以下场景：

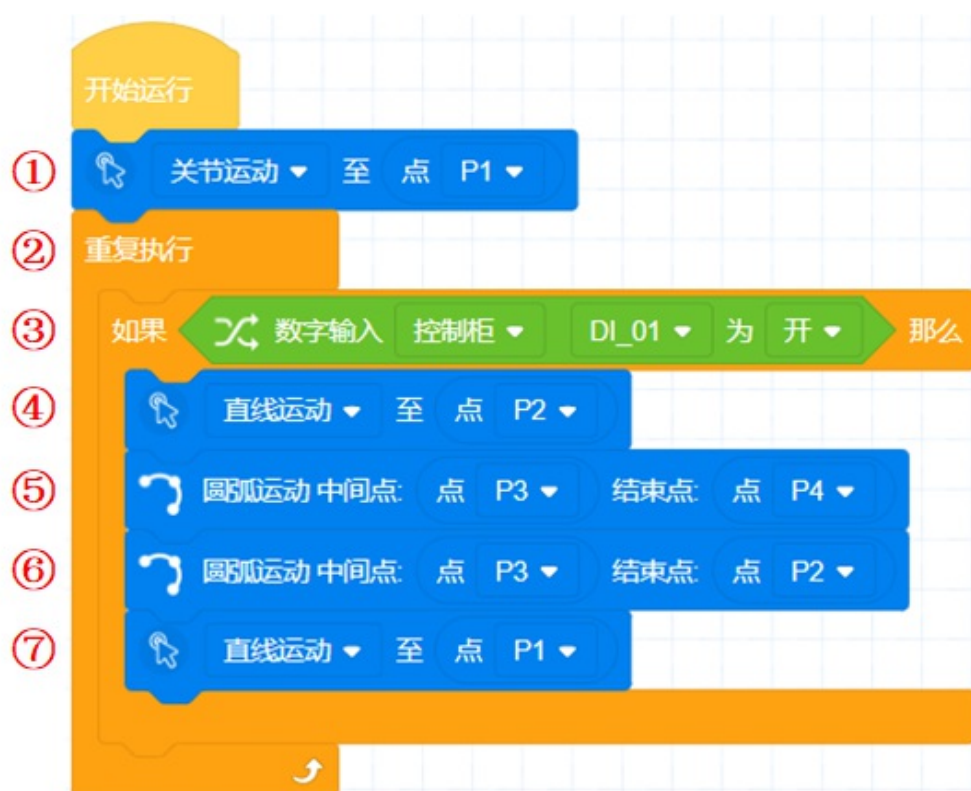
当控制柜DI1为ON时，机器人从P1直线运动到P2后，经由P3圆弧运动到P4，然后再原路返回。
当控制柜DI1为OFF时，机械臂不运动。



请先参考上图示教P1~P4。

编程步骤

为实现上述场景，我们需要编写的程序如下图。



1. 机器人关节运动至起点（P1）。
2. 设置无条件循环，使后续指令在程序运行时一直循环。
3. 判断控制柜DI1是否为ON，为ON才会执行后续程序，否则直接进入下一次循环，重新获取

DI1的状态。

4. 机器人直线运动至P2。
5. 机器人经由P3圆弧运动至P4。
6. 机器人经由P3圆弧运动至P2（原路返回）。
7. 机器人直线运动P1，然后进入下一次循环，回到步骤3。

运行程序

完成点位示教和编程后，运行程序即可。DI1的状态可在IO监控面板中通过虚拟DI设置。

读写Modbus寄存器数据

场景说明

为了体验如何通过图形编程读写Modbus数据，我们先假设要实现以下场景：

机器人创建一个Modbus主站，连接外部从站并读取指定线圈寄存器的地址。若该地址的值为1，则机器人运动至P1点。

编程步骤

为实现上述场景，我们需要编写的程序如下图。



1. 创建主站，IP地址为从站地址，端口和ID保留默认值。此处为了便于快速验证，使用了机器人自带的从站，所以IP地址为机器人的地址。
2. 判断创建主站是否成功，创建成功才会执行后续步骤，否则程序直接结束。
3. 若机器人线圈寄存器0的值被修改过，可能会对后续程序逻辑造成影响，因此需要先将线圈寄存器0的值置为0。
4. 等待线圈寄存器0的值变为1。
5. 控制机器人运动至P1点，P1为用户自定义的存点。
6. 关闭主站。

运行程序

如果需要快速运行该程序，可使用DobotStudio Pro自带的调试工具修改线圈寄存器的值。



1. 打开调试工具，进入“Modbus 调试工具 > Modbus TCP”页面。
2. 将机器人运动至P1以外的点（便于后续观察机器人是否执行了运动指令），然后保存并运行程序。
3. 待运行日志提示Modbus主站创建成功后，在调试工具中勾选“激活”，然后修改Modbus服务器的网络地址与端口。
4. 将Modbus请求的功能码改为“写单线圈”，然后将寄存器0的数据改为1，再单击“发送”。
5. 观察机器人是否运动至P1。

下图为调试工具界面，图中的序号对应上述的步骤。



通过TCP通信传递数据

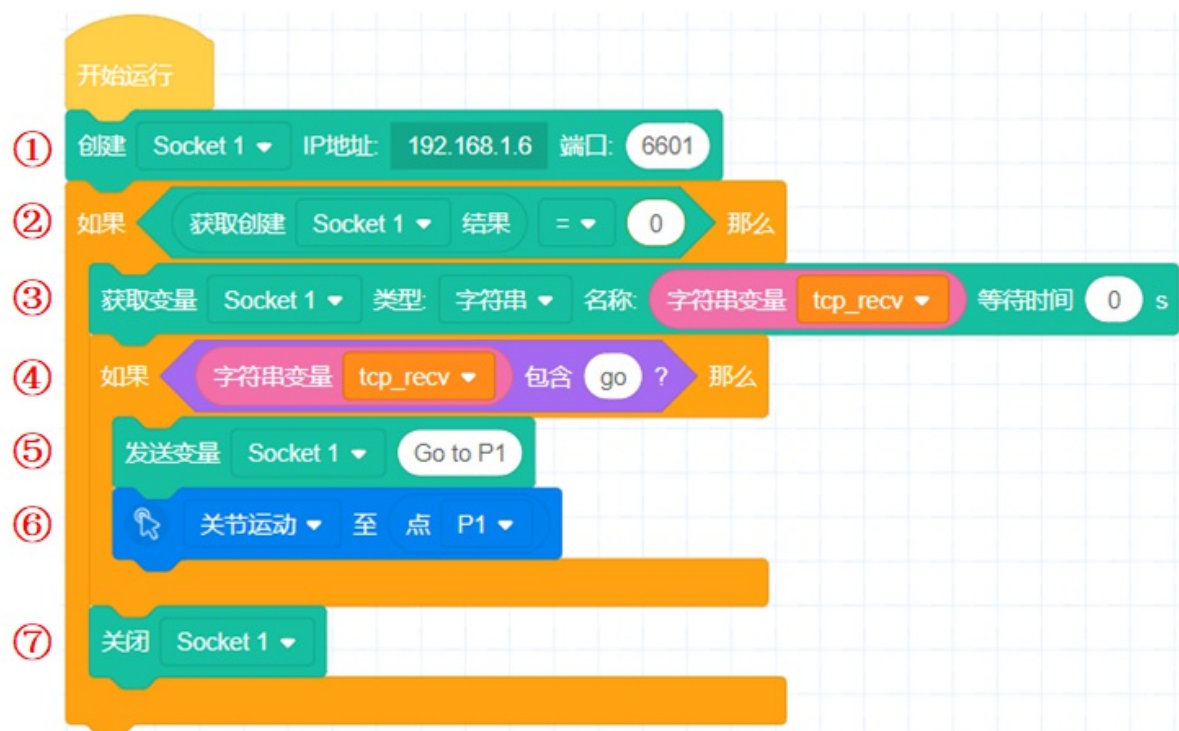
场景说明

为了体验如何通过图形编程进行TCP通信，我们先假设要实现以下场景：

机器人创建一个TCP服务端，等待客户端连接并发送"go"指令，然后返回“Go to P1”消息并开始运动到P1点。

编程步骤

为实现上述场景，我们需要编写的程序如下图。



1. 创建TCP服务端 (Socket 1) ,IP地址为机器人的地址，端口自定义。
2. 判断创建服务端是否成功，创建成功才会执行后续步骤，否则程序直接结束。
3. 等待客户端连接并发送字符串，将收到的字符串保存到字符串变量tcp_recv中。字符串变量需要用户自己创建。



4. 判断收到的字符串中是否包含“go”，如果包含，再执行步骤5和6，否则直接执行步骤7。
5. 发送字符串“Go to P1”给客户端。
6. 控制机器人运动至P1点，P1为用户自定义的存点。
7. 关闭TCP服务端。

运行程序

如果需要快速运行该程序，可使用DobotStudio Pro自带的调试工具作为TCP客户端。



1. 打开调试工具，进入“网络调试工具 > TCP客户端”页面。
2. 将机器人运动至P1以外的点（便于后续观察机器人是否执行了运动指令），然后保存并运行程序。
3. 待运行日志提示TCP服务端创建成功后，在调试工具中修改服务器的IP地址与端口，单击连接。
4. 连接成功后，在调试工具下方输入“go”并单击发送。
5. 观察调试工具是否收到了“Go to P1”消息且机器人是否运动至P1。

下图为调试工具界面，图中的序号对应上述的步骤。

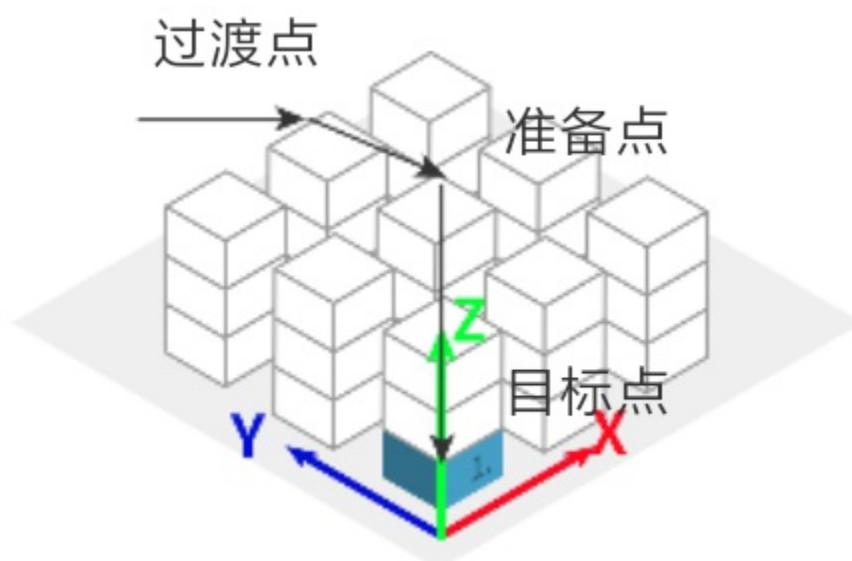


使用托盘积木进行码垛

场景说明

在搬运应用中，有些搬运的物料排列规则且间距均匀，若一一示教各个物料的位置会存在误差大、效率低的问题。使用托盘积木可通过设置踩型自动计算出各物料的位置，有效的解决该类问题。

首先，我们假设需要将物料堆放成一个立方体，我们需要先手动码垛一个目标踩型出来，然后示教相关点位：



- 示教过渡点为P1。过渡点是机械臂进出码垛区域都会经过的一个点位，用于安全过渡，可示教为取料点上方。
- 示教取料箱的点为P2。
- 准备点和目标点不需要逐个示教，请参考后续配置踩型。

然后，我们假设机械臂末端已安装夹爪或吸盘等机构，通过控制柜的DO1控制抓取或放置物料。

配置踩型

将创建托盘积木拖动到编程区后，单击积木可打开托盘面板。



托盘面板

X

托盘维度

三维



1点X方向个数

10

1点Y方向个数

10

1点Z方向个数

10

点位配置

1

2

3

4

5

6

7

8

点位1

P3

自定义

取消

保存

托盘维度

- 一维：物料排成一行，物料总数等于X方向上的个数。
- 二维：物料排成方形，物料总数等于X方向和Y方向上的个数的乘积。
- 三维：物料堆放成立方体，物料总数等于三个方向上的个数的乘积。

本文以三维踩型为例，分别设置各个方向上的物料个数为10，则一个完整的踩型总共有1000个物料。

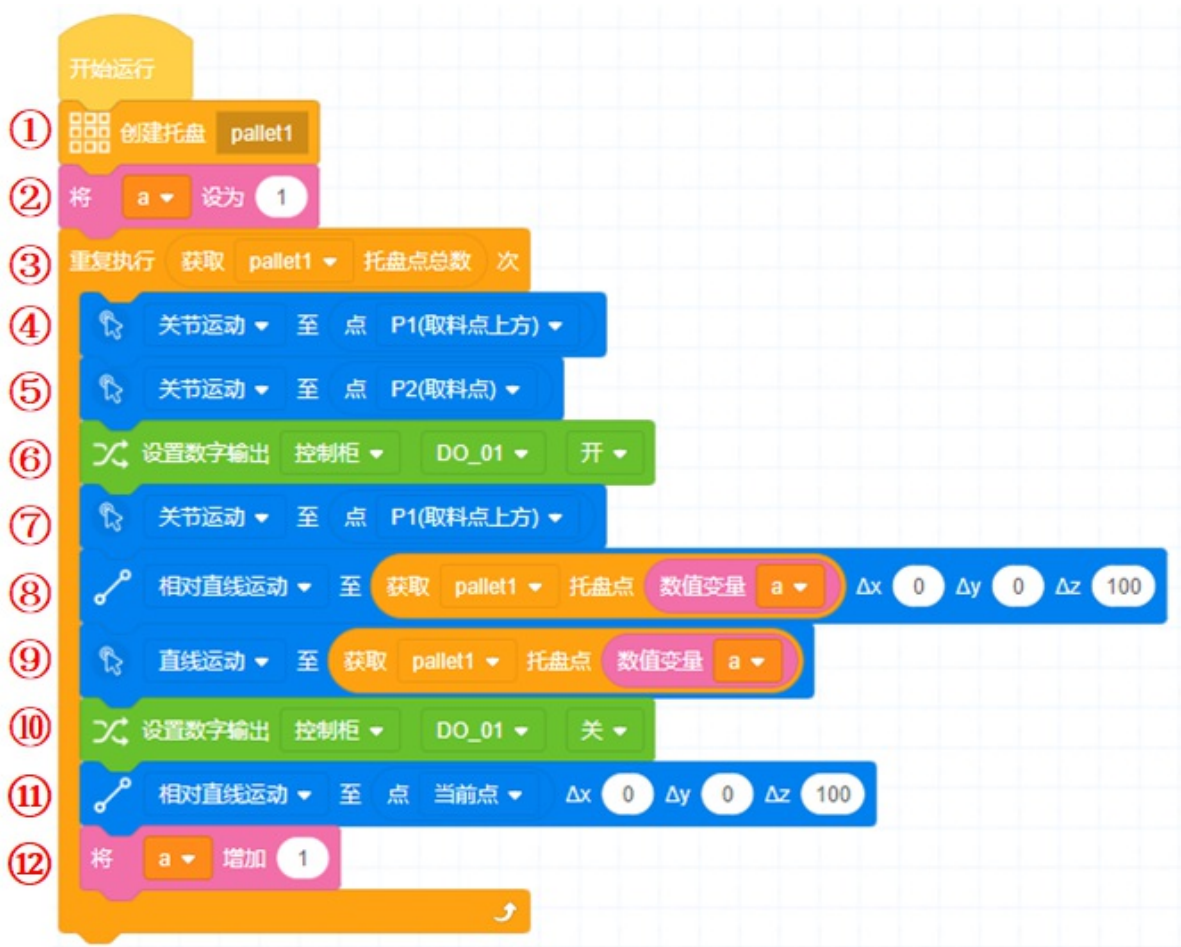
点位配置

以三维踩型为例，此处需要配置八个点，分别对应立方体的八个角上的物料的目标点（放料时的机械臂点位），控制系统会通过这八个点和物料的数量自动计算出每个物料的目标点，然后按照X->Y->Z的坐标轴顺序进行排序。

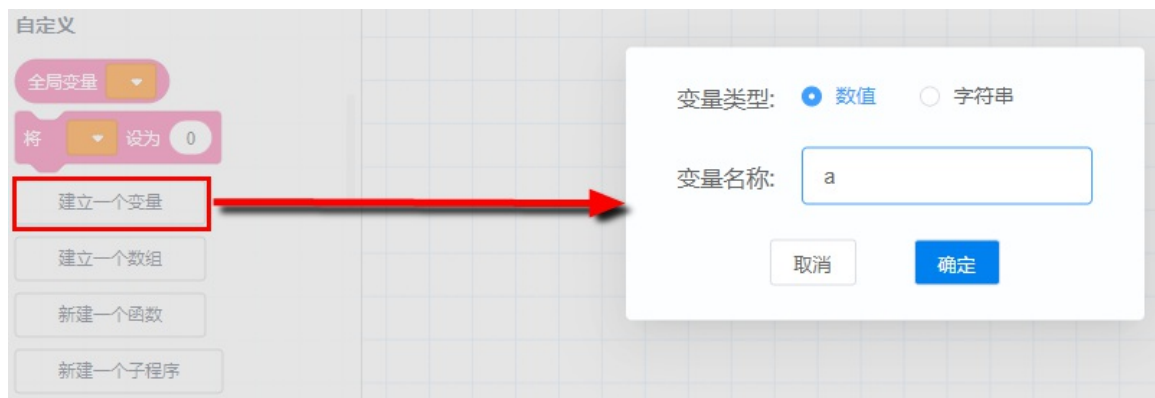
配置点位时可以选择工程中已示教的存点，也可以单击“自定义”后获取机械臂当前点位坐标。已配置的点位图标会变为绿色。

编程步骤

为实现上述场景，我们需要编写的程序如下图。



1. 创建托盘pallet1。
2. 新建一个自定义数值变量，将值设为1，用于记录循环次数。



3. 循环执行后续代码，次数为对应托盘的点总数。
4. 机器人运动至取料点上方（P1）。
5. 运动至取料点（P2）。
6. 设置DO1为开，控制夹爪抓取物料。
7. 机器人回到取料点上方（P1）。
8. 运动至当前托盘点上方100mm处。

9. 运动至当前托盘点。
10. 设置DO1为关，控制夹爪放开物料。
11. 回到当前托盘点上方100mm处。
12. 循环次数加一，然后循环回到步骤4。

本文编写的程序仅作为一个简单的示例，实际应用可以根据实际情况加上更多的IO控制和情况判断，例如没取到物料时就不执行后续动作。

运行程序

完成点位示教、踪型配置和编程后，运行程序即可。DO1的状态可在IO监控面板中查看。

积木说明

- 事件积木组
- 控制积木组
- 运算积木组
- 字符积木组
- 自定义积木组
- IO积木组
- 运动积木组
- 运动高级配置
- 位姿积木组
- Modbus积木组
- TCP积木组

事件积木组

事件积木组用于作为程序开始运行的标识。

开始运行



描述：程序主线程的标识。创建新工程后编程区默认会有一个开始运行标识，请将其他非事件积木接在其下方进行编程。

使用限制：一个工程中只能有一个“开始运行”积木。

子线程启动



描述：程序子线程的标识。子线程会和主线程同步运行，但子线程不可以使用机械臂控制命令，仅可进行变量运算或IO控制等。请根据程序逻辑需要选择是否使用子线程。

使用限制：一个工程中最多只能有5个子线程。

控制积木组

控制积木组用于控制程序的运行路径。

等待直到满足条件



描述：程序暂停，直到参数为true再继续运行。

参数：使用其他六边形积木作为参数。

重复执行n次



描述：把其他积木嵌套到该积木中间，被嵌套的积木指令会被重复执行指定的次数。

参数：重复执行的次数。

持续重复执行



描述：把其他积木嵌套到该积木中间，被嵌套的积木指令会被一直重复执行，直到遇到结束重复积木。

结束重复

结束重复

描述：用于嵌套在重复执行类的积木中，程序执行到该基本的时候会直接结束重复，执行重复积木之后的积木指令。

满足条件后执行



描述：如果参数为true，则执行嵌套的积木指令。参数为false时直接跳到下一个积木指令。

参数：使用其他六边形积木（返回值为布尔值，即true或者false）作为参数。

满足或不满足条件后分别执行



描述：如果参数为true，则执行“否则”之前嵌套的积木指令。参数为false时，执行“否则”之后嵌套的积木指令。

参数：使用其他六边形积木（返回值为布尔值，即true或者false）作为参数。

重复执行直到满足条件



描述：重复执行嵌套的积木指令直到参数为true。

参数：使用其他六边形积木（返回值为布尔值，即true或者false）作为参数。

设置标签



描述：设置一个标签，设置后可以通过标签跳转积木进行跳转。

参数：标签名称，必须以字母开头，不能使用空格等特殊字符。

标签跳转



描述：程序执行到该积木后，会直接跳转到指定的标签，执行标签之后的积木指令。

参数：已设置的标签名称。

指令折叠



描述：可以将嵌套的积木折叠显示。不起控制作用，仅用于使程序更加美观和易读。

参数：描述被折叠的积木块，建议起一个间接且直观的名称。

暂停

An orange block with a notch on the left and a bump on the right, containing the text "暂停" (Pause).

描述：程序运行到该积木后自动暂停，需要通过控制软件或者远程控制操作才可继续运行。

设置碰撞检测功能

An orange block with a notch on the left and a bump on the right. It contains the text "设置碰撞检测为" (Set collision detection to) followed by a dropdown menu showing "关闭" (Off) with a downward arrow.

描述：设置碰撞检测功能。通过该积木设置的碰撞检测等级仅在工程运行期间生效，工程停止后会恢复为修改前的值。

参数：选择碰撞检测功能的灵敏度，可选择关闭或等级1到5，等级数字越大碰撞检测越灵敏。

修改用户坐标系

An orange block with a notch on the left and a bump on the right. It contains the text "修改用户坐标系" (Modify user coordinate system) followed by a dropdown menu showing "0" with a downward arrow, then the text "为 X" (to X), a circular input field with "0", the text "Y", a circular input field with "0", the text "Z", a circular input field with "0", the text "R", and a circular input field with "0".

描述：修改指定的用户坐标系。该修改仅在当前工程运行中生效，工程停止后坐标系会恢复为修改前的值。

参数：

- 指定要修改的用户坐标系的编号。
- 指定修改后的用户坐标系参数。

修改工具坐标系

An orange block with a notch on the left and a bump on the right. It contains the text "修改工具坐标系" (Modify tool coordinate system) followed by a dropdown menu showing "0" with a downward arrow, then the text "为 X" (to X), a circular input field with "0", the text "Y", a circular input field with "0", the text "Z", a circular input field with "0", the text "R", and a circular input field with "0".

描述：修改指定的工具坐标系。该修改仅在当前工程运行中生效，工程停止后坐标系会恢复为修改前的值。

参数：

- 指定要修改的工具坐标系的编号。
- 指定修改后的工具坐标系参数。

创建托盘



描述：用于创建一个托盘的垛型，详见[使用托盘积木进行码垛](#)。

参数：要创建的托盘的名称。

获取指定托盘总点数



描述：获取指定的托盘的目标垛点的总数。

参数：托盘的名称。

获取指定托盘点位坐标



描述：获取指定的托盘的指定点位坐标。

参数：

- 托盘的名称。
- 点位的索引序号，从1开始。

设置负载参数



描述：设置机械臂的负载参数。

参数：

- 末端负载重量，不可超过机械臂的最大负载重量，单位：克。
- 若末端装备了偏心工具，需要设置对应的偏心坐标，未装备时设置为0，单位：毫米。
- 驱动参数索引为高级功能，可不填写。如需使用，请在现场工程师的指导下设置。

延时执行指令



描述：程序运行到该积木后，会暂停指定的时间后再继续运行。

参数：程序暂停的时间。

运动等待指令



描述：接在运动积木前后，起到延时下发运动指令或运动完成后延时下发下一条指令的作用。

参数：指令延时下发的时间。

获取系统时间



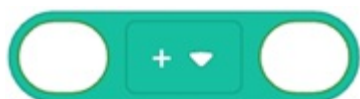
描述：获取系统当前的时间。

返回值：系统当前时间的Unix时间戳。

运算积木组

运算积木组用于对变量或常量进行运算。

四则运算



描述：对参数进行四则运算。

参数：

- 两边填写参与运算的变量或常量，可以使用返回值为数值的椭圆形积木或者直接填写。
- 中间选择四则运算符。

返回值：运算结果的数值。

比较运算



描述：对参数进行比较运算。

参数：

- 两边填写参与运算的变量或常量，可以使用返回值为数值的椭圆形积木或者直接填写。
- 中间选择比较运算符。

返回值：比较结果为真时返回true，为假时返回false。

与运算



描述：对参数进行与运算。

参数：两边填写参与运算的变量，使用其他六边形积木。

返回值：两个参数都为真时返回true，任一个为假返回false。

或运算



描述：对参数进行或运算。

参数：两边填写参与运算的变量，使用其他六边形积木。

返回值：两个参数任一个为真时返回true，都为假时false。

非运算



描述：对参数进行非运算。

参数：填写参与运算的变量，使用其他六边形积木。

返回值：参数为真时返回false，为假时true。

取余运算



描述：对参数进行取余运算。

参数：两边填写参与运算的变量或常量，可以使用返回值为数值的椭圆形积木或者直接填写。

返回值：运算结果的数值。

四舍五入运算



描述：对参数进行四舍五入运算。

参数：填写参与运算的变量或常量，可以使用返回值为数值的椭圆形积木或者直接填写。

返回值：运算结果的数值。

单值运算



描述：对参数进行各种单值运算。

参数：

- 选择运算方式。
 - 绝对值
 - 向下取整
 - 向上取整
 - 平方根
 - sin
 - cos
 - tan
 - asin
 - acos
 - atan
 - ln
 - loh
 - e^
 - 10^
- 填写参与运算的变量或常量，可以使用返回值为数值的椭圆形积木或者直接填写。

返回值：运算结果的数值。

打印



描述：将参数输出到控制台查看，主要用于调试。

参数：

- 选择同步打印还是异步打印。同步打印会等已下发的全部指令执行完毕后再打印信息，异步打印则程序执行到该积木时立即打印信息。
- 要输出到控制台的变量或常量，可以使用其他椭圆形积木或者直接填写。

字符积木组

字符积木组包含了字符串和数组的常用功能。。

获取字符串的第n个字符



描述：获取作为变量的字符串的第n个字符。

参数：

- 第一个参数填写字符串，可以使用其他椭圆形积木或者直接填写。
- 第二个参数指定要返回字符串的第几个字符。

返回值：字符串指定位置的字符。

判断字符串一是否包含字符串二



描述：判断第一个参数的字符串中是否包含了第二个参数的字符串。

参数：要判断的两个字符串，可以使用返回值为字符串的椭圆形积木或者直接填写。

返回值：字符串一包含了字符串二时返回true，否则返回false。

拼接两个字符串

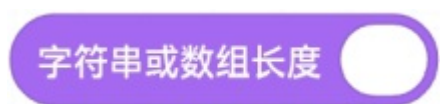


描述：将两个字符串拼接成一个字符串，第二个字符串会接在第一个字符串后面。

参数：要拼接的两个字符串，可以使用返回值为字符串的椭圆形积木或者直接填写。

返回值：拼接后的字符串。

获取字符串或数组长度



描述：获取指定的字符串或数组的长度。字符串的长度是指该字符串有多少个字符，数组的长度是指该数组有多少个元素。

参数：要计算长度的字符串或数组，可以使用返回值为字符串或数组的椭圆形积木。

返回值：字符串或数组的长度数值。

比较两个字符串

字符串比较

描述：根据ASCII码比较两个字符串的大小。

参数：要比较的两个字符串，可以使用返回值为字符串的椭圆形积木或者直接填写。

返回值：字符串一和字符串二相等时返回0，字符串一小于字符串二时返回-1，字符串一大于字符串二时返回1。

将数组转换为字符串

数组转换为字符串 数组： 分隔符：

描述：将指定数组转换为字符串，字符串中不同数组元素使用指定分隔符分隔。例如数组为{1,2,3}，分隔符为|，则转换后的字符串为“1|2|3”。

参数：

- 要转成字符串的数组，使用返回值为数组的椭圆形积木。
- 转换使用的分隔符。

返回值：转换后的字符串。

将字符串转换为数组

字符串转换为数组 字符串： 分隔符：

描述：将指定字符串转换为数组，根据指定的分隔符对字符串进行分隔。例如字符串为“1|2|3”，分隔符为|，则转换后的数组为{[1]=1,[2]=2,[3]=3}。

参数：

- 要转成数组的字符串，可以使用返回值为字符串的椭圆形积木或直接输入。
- 转换使用的分隔符。

返回值：转换后的数组。

获取数组指定下标的元素



描述：获取指定数组中指定下标位置的元素。下标表示元素在数组中的位置，例如数组{7,8,9}中8的下标就是2。

参数：

- 目标数组，使用返回值为数组的椭圆形积木。
- 指定元素的下标。

返回值：数组指定位置的元素的值。

获取数组中多个指定元素



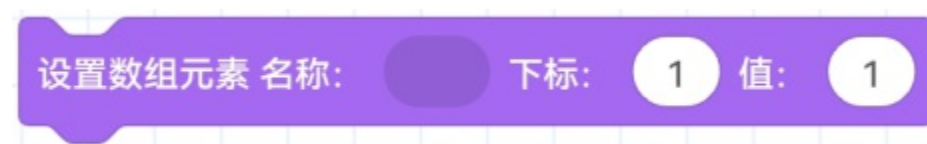
描述：获取指定数组中多个指定下标位置的元素。在开始下标和结束下标的范围内，根据步进值获取元素。

参数：

- 目标数组，使用返回值为数组的椭圆形积木。
- 通过开始下标和结束下标指定获取元素的范围。
- 步进值用于确定获取元素的频率，1就是全部获取，2就是隔一个元素进行获取，以此类推。

返回值：指定的元素组成的新数组。

设置数组中的指定元素



描述：设置数组中指定下标位置的元素的值。

参数：

- 目标数组，使用返回值为数组的椭圆形积木。
- 指定元素的下标。
- 指定元素的值。

自定义积木组

自定义积木组用于新建和管理自定义积木，以及调用全局变量。

调用全局变量



描述：调用控制软件中设置的全局变量。

参数：选择全局变量的名称。

返回值：全局变量的值。

设置全局变量



描述：设置指定的变量。需要注意的是，设置全局变量和设置自定义变量的积木外形相同，但功能略有不同。

参数：

- 选择要修改的变量的名称。
- 修改后的值，可以直接填写或使用其他椭圆形积木。

新建自定义变量



单击可新建一个自定义变量，变量类型可选择数值或者字符串，变量名称必须以字母开头，不能使用空格等特殊字符。创建至少一个变量后，积木列表中会出现下述自定义变量相关积木。

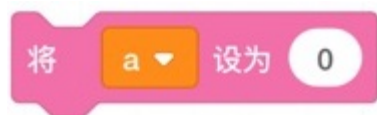
自定义数值变量



描述：新建的自定义数值变量，默认值为nil，建议赋值后再使用。可以通过变量选择下拉框修改现在选中的变量的名称或删除该变量。

返回值：变量的值。

设置自定义数值变量的值



描述：设置指定数值变量。需要注意的是，设置全局变量和设置自定义变量的积木外形相同，但功能略有不同。

参数：

- 选择要修改的变量的名称。
- 修改后的值，可以直接填写或使用其他椭圆形积木。

增减自定义数值变量的值



描述：将指定数值变量增加指定的值。

参数：

- 选择要修改的变量的名称。
- 要增加的值，可以直接填写或使用其他椭圆形积木。设置为负数时可实现减少值。

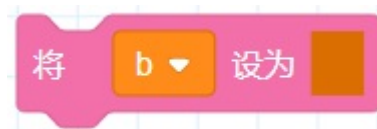
自定义字符串变量



描述：新建的自定义字符串变量，默认值为nil，建议赋值后再使用。可以通过变量选择下拉框修改现在选中的变量的名称或删除该变量。

返回值：变量的值。

设置自定义字符串变量的值



描述：设置指定字符串变量。

参数：

- 选择要修改的变量的名称。
- 修改后的值，直接输入字符串。

建立一个数组



单击可新建一个自定义数组，数组名称必须以字母开头，不能使用空格等特殊字符。创建至少一个数组后，积木列表中会出现下述数组相关积木。

自定义数组



描述：新建的自定义数组，默认为空数组，建议赋值后再使用。在积木列表中右键（PC端）/长按（移动端）积木可以修改数组的名称或删除数组。也可以通过其他数组积木中数组选择下拉框修改现在选中的数组的名称或删除该数组。数组积木前的勾选框暂无作用，可忽略。

返回值：数组的值。

将变量加入数组



描述：将变量加入指定数组。新加入的变量会成为数组的最后一项。

参数：

- 要添加的变量，可以直接填写或使用其他椭圆形积木。
- 选择要修改的数组。

删除数组指定项

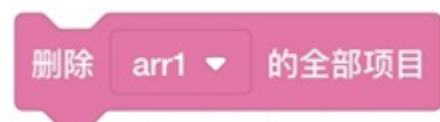


描述：删除指定数组的指定项。

参数：

- 选择要修改的数组。
- 要删除的项的编号，可以直接填写或使用其他返回值为数值的椭圆形积木。

删除数组全部项



描述：删除指定数组的全部项。

参数：选择要修改的数组。

在数组中插入项



描述：在数组中的指定位置插入变量。

参数：

- 选择要修改的数组。
- 插入的位置，可以直接填写或使用其他返回值为数值的椭圆形积木。
- 要添加的变量，可以直接填写或使用其他椭圆形积木。

替换数组中的指定项



描述：将数组中的指定项替换为指定变量。

参数：

- 选择要修改的数组。
- 要替换的项的编号，可以直接填写或使用其他返回值为数值的椭圆形积木。
- 替换后的变量，可以直接填写或使用其他椭圆形积木。

获取数组中的指定项



描述：获取数组中指定项的值。

参数：

- 选择要获取项的数组。
- 要获取的项的编号，可以直接填写或使用其他返回值为数值的椭圆形积木。

返回值：指定项的值。

获取数组中的总项数



描述：获取数组中项目的总数。

参数：选择要获取项的数组。

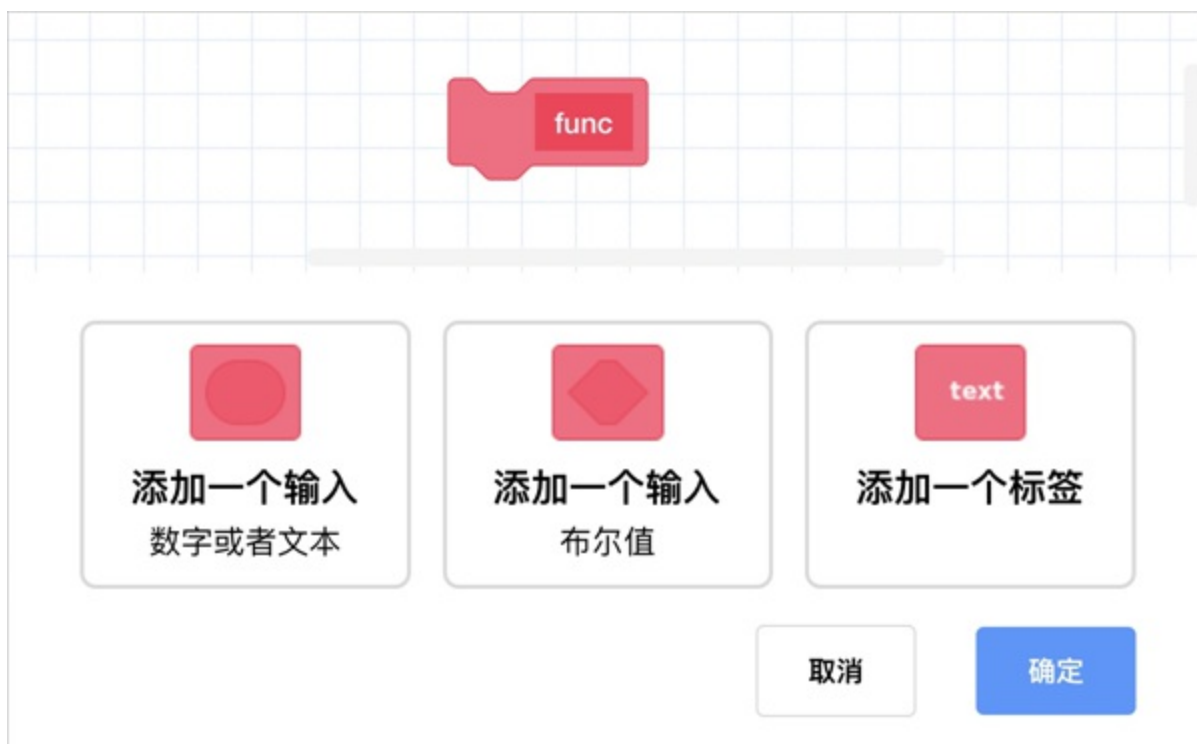
返回值：指定数组的项目总数。

新建一个函数



单击可新建一个函数。函数是一个固定的程序段，您可以将一组实现特定常用功能的积木定义为一个函数，之后每次要用到该功能时只需调用这个函数，不需要再重复搭建相同积木组。新建一个函数需要声明并定义该函数。新建函数成功后，积木列表中会出现对应的函数积木。

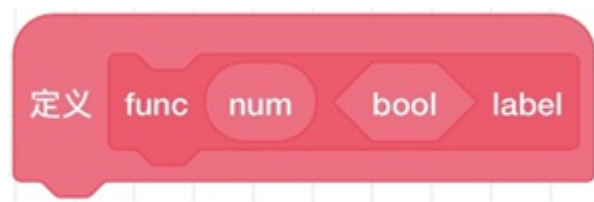
1. 声明函数



在这个界面，需要定义函数的名称，输入（参数）的类型、数量与名称。函数和参数名称不能包含空格等特殊字符。您还可以给函数添加标签，标签可以作为注释使用，对函数或输入进行补充说明。

1. 定义函数

完成函数声明后，编程区域内会出现该函数的定义头积木。



您需要在该积木下方接续积木进行编程，定义该函数的功能。

定义头中的输入可拖出来在下方的积木中使用，表示使用实际调用该函数时的输入作为参数。

自定义函数



描述：用户自定义的函数积木，名称与输入参数由用户自定义，用于调用已定义的函数。在积木列表中右键（PC端）/长按（移动端）积木可以修改该函数的声明。如果需要删除该函数，请删除该函数的定义头积木。

新建一个子程序

新建一个子程序

单击可新建一个子程序。图形编程支持嵌套调用子程序，子程序支持图形编程和脚本编程，最大嵌套两层。新建子程序成功后，积木列表中会出现对应的子程序积木。



- 选择积木编程后页面变为子程序积木编程页面，可以设置子程序描述和编写子程序。



- 选择脚本编程后弹出子程序脚本编程窗口，可以设置子程序描述和编写子程序。



子程序

- 图形化编程子程序



- 脚本编程子程序



描述：子程序积木，描述由用户在创建子程序时定义，用于调用已保存的子程序。在积木列表中右键（PC端）/长按（移动端）积木可以修改或删除该子程序。

IO积木组

IO积木组用于管理机械臂IO端子的输入输出。输入输出端口的取值范围由机械臂的对应端子数量决定，请参考对应机械臂的硬件手册。

获取数字输入



描述：获取指定DI的状态。

参数：

- 选择DI端子的位置，包括控制柜（底座）和末端。
- 选择DI端子的编号。

返回值：指定DI的状态，0表示关，1表示开。

等待数字输入



描述：等待指定的DI满足条件或等待超时后再执行后续的积木指令。

参数：

- 选择DI端子的位置，包括控制柜和末端。
- 选择DI端子的编号。
- 选择要等待的状态（开或者关）。
- 等待超时时间，如果设置为0，则一直等到满足条件。

设置数字输出



描述：设置指定DO的开启或关闭。

参数：

- 选择DO端子的位置，包括控制柜（底座）和末端。

- 选择DO端子的编号。
- 选择要输出的状态（开或者关）。

设置数字输出（子程序用）



描述：设置指定DO的开启或关闭。在子程序中设置DO请使用此积木。

参数：

- 选择DO端子的位置，包括控制柜和末端。
- 选择DO端子的编号。
- 选择要输出的状态（开或者关）。

设置一组数字输出



描述：设置一组DO。将积木拖进编程区后单击进行设置。

参数：

设置一组数字输出

×

分配控制柜DO索引:

—

+

DO_01	ON
DO_02	ON
DO_03	ON

取消

保存

- 通过“+”或“-”可以增加或减少要设置的DO数量。

- 选择DO端子的编号。
- 选择要输出的状态（开或者关）。

运动积木组

运动积木组用于控制机械臂运动及进行运动相关设置。

控制运动的积木均为异步指令，即下发命令成功后就会执行下一条指令，不会等待机器人运动完成，机器人会按照下发顺序逐条执行指令。如果需要等待已下发的指令都执行完成后再执行后续指令，可使用同步指令。

点位参数可以在该工程的“示教点”页面添加后在此处选择；控制运动的积木中还支持将默认的变量积木拖出来，使用其他的返回值为点位笛卡尔坐标值的椭圆形积木替代。

高级配置

高级配置

当其他的运动积木无法满足编程需求时，可通过高级配置方式创建控制机器人运动的积木。创建成功的积木会直接出现在编程区域。详情请参考[运动高级配置](#)。

运动至目标点



描述：控制机械臂从当前位置运动到指定点。积木拖动到编程区后，双击可进行高级配置，详情请参考[运动高级配置](#)。

参数：

- 选择运动方式，支持关节运动和直线运动。关节运动的轨迹非直线，所有关节会同时完成运动。
- 目标点。

运动至目标点（带偏移）



描述：控制机械臂从当前位置运动到指定点偏移后的坐标，目标点可设置为当前点。

参数：

- 选择运动方式，支持相对关节运动和相对直线运动。
- 目标点。
- 笛卡尔坐标系下相对于目标点的X轴、Y轴、Z轴（单位：毫米）和R轴（单位：度）方向上的偏移量。

门型运动



描述：控制机械臂从当前位置以门型方式运动至笛卡尔坐标系下的目标位置。

1. 机械臂会先垂直抬高指定高度，然后过渡至最大高度。
2. 机械臂以直线方式向目标点运动。
3. 运动至目标点附近时，过渡至目标点上方的指定高度，然后垂直下降至目标点。

参数：

- 目标点。
- 起始点抬升高度，单位：毫米。
- 结束点下降高度，单位：毫米。
- 最大抬升高度，单位：毫米。

门型运动（使用预设门型参数）



描述：控制机械臂从当前位置以门型方式运动至笛卡尔坐标系下的目标位置（使用预设门型参数）。

参数：

- 目标点。
- 选择门型参数的索引值，对应参数需要在设置中的门型参数设置页面设置并使能。

进行整圆运动



描述：控制机械臂从当前位置进行整圆插补运动，运动指定圈数后重新回到当前位置。当前位置的坐标必须不在中间点和结束点确定的直线上。

参数：

- 中间点指用于确定整圆的中间点。
- 结束点指用于确定整圆的结束点。
- 输入进行整圆运动的圈数，取值范围1~999。

进行圆弧运动

圆弧运动：中间 点 P1 ▼ 结束 点 P1 ▼

描述：控制机械臂从当前位置以圆弧插补方式运动至笛卡尔坐标系下的指定点。当前位置的坐标必须不在中间点和结束点确定的直线上。

参数：

- 中间点指用于确定圆弧的中间点。
- 结束点为目标点。

控制扩展轴运动

扩展轴运动：运动角度/距离 20 速度比例 50 加速度比例 50 同步 ▼

描述：需要安装并在工艺中配置了扩展轴后才可使用。控制扩展轴进行运动。

参数：

- 设置运动的角度或者距离。该参数的含义取决于扩展轴工艺中高级设置里设置的运动类型（关节/直线）。单位：类型为关节时为度，类型为直线时为毫米。
- 设置运动时的速度比例。
- 设置运动时的加速度比例。
- 设置指令的模式：
 - 同步：运动完成后再执行下一条指令。
 - 异步：下发指令后，不等待运动完成直接执行下一条指令。

如果在循环语句中调用该指令，建议设置为同步，保证每次拓展轴运动完成后再下发后续指令。如果设置为异步，可能会导致拓展轴运动异常。

设置关节加速度比例

设置关节加速度百分比 10 %

描述：设置关节运动的加速度比例。机械臂实际运动加速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

参数：关节加速度比例，取值范围：0~100。

设置关节速度比例

设置关节速度百分比 10 %

描述：设置关节运动的速度比例。机械臂实际运动速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

参数：关节速度比例，取值范围：0~100。

设置直线加速度比例

设置直线加速度百分比 10 %

描述：设置直线运动的加速度比例。机械臂实际运动加速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

参数：直线加速度比例，取值范围：0~100。

设置直线速度比例

设置直线速度百分比 10 %

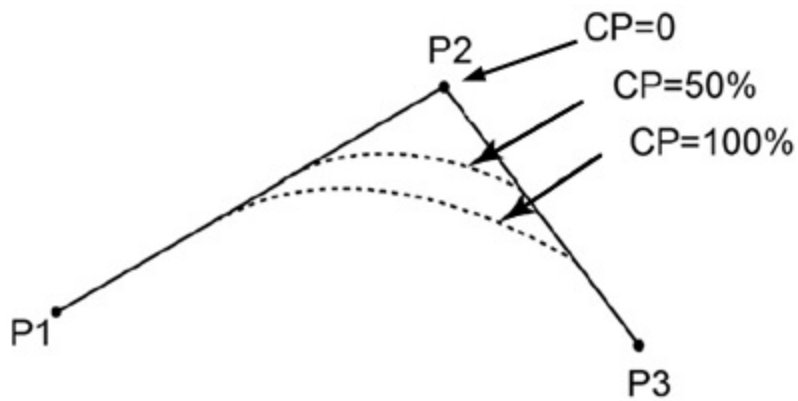
描述：设置直线运动的速度比例。机械臂实际运动速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

参数：直线速度比例，取值范围：0~100。

设置平滑过渡比例

设置平滑过渡百分比 10 %

描述：设置运动中的平滑过渡比例，即机械臂从起始点经过中间点到达终点时，经过中间点是以直角方式过渡还是以曲线方式过渡，如下图所示。



参数：平滑过渡比例，取值范围：0~100。

同步指令

同步指令

描述：程序运行到这条指令时，会等待机械臂将之前已下发的指令全部执行完成，再继续执行后续指令。

运动高级配置

设置面板

名称

新点位

运动方式

MovJ

MovL

Jump

JointMovJ

RelMovJ

RelMovL

Arc

Circle

参数配置

P点坐标:

P1

自定义

高级设置

取消

保存

通过高级配置方式创建控制机器人运动的积木。配置的内容包括积木名称，运动方式和运动参数。不同的运动方式需配置的运动参数不同。不同的运动方式需配置的运动参数不同。其中，机械臂实际运动速度/加速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

MovJ

运动方式：从当前位置以关节插补方式运动至笛卡尔坐标系下的目标位置。

运动方式

MovJ

MovL

Jump

JointMovJ

RelMovJ

RelMovL

Arc

Circle



基础设置： P点坐标，即目标点的坐标。可在该工程的“示教点”页面添加后在此处选择，也可直接在此页面示教进行自定义。

参数配置

P点坐标:

P1

▼

自定义

x

268.616

z

-9.725

User

0

y

-0.358

r

-0.120

Tool

0

获取坐标

高级设置：

勾选并配置需要设置的高级参数。

- 速度(Speed)：运动速度比例，取值范围：1~100。
- 加速度(Accel)：运动加速度比例，取值范围：1~100。
- 平滑过渡(CP)：运动中的平滑过渡比例，详见本章最后的“平滑过渡（CP）说明”。取值范围：0~100。
- 过程I/O设置：运动到指定距离或百分比时，触发指定DO。距离为正时，表示离起点的距离，距离为负时，表示离目标点的距离。单击下方的“+”可以添加过程IO，单击右侧的“-”可以删除对应过程IO。

高级设置

☐ 速度(Speed)

☐ 加速度(Accel)

☐ 平滑过渡(CP)

☐ 过程I/O设置

DO_01



=

关



触发模式

距离



距离

0

mm



MovL

运动方式：从当前位置以直线插补方式运动至笛卡尔坐标系下的目标位置。

运动方式

MovJ

MovL

Jump

JointMovJ

RelMovJ

RelMovL

Arc

Circle



基础设置： P点坐标，即目标点的坐标。可在该工程的“示教点”页面添加后在此处选择，也可直接在此页面示教进行自定义。

参数配置

P点坐标: P1 自定义

x	268.616	z	-9.725	User	0
y	-0.358	r	-0.120	Tool	0

获取坐标

高级设置：

勾选并配置需要设置的高级参数。

- 速度(Speed)：运动速度比例，取值范围：1~100。
- 加速度(Accel)：运动加速度比例，取值范围：1~100。
- 平滑过渡(CP)：运动中的平滑过渡比例，详见本章最后的“平滑过渡（CP）说明”。取值范围：0~100。
- 过程I/O设置：运动到指定距离或百分比时，触发指定DO。距离为正时，表示离起点的距离，距离为负时，表示离目标点的距离。单击下方的“+”可以添加过程IO，单击右侧的“-”可以删除对应过程IO。

高级设置

☐ 速度(Speed)

☐ 加速度(Accel)

☐ 平滑过渡(CP)

☐ 过程I/O设置

?

DO_01

▼

=

关

▼

+

触发模式

距离

▼

距离

0

mm

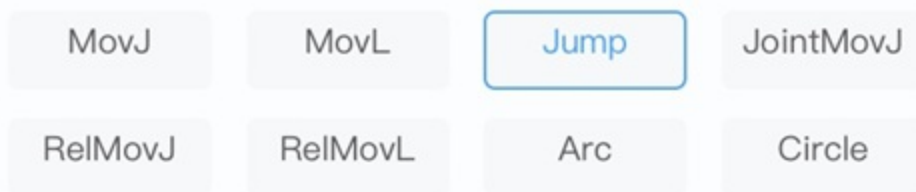
+

Jump

运动方式：从当前位置以门型方式运动至笛卡尔坐标系下的目标位置。

1. 机械臂会先垂直抬高指定高度，然后过渡至最大高度。
2. 机械臂以直线方式向目标点运动。
3. 运动至目标点附近时，过渡至目标点上方的指定高度，然后垂直下降至目标点。

运动方式



基础设置:

- P点坐标: 目标点的坐标。可在该工程的“示教点”页面添加后在此处选择, 也可直接在此页面示教进行自定义。
- 抬高高度h1: 起始点抬高高度。
- 下降高度h2: 结束点下降高度。
- 最大高度z_limit: 最大抬高高度。三个高度的关系可参考上方运动方式示意图。

参数配置

P点坐标:

P1

自定义

抬高高度h1

10

mm

下降高度h2

20

mm

最大高度z_limit

100

mm

高级设置:

勾选并配置需要设置的高级参数。

- 速度(Speed): 运动速度比例, 取值范围: 1~100。
- 加速度(Accel): 运动加速度比例, 取值范围: 1~100。

高级设置

☐ 速度(Speed)

☐ 加速度(Accel)

JointMovJ

运动方式：从当前位置以关节插补方式运动至目标关节角度。

运动方式

MovJ MovL Jump **JointMovJ**

RelMovJ RelMovL Arc Circle



基础设置：P点坐标，即目标点的坐标。可在该工程的“示教点”页面添加后在此处选择，也可直接在此页面示教进行自定义。

P点坐标:

P1



自定义

j1 -0.076

j3 46.350

j2 12.692

j4 -0.044

获取坐标

高级设置：

勾选并配置需要设置的高级参数。

- 速度(Speed): 运动速度比例, 取值范围: 1~100。
- 加速度(Accel): 运动加速度比例, 取值范围: 1~100。
- 平滑过渡(CP): 运动中的平滑过渡比例, 详见本章最后的“平滑过渡 (CP) 说明”。取值范围: 0~100。

高级设置

☐ 速度(Speed)

☐ 加速度(Accel)

☐ 平滑过渡(CP)

RelMovJ

运动方式: 从当前位置以关节插补方式运动至笛卡尔坐标系下的偏移位置。

运动方式

MovJ

MovL

Jump

JointMovJ

RelMovJ

RelMovL

Arc

Circle



基础设置: 笛卡尔坐标系下X轴、Y轴、Z轴方向上的偏移量, 单位: 毫米。

参数配置

偏移量

ΔX

mm

ΔZ

mm

ΔY

mm

ΔR

mm

高级设置：

勾选并配置需要设置的高级参数。

- 速度(Speed)：运动速度比例，取值范围：1~100。
- 加速度(Accel)：运动加速度比例，取值范围：1~100。
- 平滑过渡(CP)：运动中的平滑过渡比例，详见本章最后的“平滑过渡（CP）说明”。取值范围：0~100。

高级设置

☐ 速度(Speed)

☐ 加速度(Accel)

☐ 平滑过渡(CP)

RelMovL

运动方式：从当前位置以直线插补方式运动至笛卡尔坐标系下的偏移位置。

 运动方式

MovJ

MovL

Jump

JointMovJ

RelMovJ

RelMovL

Arc

Circle



基础设置：笛卡尔坐标系下X轴、Y轴、Z轴方向上的偏移量，单位：毫米。

参数配置

偏移量

ΔX	0	mm	ΔZ	0	mm
ΔY	0	mm	ΔR	0	mm

高级设置：

勾选并配置需要设置的高级参数。

- 速度(Speed)：运动速度比例，取值范围：1~100。
- 加速度(Accel)：运动加速度比例，取值范围：1~100。
- 平滑过渡(CP)：运动中的平滑过渡比例，详见本章最后的“平滑过渡（CP）说明”。取值范围：0~100。

高级设置

☐ 速度(Speed)	<input type="range" value="1"/>
☐ 加速度(Accel)	<input type="range" value="1"/>
☐ 平滑过渡(CP)	<input type="range" value="0"/>

Arc

运动方式：从当前位置以圆弧插补方式运动至笛卡尔坐标系下的指定点。当前位置的坐标必须不在AB点确定的直线上。

运动方式



基础设置:

- 中间点A坐标：圆弧中间点的坐标。
- 结束点B坐标：目标点的坐标。两个点都可在该工程的“示教点”页面添加后在此处选择，或者直接在此页面示教进行自定义。

参数配置

中间点A坐标:	P1 ▼	自定义
结束点B坐标:	P1 ▼	自定义

高级设置:

勾选并配置需要设置的高级参数。

- 速度(Speed)：运动速度比例，取值范围：1~100。
- 加速度(Accel)：运动加速度比例，取值范围：1~100。
- 平滑过渡(CP)：运动中的平滑过渡比例，详见本章最后的“平滑过渡（CP）说明”。取值范围：0~100。

高级设置

☐ 速度(Speed)

☐ 加速度(Accel)

☐ 平滑过渡(CP)

Circle

运动方式：从当前位置进行整圆插补运动，运动指定圈数后重新回到当前位置。当前位置的坐标必须不在AB点确定的直线上，且三个点确定的整圆不能超出机械臂的运动范围。

运动方式

MovJ

MovL

Jump

JointMovJ

RelMovJ

RelMovL

Arc

Circle



基础设置：

- 中间点A坐标：用于确定整圆的中间点的坐标。
- 结束点B坐标：用于确定整圆的结束点的坐标。两个点都可在该工程的“示教点”页面添加后在此处选择，或者直接在此页面示教进行自定义。
- 循环次数：进行整圆运动的圈数。

参数配置

中间点A坐标:

P1



自定义

结束点B坐标:

P1



自定义

循环次数:

1

高级设置:

勾选并配置需要设置的高级参数。

- 速度(Speed): 运动速度比例, 取值范围: 1~100。
- 加速度(Accel): 运动加速度比例, 取值范围: 1~100。
- 平滑过渡(CP): 运动中的平滑过渡比例, 详见本章最后的“平滑过渡 (CP) 说明”。取值范围: 0~100。

高级设置



☐ 速度(Speed)



☐ 加速度(Accel)

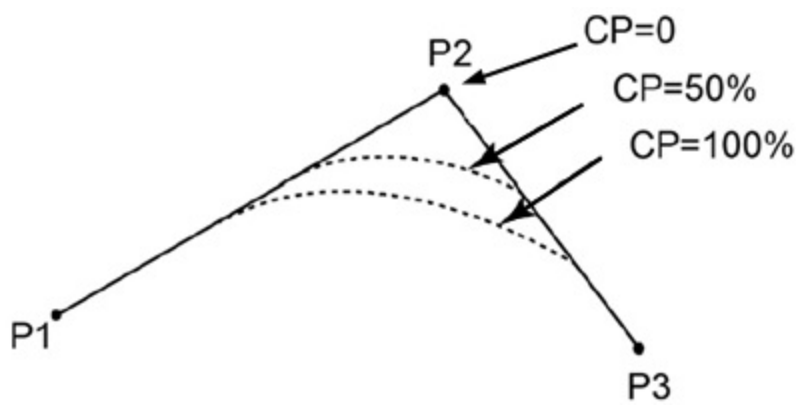


☐ 平滑过渡(CP)



平滑过渡 (CP) 说明

平滑过渡指机械臂从起始点经过中间点到达终点时, 经过中间点是以直角方式过渡还是以曲线方式过渡, 如下图所示。



位姿积木组

位姿积木组用于进行位姿相关操作。

获取当前位姿的笛卡尔坐标

获取当前笛卡尔位置的值

描述：获取机械臂当前位姿的笛卡尔坐标值。

返回值：当前位姿的笛卡尔坐标值。

获取当前位姿的笛卡尔坐标指定维度的值

获取当前笛卡尔位置的 X ▼ 值

描述：获取机械臂当前位姿的笛卡尔坐标系下指定维度的值。

参数：要获取值的维度。

返回值：当前位姿的笛卡尔坐标系下指定维度的值。

获取当前位姿的关节坐标

获取当前关节位置的值

描述：获取机械臂当前位姿的关节坐标值。

返回值：当前位姿的关节坐标值。

获取当前位姿的指定关节的值

获取当前关节位置的 J1 ▼ 值

描述：获取机械臂当前位姿的关节坐标系下指定关节的值。

参数：要获取值的关节。

返回值：当前位姿的关节坐标系下指定关节的值。

获取指定位置偏移后的坐标值

笛卡尔位置偏移：从位置 偏移 ΔX 0 ΔY 0 ΔZ 0 ΔR 0

描述：获取指定的位置偏移指定坐标后的笛卡尔坐标值。

参数：

- 偏移的初始位置，使用返回值为笛卡尔坐标值的椭圆形积木。
- 各个坐标轴上的偏移量。

返回值：偏移后的笛卡尔坐标值。

自定义笛卡尔坐标点

自定义笛卡尔点位 X 0 Y 0 Z 0 R 0 User 0 Tool 0

描述：自定义笛卡尔坐标系下的点位。

参数：

- 自定义点位在各个坐标轴上的值。
- 点位所属的用户坐标系索引值。
- 点位所属的工具坐标系索引值。

返回值：点位的笛卡尔坐标值。

获取指定点的坐标值

点 P1 ▼

描述：获取指定点的笛卡尔坐标值。

参数：选择要获取坐标值的点。

返回值：指定点的笛卡尔坐标值。

获取指定点的指定坐标维度的值

获取 P1 ▼ 的 X ▼ 值

描述：获取指定点的指定笛卡尔坐标维度的值。

参数：

- 选择要获取坐标值的点。
- 选择要获取的坐标维度。

返回值：指定点的指定笛卡尔坐标维度的值。

修改指定点的坐标值



描述：修改指定点的指定笛卡尔坐标维度的值。

参数：

- 选择要修改的点。
- 选择要修改的坐标维度。
- 选择修改后的值。

Modbus积木组

Modbus积木组用于进行Modbus通信相关操作。

创建Modbus主站



描述：创建Modbus的主站，并和从站建立连接。

参数：

- Modbus从站的IP地址。
- Modbus从站的端口。
- Modbus从站的ID，取值范围：1 ~ 4。

获取创建主站结果



描述：获取创建Modbus主站的结果。

返回值：创建成功返回0，创建失败返回1。

等待输入寄存器



描述：等待输入寄存器指定地址的值满足条件后再继续执行下一条指令。

参数：

- 输入寄存器起始地址，取值范围：0 ~ 4095。
- 选择要读取的数据类型：
 - U16：16位无符号整数（2个字节，占用1个寄存器）
 - U32：32位无符号整数（4个字节，占用2个寄存器）
 - F32：32位单精度浮点数（4个字节，占用2个寄存器）
 - F64：64位双精度浮点数（8个字节，占用4个寄存器）

- 输入寄存器指定地址的值要满足的条件。

等待保持寄存器

该界面用于配置等待保持寄存器的参数。它包含三个主要部分：地址输入框，显示为“0”；类型选择下拉菜单，当前选择为“U16”；以及为（值）输入框，显示为“50”。

等待保持寄存器 地址 0 类型 U16 ▼ 为 50

描述：等待保持寄存器指定地址的值满足条件后再继续执行下一条指令。

参数：

- 保持寄存器起始地址，取值范围：0 ~ 4095。
- 选择要读取的数据类型：
 - U16：16位无符号整数（2个字节，占用1个寄存器）
 - U32：32位无符号整数（4个字节，占用2个寄存器）
 - F32：32位单精度浮点数（4个字节，占用2个寄存器）
 - F64：64位双精度浮点数（8个字节，占用4个寄存器）
- 输入寄存器指定地址的值要满足的条件。

等待触点寄存器

该界面用于配置等待触点寄存器的参数。它包含两个主要部分：地址输入框，显示为“0”；以及为（值）选择下拉菜单，当前选择为“1”。

等待触点寄存器 地址 0 为 1 ▼

描述：等待触点寄存器指定地址的值满足条件后再继续执行下一条指令。

参数：

- 触点寄存器起始地址，取值范围：0 ~ 4095。
- 触点寄存器指定地址的值要满足的条件。

等待线圈寄存器

该界面用于配置等待线圈寄存器的参数。它包含两个主要部分：地址输入框，显示为“0”；以及为（值）选择下拉菜单，当前选择为“1”。

等待线圈寄存器 地址 0 为 1 ▼

描述：等待线圈寄存器指定地址的值满足条件后再继续执行下一条指令。

参数：

- 线圈寄存器起始地址，取值范围：0 ~ 4095。
- 线圈寄存器指定地址的值要满足的条件。

读取输入寄存器

获取输入寄存器 地址 类型

描述：获取输入寄存器指定地址的值。

参数：

- 输入寄存器起始地址，取值范围：0 ~ 4095。
- 选择要读取的数据类型：
 - U16：16位无符号整数（2个字节，占用1个寄存器）
 - U32：32位无符号整数（4个字节，占用2个寄存器）
 - F32：32位单精度浮点数（4个字节，占用2个寄存器）
 - F64：64位双精度浮点数（8个字节，占用4个寄存器）

返回值：输入寄存器指定地址的值。

读取保持寄存器

获取保持寄存器 地址 类型

描述：获取保持寄存器指定地址的值。

参数：

- 保持寄存器起始地址，取值范围：0 ~ 4095。
- 选择要读取的数据类型：
 - U16：16位无符号整数（2个字节，占用1个寄存器）
 - U32：32位无符号整数（4个字节，占用2个寄存器）
 - F32：32位单精度浮点数（4个字节，占用2个寄存器）
 - F64：64位双精度浮点数（8个字节，占用4个寄存器）

返回值：输入寄存器指定地址的值。

读取触点寄存器

获取触点寄存器 地址

描述：获取触点寄存器指定地址的值。

参数：触点寄存器起始地址，取值范围：0 ~ 4095。

返回值：触点寄存器指定地址的值。

读取线圈寄存器

获取线圈寄存器 地址 0

描述：获取线圈寄存器指定地址的值。

参数：线圈寄存器起始地址，取值范围：0 ~ 4095。

返回值：线圈寄存器指定地址的值。

连续读取线圈寄存器

获取线圈寄存器数组 地址 0 位数 1

描述：连续读取线圈寄存器指定地址的值。

参数：

- 线圈寄存器起始地址，取值范围：0 ~ 4095。
- 连续读取的寄存器位数。

返回值：线圈寄存器指定地址的值，存储在table中。table中第一个值对应线圈寄存器起始地址的值。

连续读取保持寄存器

获取保持寄存器数组 地址 0 位数 1 类型 U16 ▼

描述：连续读取保持寄存器指定地址的值。

参数：

- 保持寄存器起始地址，取值范围：0 ~ 4095。
- 连续读取的值的数量。
- 选择要读取的数据类型：
 - U16：16位无符号整数（2个字节，占用1个寄存器）
 - U32：32位无符号整数（4个字节，占用2个寄存器）
 - F32：32位单精度浮点数（4个字节，占用2个寄存器）
 - F64：64位双精度浮点数（8个字节，占用4个寄存器）

返回值：线圈寄存器指定地址的值，存储在table中。table中第一个值对应线圈寄存器起始地址的值。

写入线圈寄存器



描述：将指定的值写入线圈寄存器指定的地址。

参数：

- 线圈寄存器起始地址，取值范围：6 ~ 4095。
- 选择写入的值，只能是0或1。

连续写入线圈寄存器



描述：将指定的值连续写入线圈寄存器指定的地址。

参数：

- 线圈寄存器起始地址，取值范围：0 ~ 4095。
- 要写入的值的位数。
- 输入写入的值，填写一个数组，长度与写入的位数相同，每一位只能是0或者1。

写入保持寄存器



描述：将指定的值写入保持寄存器指定的地址。

参数：

- 保持寄存器起始地址，取值范围：0 ~ 4095。
- 选择写入的值，需要与选择的数据类型对应。
- 选择要写入的数据类型：
 - U16：16位无符号整数（2个字节，占用1个寄存器）
 - U32：32位无符号整数（4个字节，占用2个寄存器）
 - F32：32位单精度浮点数（4个字节，占用2个寄存器）
 - F64：64位双精度浮点数（8个字节，占用4个寄存器）

关闭主站



描述： 关闭Modbus主站，断开与所有从站的连接。

TCP积木组

TCP积木组用于进行TCP通信相关的操作。

打开SOCKET

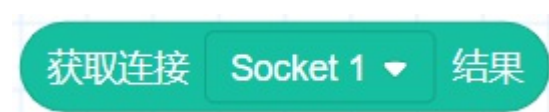


描述：创建TCP客户端，和指定的TCP服务端进行通信。

参数：

- 选择SOCKET编号，最多建立4条TCP通信链路。
- TCP服务端的IP地址。
- TCP服务端的端口。

获取连接SOCKET结果



描述：获取TCP通信连接结果。

参数：选择SOCKET编号。

返回值：连接成功返回0，连接失败返回1。

创建SOCKET



描述：创建TCP服务端，等待客户端进行连接。

参数：

- 选择SOCKET编号，最多建立4条TCP通信链路。
- TCP服务端的IP地址。
- TCP服务端的端口。机械臂作为服务端时，请勿使用下述已被系统占用的端口：

22, 23, 502 (0~1024之间的端口是linux系统自定义端口, 被占用可能性高, 请尽量避免使用),

5000~5004, 6000, 8080, 11000, 11740, 22000, 22002, 29999, 30003, 30004, 60000, 65500~65515

获取创建SOCKET结果

获取创建 Socket 1 ▼ 结果

描述: 获取TCP服务端创建结果。

参数: 选择SOCKET编号。

返回值: 创建成功返回0, 创建失败返回1。

关闭SOCKET

关闭 Socket 1 ▼

描述: 关闭指定的SOCKET, 断开该通信链路。

参数: 选择SOCKET编号。

获取变量

获取变量 Socket 1 ▼ 类型: string ▼ 名称: 等待时间 0 s

描述: 通过TCP通信获取变量并保存。

参数:

- 选择SOCKET编号。
- 选择要接收的变量的类型, 支持字符串和数值。
- 用于保存接收到的数据的变量, 使用已创建的变量积木。
- 填写等待超时时间, 如果设置为0, 则不会超时, 会一直等待直到接收到变量。

发送变量



描述：通过TCP通信发送变量。

参数：

- 选择SOCKET编号。
- 发送的数据，可以直接填写或者使用其他的返回值为字符串或数值的椭圆形积木。

获取发送变量结果



描述：获取TCP发送变量的结果。

参数：选择SOCKET编号。

返回值：发送成功返回0，发送失败返回1。

附录C 脚本编程函数说明

- C.1 Lua基础语法
 - C.1.1 变量与数据类型
 - C.1.2 运算符
 - C.1.3 流程控制
- C.2 函数说明
 - C.2.1 运动指令
 - C.2.2 运动参数
 - C.2.3 相对运动指令
 - C.2.4 IO
 - C.2.5 TCP/UDP
 - C.2.6 Modbus
 - C.2.7 程序控制
 - C.2.8 高级指令_视觉

Lua基础语法

- 变量与数据类型
- 运算符
- 流程控制

变量与数据类型

如果您想系统学习Lua编程相关知识，请自行在网络上搜索Lua教程。本手册仅列举部分Lua基础语法，便于您快速查阅。

变量用于存储值，将值作为参数传递或结果返回。变量通过"="进行赋值。

Lua中的变量默认为全局变量，除非使用 `local` 显式声明为局部变量。局部变量的作用域为从声明位置开始到所在语句块结束。

```
a = 5          -- 全局变量
local b = 5     -- 局部变量
```

变量名称可以是任意非数字打头的字母、下划线和数字组成的字符串，Lua保留的关键字除外。

Lua 变量不要类型定义，只需要为变量赋值，Lua会根据值自动判断变量的类型。

Lua支持多种数据类型，较为常见的包括数字（number），布尔值（boolean），字符串（string）和表（table）。Lua中的数组是table的一种。

Lua中还有一种特殊的数据类型为nil，nil 表示空（没有任何有效值），例如打印一个没有赋值的变量，便会输出一个 nil 值。

数字

Lua中的number为双精度类型的实浮点数，支持各种运算，以下写法均被视为number：

- 2
- 2.2
- 0.2
- 2e+1
- 0.2e-1
- 7.8263692594256e-06

布尔值

boolean 类型只有两个可选值：true（真）和 false（假），Lua 把 false 和 nil 看作是 false，其他的都为 true，数字 0 也是 true。

字符串

string是由数字、字母、下划线组成的一串字符。string可以使用以下三种方式来表示：

- 单引号间的一串字符。
- 双引号间的一串字符。
- [[与]] 间的一串字符。

在对一个数字字符串上进行算术操作时，Lua 会尝试将这个数字字符串转成一个数字。

Lua 提供了很多的方法来支持字符串的操作：

方法	说明
string.upper (argument)	字符串全部转为大写字母
string.lower (argument)	字符串全部转为小写字母
string.gsub(mainString, findString, replaceString, num)	在字符串中替换。mainString 为要操作的字符串，findString 为被替换的字符，replaceString 要替换的字符，num 替换次数（可以忽略，则全部替换）
string.find (str, substr, [init, [end]])	在一个指定的目标字符串 str 中搜索指定的内容 substr，如果找到了一个匹配的子串，就会返回这个子串的起始索引和结束索引，不存在则返回 nil。
string.reverse(arg)	字符串反转
string.format(...)	返回一个类似printf的格式化字符串
string.char(arg) 和 string.byte(arg[,int])	char 将整型数字转成字符并连接， byte 转换字符为整数值(可以指定某个字符，默认第一个字符)
string.len(arg)	计算字符串长度
string.rep(string, n)	返回字符串string的n个拷贝
..	链接两个字符串
string.gmatch(str, pattern)	回一个迭代器函数，每一次调用这个函数，返回一个在字符串 str 找到的下一个符合 pattern 描述的子串。如果参数 pattern 描述的字符串没有找到，迭代函数返回nil
string.match(str, pattern, init)	string.match()只寻找源字符串str中的第一个配对. 参数init可选, 指定搜寻过程的起点, 默认为1。在成功配对时, 函数将返回配对表达式中的所有捕获结果; 如果没有设置捕获标记, 则返回整个配对字符串. 当没有成功的配对时, 返回nil
string.sub(s, i [, j])	用于截取字符串。s为要截取的字符串，i为截取开始位置，j为截取结束位置，默认为-1，最后一个字符。

示例：

```
str = "Lua"
print(string.upper(str))      --字符串全部转为大写字母，打印结果：LUA
print(string.lower(str))     --字符串全部转为小写字母，打印结果：lua
print(string.reverse(str))   --字符串反转，打印结果：aUL
print(string.len("abc"))     --计算字符串abc的长度，打印结果：3
print(string.format("the value is: %d",4))  --打印结果：the value is:4
print(string.rep(str,2))     --字符串复制2次，打印结果：LuaLua
string1 = "cn."
string2 = "dobot"
```

```

string3 = ".cc"
print("连接字符串", string1..string2..string3) -- 使用..进行字符串连接, 打印结果: cn.dobot.cc

string1 = [[aaaa]]
print(string.gsub(string1, "a", "z", 3)) -- 在字符串中替换, 打印结果: zzza

print(string.find("Hello Lua user", "Lua", 1)) -- 在字符串中搜索Lua, 返回子串的起始索引和
结束索引, 打印结果: 7, 9

sourcestr = "prefix--runoobgoogletaobao--suffix"
sub = string.sub(sourcestr, 1, 8) -- 取字符串前缀, 第1个到第8个
print("\n截取", string.format("%q", sub)) -- 打印结果: 截取 "prefix--"

```

表

table是一组带索引的数据。

- 最简单的构造函数是{}，用来创建一个空表。可以直接初始化表。
- table实际为关联型数组，可以用任意类型的值来作数组的索引，但这个值不能是 nil。
- table是不固定大小的，可以根据自己需要进行扩容。
- 通过#符号可以获取表的长度。

```

tbl = {[1] = 2, [2] = 6, [3] = 34, [4] = 5}
print("tbl 长度 ", #tbl) -- 打印结果为4

```

Lua 提供了很多的方法来支持表的操作：

方法	说明
table.concat (table [, sep [, start [, end]])	concat是concatenate(连锁, 连接)的缩写. table.concat()函数列出参数中指定table的数组部分从start位置到end位置的所有元素, 元素间以指定的分隔符(sep)隔开
table.insert (table, [pos,] value)	在table的数组部分指定位置(pos)插入值为value的一个元素. pos参数可选, 默认为表的末尾
table.remove (table [, pos])	返回table数组部分位于pos位置的元素. 其后的元素会被前移. pos参数可选, 默认为table长度, 即从最后一个元素删起
table.sort (table [, comp])	对给定的table进行升序排序

示例1：

```

fruits = {} -- 初始化表
fruits = {"banana", "orange", "apple"} -- 为表赋值

print("连接后的字符串 ", table.concat(fruits, ", ", 2, 3)) -- 指定索引来连接table, 连接后的字符串 o

```

```

range, apple

-- 在末尾插入
table.insert(fruits,"mango")
print("索引为 4 的元素为 ",fruits[4])    --打印结果: 索引为4的元素为 mango

-- 在索引为 2 的键处插入
table.insert(fruits,2,"grapes")
print("索引为 2 的元素为 ",fruits[2])    --打印结果: 索引为2的元素为 grapes

print("最后一个元素为 ",fruits[5])        --打印结果: 最后一个元素为 mango
table.remove(fruits)
print("移除后最后一个元素为 ",fruits[5])  --打印结果: 移除后最后一个元素为 nil

```

示例2:

```

fruits = {"banana","orange","apple","grapes"}
print("排序前")
for k,v in ipairs(fruits)
do
    print(k,v)        --打印结果: banana orange apple grapes
end
--进行升序排序
table.sort(fruits)
print("排序后")
for k,v in ipairs(fruits) do
    print(k,v)        --打印结果: apple banana grapes orange
end

```

数组

数组，就是相同数据类型的元素按一定顺序排列的集合，可以是一维数组和 multidimensional array。Lua 数组的索引键值可以使用整数表示，数组的大小不是固定的。

- 一维数组：最简单的数组，其逻辑结构是线性表。
- 多维数组：即数组中包含数组或一维数组的索引键对应一个数组。

示例1：一维数组可以用for循环出数组中的元素。使用整数索引来访问数组元素，如果索引没有值则返回nil。

```

array = {"Lua", "Tutorial"} --创建一维数组
for i= 0, 2 do
    print(array[i])        --打印结果分别为: nil Lua Tutorial
end

```

在 Lua 索引值是以 1 为起始，但也可以指定 0 开始。除此外还可以以负数为数组索引值。

```

array = {}
for i= -2, 2 do
    array[i] = i*2+1        --为一维数组赋值
end

```

```

end
for i = -2,2 do
    print(array[i])          --打印结果分别为: -3 -1 1 3 5
end

```

示例2：一个三行三列的阵列多维数组

```

-- 初始化数组
array = {}
for i=1,3 do
    array[i] = {}
    for j=1,3 do
        array[i][j] = i*j
    end
end

-- 访问数组
for i=1,3 do
    for j=1,3 do
        print(array[i][j])    --打印结果分别为: 1 2 3 2 4 6 3 6 9
    end
end
end

```

运算符

算术运算符

指令符号	说明
+	加法运算
-	减法运算
*	乘法运算
/	浮点除法运算
//	向下取整除法运算
%	取余除法运算
^	指数运算
&	按位与运算
\	按位或运算
~	按位异或运算
<<	按位左移运算
>>	按位右移运算

示例：

```
a=20
b=5
print(a+b)      --打印a加b的结果：25
print(a-b)      --打印a减b的结果：15
print(a*b)      --打印a乘b的结果：100
print(a/b)      --打印a除以b的结果：4
print(a//b)     --打印a整除b的结果：4
print(a%b)      --打印a除以b的余数结果：0
print(a^b)      --打印a的b次幂的结果：3200000
print(a&b)      --打印a和b按位与的结果：4
print(a|b)      --打印a和b按位或的结果：21
print(a~b)      --打印a异或b的结果：17
print(a<<b)     --打印a左移b个单位的结果：640
print(a>>b)     --打印a右移b个单位的结果：0
```

关系运算符

指令符号	说明

==	等于
~=	不等于
<=	小于等于
>=	大于等于
<	小于
>	大于

示例：

```

a=20          --创建变量a
b=5           --创建变量b
print(a==b)   --打印a等于b的对比结果：false
print(a~=b)   --打印a不等于b的对比结果：true
print(a<=b)   --打印a小于等于b的对比结果：false
print(a>=b)   --打印a大于等于b的对比结果：true
print(a<b)    --打印a小于b的对比结果：false
print(a>b)    --打印a大于b的对比结果：true

```

逻辑运算符

指令符号	说明
and	逻辑与，两侧均为true，其结果才为true，只要有一侧为false，其结果即为false
or	逻辑或，一侧结果为true，其结果即为true，如or两侧均为false，则结果为false
not	逻辑非为将判断结果直接取反

```

a=true
b=false
print(a and b)    --真与假，结果为假
print(a or b)     --真或假，结果为真
print(20 > 5 not true) --真与非真，等于真与假，最后结果为假

```

流程控制

指令符号	说明
if...then... elseif...then... else...end	if条件判断指令。从上到下依次判断条件是否成立，如果某个判断为 true，执行完对应的代码块，后面的条件判断直接忽略，不再执行
while...do...end	while循环控制指令。在条件为 true 时，让程序重复地执行某些语句。执行语句前会先检查条件是否为 true
for...do...end	for循环控制指令，重复执行指定语句，重复次数可在 for 语句中控制
repeat... until()	repeat循环控制指令。重复执行循环，直到 指定的条件为真时为止

示例：

1、if条件判断指令

```
a = 100;
b = 200;
--[ 检查条件 --]
if(a == 100)
then
    --[if条件为true时执行以下if条件判断--]
    if(b == 200)
    then
        --[if条件为true时执行该语句块--]
        print("a 的值为 :", a );
        print("b 的值为 :", b );
    end
end
end
```

2、while循环控制指令

```
a=10
while( a < 20 )
do
    print("a 的值为:", a)
    a = a+1
end
```

3、for循环控制指令

```
for i=10,1,-1 do
    print(i)
end
```

4、repeat循环控制指令

```
a = 10
repeat
  print("a的值为:", a)
  a = a + 1
until(a > 15)
```

函数说明

- [运动指令](#)
- [运动参数](#)
- [相对运动指令](#)
- [IO](#)
- [TCP/UDP](#)
- [Modbus](#)
- [程序控制](#)
- [视觉](#)

运动指令

运动指令函数用于控制机械臂进行运动。运动速度比例/加速度比例还可以在[运动参数](#)中设置，如果两处都设置了，以运动指令的值为准。机械臂实际运动速度/加速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

MovJ

原型：

```
MovJ(P, {CP=1, SpeedJ=50, AccJ=20, SYNC=0})
```

描述：

从当前位置以点到点（关节运动）方式运动至笛卡尔坐标系下的目标位置。关节运动的轨迹非直线，所有关节会同时完成运动。

必选参数：

P：表示目标点，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持笛卡尔坐标点位。

可选参数：

- CP：运动时设置平滑过渡（详见[运动参数](#)中的CP指令），取值范围：0~100。
- SpeedJ：运动速度比例，取值范围：1~100。
- AccJ：运动加速度比例，取值范围：1~100。
- SYNC：同步标识，取值范围：0或1。默认值为0。
SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
SYNC = 1表示同步执行，调用后，待指令执行完才返回。

示例：

```
MovJ(P1)
```

机械臂以默认设置点对点运动至P1点。

MovL

原型：

```
MovL(P, {CP=1, SpeedL=50, AccL=20, SYNC=0})
```

描述:

从当前位置以直线方式运动至笛卡尔坐标系下的目标位置。

必选参数:

P: 表示目标点, 可在该工程的“示教点”页面添加后在此引用, 也可自定义点位, 但只支持笛卡尔坐标点位。

可选参数:

- CP: 运动时设置平滑过渡 (详见[运动参数](#)中的CP指令), 取值范围: 0~100。
- SpeedL: 运动速度比例, 取值范围: 1~100。
- AccL: 运动加速度比例, 取值范围: 1~100。
- SYNC: 同步标识, 取值范围: 0或1。默认值为0。
SYNC = 0表示异步执行, 调用后立即返回, 但不关注指令执行情况;
SYNC = 1表示同步执行, 调用后, 待指令执行完才返回。

示例:

```
MovL(P1)
```

机械臂以默认设置直线运动至P1点。

Jump

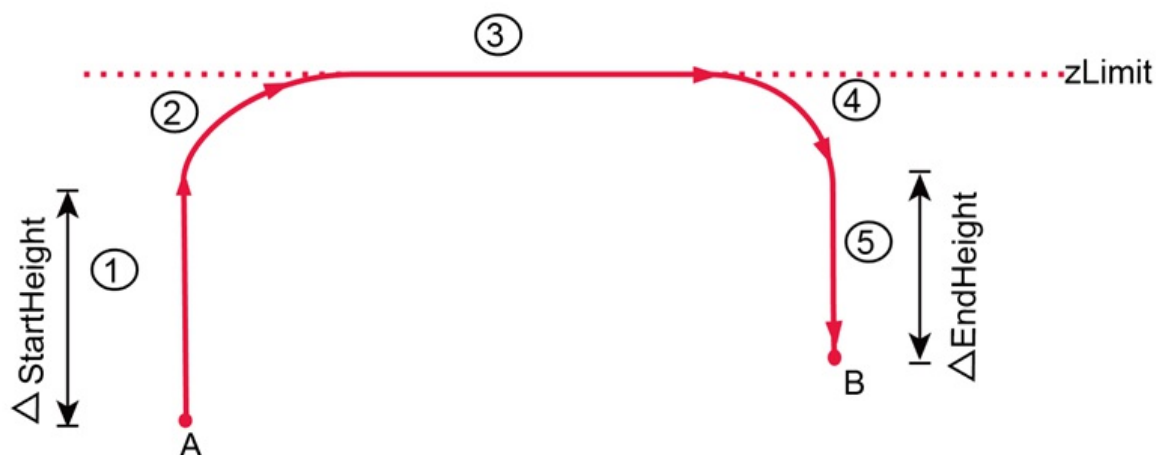
原型:

```
Jump(P,{SpeedL=50, AccL=20, Start=10, ZLimit=100, End=20, SYNC=0})  
Jump(P,{SpeedL=50, AccL=20, Arch=1, SYNC=0})
```

描述:

从当前位置以门型运动方式移动至笛卡尔坐标系下的目标位置。

1. 机械臂会先垂直抬高指定高度。
2. 过渡至最大高度。
3. 以直线方式向目标点运动。
4. 运动至目标点附近时, 过渡至目标点上方的指定高度。
5. 垂直下降至目标点。



必选参数:

P: 表示目标点, 可在该工程的“示教点”页面添加后在此引用, 也可自定义点位, 但只支持笛卡尔坐标点位。P点高度不能高于ZLimit。

可选参数:

- SpeedL: 运动速度比例, 取值范围: 1~100。
- Accl: 运动加速度比例, 取值范围: 1~100。
- Start: 起始抬升高度。
- ZLimit: 最大抬升高度。
- End: 结束点下降高度。
- Arch: 门型参数索引, 在控制软件中设置。
- SYNC: 同步标识, 取值范围: 0或1。默认值为0。
 SYNC = 0表示异步执行, 调用后立即返回, 但不关注指令执行情况;
 SYNC = 1表示同步执行, 调用后, 待指令执行完才返回。

示例:

```
MovJ(P4)
Jump(P5,{Start=10 ZLimit=600 End=10})
```

机械臂运动至P4, 然后门型运动至P5。

JointMovJ

原型:

```
JointMovJ(P,{CP=1, SpeedJ=50, AccJ=20, SYNC=0})
```

描述:

从当前位置以点到点（关节运动）方式运动至目标关节角度。

必选参数:

P: 表示目标点, 只能通过关节角度定义。

可选参数:

- CP: 运动时设置平滑过渡 (详见[运动参数](#)中的CP指令), 取值范围: 0~100。
- SpeedJ: 运动速度比例, 取值范围: 1~100。
- AccJ: 运动加速度比例, 取值范围: 1~100。
- SYNC: 同步标识, 取值范围: 0或1。默认值为0。
SYNC = 0表示异步执行, 调用后立即返回, 但不关注指令执行情况;
SYNC = 1表示同步执行, 调用后, 待指令执行完才返回。

示例:

```
local P = {joint={0,-0.0674194,0,0}}  
JointMovJ(P)
```

自定义关节坐标点P, 机械臂以默认设置运动至P点。

Circle

原型:

```
Circle(P1,P2,Count,{CP=1, SpeedL=50, AccL=20, SYNC=0})
```

描述:

从当前位置进行整圆插补运动, 运动指定圈数后重新回到当前位置。需要通过当前位置, P1, P2三个点确定一个整圆, 因此当前位置不能在P1和P2确定的直线上, 且三个点确定的整圆不能超出机械臂的运动范围。

必选参数:

- P1: 表示整圆中间点, 可在该工程的“示教点”页面添加后在此引用, 也可自定义点位, 但只支持笛卡尔坐标点位。
- P2: 表示整圆结束点, 可在该工程的“示教点”页面添加后在此引用, 也可自定义点位, 但只支持笛卡尔坐标点位。
- Count: 进行整圆运动的圈数, 取值范围1~999。

可选参数:

- CP: 运动时设置平滑过渡 (详见[运动参数](#)中的CP指令), 取值范围: 0~100。
- SpeedL: 运动速度比例, 取值范围: 1~100。
- AccL: 运动加速度比例, 取值范围: 1~100。
- SYNC: 同步标识, 取值范围: 0或1。默认值为0。

SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
SYNC = 1表示同步执行，调用后，待指令执行完才返回。

示例：

```
MovJ(P1)  
Circle(P2,P3,1)
```

机械臂运动至P1，然后进行沿着P1，P2，P3确定的整圆运动一圈。

Arc

原型：

```
Arc(P1,P2,{CP=1, SpeedL=50, AccL=20, SYNC=0})
```

描述：

从当前位置以圆弧插补方式运动至笛卡尔坐标系下的指定点。需要通过当前位置，P1，P2三个点确定一个圆弧，因此当前位置不能在P1和P2确定的直线上。

必选参数：

- P1：表示圆弧中间点，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持笛卡尔坐标点位。
- P2：表示目标点，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持笛卡尔坐标点位。

可选参数：

- CP：运动时设置平滑过渡（详见[运动参数](#)中的CP指令），取值范围：0~100。
- SpeedL：运动速度比例，取值范围：1~100。
- AccL：运动加速度比例，取值范围：1~100。
- SYNC：同步标识，取值范围：0或1。默认值为0。

SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
SYNC = 1表示同步执行，调用后，待指令执行完才返回。

示例：

```
MovJ(P1)  
Arc(P2,P3)
```

机械臂运动至P1，然后经由P2圆弧运动至P3。

MovJIO

原型:

```
MovJIO(P, { {Mode, Distance, Index, Status},{Mode, Distance, Index, Status}...},{CP=1, SpeedJ=50, AccJ=20, SYNC=0})
```

描述:

从当前位置以点到点（关节运动）方式运动至笛卡尔坐标系下的目标位置，运动时并行设置数字输出端口状态。

必选参数:

- P: 表示目标点，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持笛卡尔坐标点位。
- 并行数字输出参数：设置当机械臂运动到指定距离或百分比时，触发指定DO。可设置多组，每组含有以下参数：
 - Mode: 触发模式。0表示距离百分比，1表示距离数值。
 - Distance: 指定距离。
 - Distance为正数时，表示离起点的距离。
 - Distance为负数时，表示离目标点的距离。
 - Mode为0时，Distance表示和总距离的百分比，取值1~100。
 - Mode为1时，Distance表示距离的值，单位：毫米。
 - Index: DO端子的编号。
 - Status: 要设置的DO状态，0表示关，1表示开。

可选参数:

- CP: 运动时设置平滑过渡（详见[运动参数](#)中的CP指令），取值范围：0~100。
- Speed: 运动速度比例，取值范围：1~100。
- Accel: 运动加速度比例，取值范围：1~100。
- SYNC: 同步标识，取值范围：0或1。默认值为0。
SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
SYNC = 1表示同步执行，调用后，待指令执行完才返回。

示例:

```
MovJIO(P1, { {0, 10, 1, 1} })
```

机械臂以默认设置向P1点运动，当运动到距离起点10%的位置时，将DO1设置为开。

MovLIO

原型:

```
MovLIO(P, { {Mode, Distance, Index, Status},{Mode, Distance, Index, Status}...},{CP=1, SpeedL=
```

```
50, AccL=20, SYNC=0})
```

描述:

从当前位置以直线方式运动至笛卡尔坐标系下的目标位置，运动时并行设置数字输出端口状态。

必选参数:

- P: 表示目标点，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持笛卡尔坐标点位。
- 并行数字输出参数：设置当机械臂运动到指定距离或百分比时，触发指定DO。可设置多组，每组含有以下参数：
 - Mode: 触发模式。0表示距离百分比，1表示距离数值。
 - Distance: 指定距离。
 - Distance为正数时，表示离起点的距离。
 - Distance为负数时，表示离目标点的距离。
 - Mode为0时，Distance表示和总距离的百分比，取值1~100。
 - Mode为1时，Distance表示距离的值，单位：毫米。
 - Index: DO端子的编号。
 - Status: 要设置的DO状态，0表示关，1表示开。

可选参数:

- CP: 运动时设置平滑过渡（详见[运动参数](#)中的CP指令），取值范围：0~100。
- SpeedL: 运动速度比例，取值范围：1~100。
- AccL: 运动加速度比例，取值范围：1~100。
- SYNC: 同步标识，取值范围：0或1。默认值为0。
SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
SYNC = 1表示同步执行，调用后，待指令执行完才返回。

示例:

```
MovLIO(P1, { {0, 10, 1, 1} })
```

机械臂以默认设置向P1点运动，当运动到距离起点10%的位置时，将DO1设置为开。

MovJExt

原型:

```
MovJExt(AD, {SpeedE=50, AccE=20, SYNC=0})
```

描述:

扩展轴运动到目标角度或位置。

必选参数:

AD: 运动的角度或者距离。该参数的含义取决于扩展轴工艺中高级设置里设置的运动类型（关节/直线）。单位：类型为关节时为度，类型为直线时为毫米。

可选参数:

- SpeedE: 运动速度比例，取值范围：1~100。
- AccE: 运动加速度比例，取值范围：1~100。
- SYNC: 同步标识，取值范围：0或1。默认值为0。
SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
SYNC = 1表示同步执行，调用后，待指令执行完才返回。

如果在循环语句中调用该指令，建议设置为1，保证每次拓展轴运动完成后再下发后续指令。
如果设置为0，可能会导致拓展轴运动异常。

示例:

```
MovJExt(20, {SYNC=1})
```

扩展轴运动到指定位置20。

运动参数

运动参数函数用于设置或获取机械臂运动相关参数。

Sync

原型：

```
Sync()
```

描述：

阻塞程序执行队列指令，待所有队列指令执行完才返回，执行后续指令。一般用于等待机械臂完成运动。

示例：

```
MovJ(P1)  
MovJ(P2)  
Sync()
```

待机械臂先运动至P1，再运动到P2后才返回，执行后续指令。

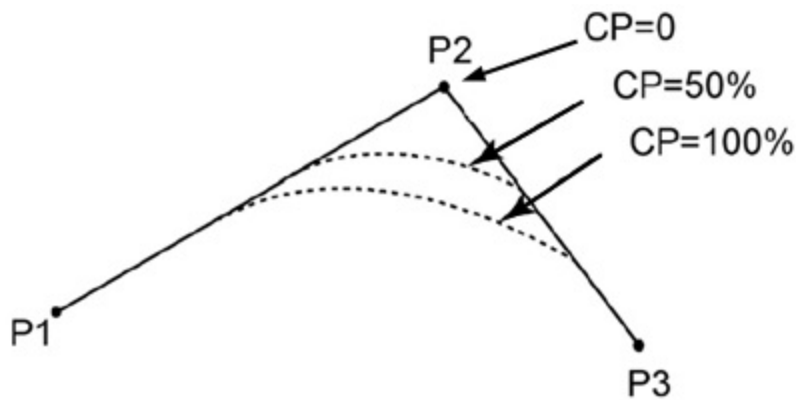
CP

原型：

```
CP(R)
```

描述：

设置平滑过渡比例，即机械臂连续运动经过多个点时，经过中间点是以直角方式过渡还是以曲线方式过渡。



必选参数:

R: 平滑过渡比例。取值范围: 0~100。

示例:

```
CP(50)
MovL(P1)
MovL(P2)
MovL(P3)
```

机械臂从P1运动至P3点经过P2点时以50%平滑过渡。

SpeedJ

原型:

```
SpeedJ(R)
```

描述:

设置关节运动方式的速度比例。机械臂实际运动速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

必选参数:

R: 速度比例。取值范围: 0~100。

示例:

```
SpeedJ(20)
MovJ(P1)
```

机械臂以软件设置速度的20%的速度比例运动至P1点。

AccJ

原型:

```
AccJ(R)
```

描述:

设置关节运动方式的加速度比例。机械臂实际运动加速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

必选参数:

R: 加速度比例。取值范围: 0~100。

示例:

```
AccJ(50)  
MovJ(P1)
```

机械臂以软件设置速度的50%的加速度比例运动至P1点。

SpeedL

原型:

```
SpeedL(R)
```

描述:

设置直线和弧线运动方式的速度比例。机械臂实际运动速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

必选参数:

R: 速度比例。取值范围: 0~100。

示例:

```
SpeedL(20)  
MovL(P1)
```

机械臂以软件设置速度的20%的速度比例运动至P1点。

AccL

原型:

```
AccL(R)
```

描述:

设置直线和弧线运动方式的加速度比例。机械臂实际运动加速度比例为参数设置的比例乘以控制软件再现设置中的值再乘以全局速率。

必选参数:

R: 加速度比例。取值范围: 0~100。

示例:

```
AccL(50)  
MovL(P1)
```

机械臂以软件设置速度的50%的加速度比例运动至P1点。

GetPose

原型:

```
GetPose()
```

描述:

获取笛卡尔坐标系下机械臂的实时位姿。如果设置了用户坐标系或工具坐标系，则获取的位姿为当前坐标系下的位姿。

返回:

机械臂当前位姿的笛卡尔坐标值。

示例:

```
local currentPose = GetPose()  
MovJ(P1)  
MovJ(currentPose)
```

机械臂先运动至P1，再返回当前位姿。

GetAngle

原型:

```
GetAngle()
```

描述:

获取关节坐标系下机械臂的实时位姿。

返回:

机械臂当前位姿的关节坐标值。

示例:

```
local currentAngle = GetAngle()  
MovJ(P1)  
JointMovJ(currentAngle)
```

机械臂先运动至P1，再返回当前位姿。

相对运动指令

运动指令函数用于控制机械臂进行偏移运动。运动速度比例/加速度比例还可以在[运动参数](#)中设置，如果两处都设置了，以运动指令的值为准。机械臂实际运动速度/加速度比例为参数设置的比列乘以控制软件再现设置中的值再乘以全局速率。

RelJoint

原型：

```
RelJoint(P, {Offset1, Offset2, Offset3, Offset4})
```

描述：

对指定点位在关节坐标系下增加J1~J4轴偏移量并返回一个新的关节坐标点。

必选参数：

- 偏移前的点位，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持关节坐标点位。
- Offset1~Offset4：关节坐标系下J1轴 ~ J4轴方向上的偏移值，单位：度。

返回：

偏移后的关节坐标点。

示例：

```
JointMovJ(RelJoint(P1, {60,50,32,30}))
```

将P1在J1~J4轴上分别偏移一定角度，然后运动至偏移后的点位。

RelPoint

原型：

```
RelPoint(P, {OffsetX, OffsetY, OffsetZ, OffsetR})
```

描述：

对指定点位在笛卡尔坐标系下增加X、Y、Z、R方向上的偏移量并返回一个新的笛卡尔坐标点。

必选参数：

- 偏移前的点位，可在该工程的“示教点”页面添加后在此引用，也可自定义点位，但只支持笛卡尔坐标点位。
- OffsetX, OffsetY, OffsetZ, OffsetR: 笛卡尔坐标系下X、Y、Z、R方向上的偏移量，单位：毫米(X、Y、Z)，度(R)。

返回：

偏移后的笛卡尔坐标点。

示例：

```
MovJ(RelPoint(P1, {30,50,10,0}))
```

将P1在X、Y、Z轴上分别偏移一定距离，然后运动至偏移后的点位。

RelMovJ

原型：

```
RelMovJ({OffsetX, OffsetY, OffsetZ, OffsetR}, {CP=1, SpeedJ=50, AccJ=20, SYNC=0})
```

描述：

从当前位置以点到点（关节运动）方式运动至笛卡尔坐标系下的偏移位置。关节运动的轨迹非直线，所有关节会同时完成运动。

必选参数：

OffsetX, OffsetY, OffsetZ, OffsetR: 笛卡尔坐标系下X、Y、Z、R方向上的偏移量，单位：毫米(X、Y、Z)，度(R)。

可选参数：

- CP: 运动时设置平滑过渡（详见[运动参数](#)中的CP指令），取值范围：0~100。
- SpeedJ: 运动速度比例，取值范围：1~100。
- AccJ: 运动加速度比例，取值范围：1~100。
- SYNC: 同步标识，取值范围：0或1。默认值为0。
 SYNC = 0表示异步执行，调用后立即返回，但不关注指令执行情况；
 SYNC = 1表示同步执行，调用后，待指令执行完才返回。

示例：

```
RelMovJ({10,10,10,0})
```

机械臂以默认设置点对点运动至偏移点。

RelMovL

原型:

```
RelMovL({OffsetX, OffsetY, OffsetZ, OffsetR}, {CP=1, SpeedL=50, AccL=20, SYNC=0})
```

描述:

从当前位置以直线方式运动至笛卡尔坐标系下的偏移位置。

必选参数:

OffsetX, OffsetY, OffsetZ, OffsetR: 笛卡尔坐标系下X、Y、Z、R方向上的偏移, 单位: 毫米 (X、Y、Z), 度 (R)。

可选参数:

- CP: 运动时设置平滑过渡 (详见[运动参数](#)中的CP指令), 取值范围: 0~100。
- SpeedL: 运动速度比例, 取值范围: 1~100。
- AccL: 运动加速度比例, 取值范围: 1~100。
- SYNC: 同步标识, 取值范围: 0或1。默认值为0。
SYNC = 0表示异步执行, 调用后立即返回, 但不关注指令执行情况;
SYNC = 1表示同步执行, 调用后, 待指令执行完才返回。

示例:

```
RelMovL({10,10,10,0})
```

机械臂以默认设置直线运动至偏移点。

IO

IO指令用于进行机械臂系统IO的读写和相关参数的设置。

DI

原型：

```
DI(index)
```

描述：

读取数字输入端口状态。

必选参数：

index：DI端子的编号。

返回：

对应的DI端子的电平（ON/OFF）。

示例：

```
if (DI(1)==ON) then
    MovL(P1)
end
```

DI1为高电平（ON）时机械臂以直线运动方式运动至P1点。

DO

原型：

```
DO(index,ON|OFF)
```

描述：

设置数字输出端口状态。

必选参数：

- index：DO端子的编号。
- ON|OF：要设置的DO端口状态，ON：高电平；OFF：低电平。

示例：

```
DO(1,ON)
```

将DO1设置为高电平（ON）。

DOInstant

原型：

```
DOExecute(index,ON|OFF)
```

描述：

无视当前指令队列，立即设置数字输出端口状态。

必选参数：

- index：DO端子的编号。
- ON|OF：要设置的DO端口状态，ON：高电平；OFF：低电平。

示例：

```
DOInstant(1,ON)
```

无视当前指令队列，立即将DO1设置为高电平（ON）。

ToolDI

原型：

```
ToolDI(index)
```

描述：

读取末端数字输入端口状态。

必选参数：

index：末端DI端子的编号。

返回：

对应的DI端子的电平（ON/OFF）。

示例：

```
if (ToolDI(1)==ON) then  
    MovL(P1)  
end
```

末端DI1为高电平（ON）时机械臂以直线运动方式运动至P1点。

TCP/UDP

TCP/UDP函数用于进行TCP或UDP通信。

TCPCreate

原型：

```
TCPCreate(isServer, IP, port)
```

描述：

创建TCP网络对象，仅可创建一个。

必选参数：

- isServer：是否创建服务端。true：表示创建服务端；false：表示创建客户端。
- IP：服务端IP地址。需与客户端IP地址在同一网段，且不冲突。创建服务端时为机械臂的IP地址，创建客户端时为对端的地址。
- port：服务端端口。机械臂作为服务端时，请勿使用下述已被系统占用的端口：

22, 23, 502 (0~1024之间的端口是linux系统自定义端口，被占用可能性高，请尽量避免使用)，

5000~5004, 6000, 8080, 11000, 11740, 22000, 22002, 29999, 30003, 30004, 60000, 65500~65515

返回：

- err：0表示创建TCP网络对象成功，1表示创建TCP网络对象失败。
- socket：创建的socket对象。

示例1：

```
local ip="192.168.1.6" -- 机械臂的IP地址作为服务端的IP地址
local port=6001 -- 服务端端口
local err=0
local socket=0
err, socket = TCPCreate(true, ip, port)
```

创建TCP服务端。

示例2：

```
local ip="192.168.1.25" -- 外部设备如相机的IP地址作为服务端的IP地址
local port=6001 -- 服务端端口
local err=0
local socket=0
err, socket = TCPCreate(false, ip, port)
```

创建TCP客户端。

TCPStart

原型：

```
TCPStart(socket, timeout)
```

描述：

建立TCP连接。机械臂作为服务端时，等待客户端来连接；机械臂作为客户端时，主动连接服务端。

必选参数：

- socket：已创建的socket对象。
- timeout：等待超时时间，单位：秒。如果设置为0，会一直等待到连接建立成功；如果不为0，则超过设定的时间后返回连接失败。

返回：

连接结果。

- 0：连接成功
- 1：输入参数错误
- 2：socket对象不存在
- 3：设置超时时间错误
- 4：连接失败

示例：

```
err = TCPStart(socket, 0)
```

开始建立TCP连接，一直等待直到连接建立成功。

TCPRead

原型：

```
TCPRead(socket, timeout, type)
```

描述:

接收TCP对端发送的数据。

必选参数:

socket: 已创建的socket对象。

可选参数:

- timeout: 等待超时时间, 单位: 秒。如果设置为0, 会一直等待直到读取完数据再往下执行; 如果不为0, 则超过设定的时间后会直接往下执行。
- type: 返回值类型。如果不设置, 则RecBuf缓存格式为table形式; 如果设置为 "string", 则RecBuf缓存为字符串。

返回:

- err: 0表示接收数据成功, 1表示接收数据失败。
- Recbuf: 接收数据缓存区。

示例:

```
err, RecBuf = TCPRead(socket, 0, "string") -- RecBuf数据类型为字符串
err, RecBuf = TCPRead(socket, 0) -- RecBuf数据类型为table
```

接收TCP数据, 接收的数据分别保存为字符串和table形式。

TCPWrite

原型:

```
TCPWrite(socket, buf, timeout)
```

描述:

发送数据给TCP对端。

必选参数:

- socket: 已创建的socket对象。
- buf: 要发送的数据。

可选参数:

timeout: 等待超时时间, 单位: 秒。如果设置为0, 会一直等待直到对端接收完数据再往下执行; 如果不为0, 则超过设定的时间后会直接往下执行。

返回：

发送结果。

- 0：发送成功。
- 1：发送失败

示例：

```
TCPWrite(socket, "test")
```

发送TCP数据，数据内容为“test”。

TCPDestroy

原型：

```
TCPDestroy(socket)
```

描述：

断开TCP连接并销毁socket对象。

必选参数：

socket：已创建的socket对象。

返回：

执行结果。

- 0：执行成功
- 1：执行失败

示例：

```
TCPDestroy(socket)
```

断开与TCP对端的连接。

UDPCreate

原型：

```
UDPCreate(isServer, IP, port)
```

描述:

创建UDP网络对象，仅可创建一个。

必选参数:

- isServer: 固定填写为false。
- IP: 对端IP地址。需与机械臂IP地址在同一网段，且不冲突。
- port: 对端端口。

返回:

- err: 0表示创建UDP网络对象成功，1表示创建UDP网络对象失败。
- socket: 创建的socket对象。

示例:

```
local ip="192.168.5.25" -- 外部设备如相机的IP地址作为对端的IP地址
local port=6001 -- 对端端口
local err=0
local socket=0
err, socket = UDPCreate(false, ip, port)
```

创建UDP网络对象。

UDPRead

原型:

```
UDPRead(socket, timeout, type)
```

描述:

接收UDP对端发送的数据。

必选参数:

socket: 已创建的socket对象。

可选参数:

- timeout: 等待超时时间，单位：秒。如果设置为0，会一直等待直到读取完数据再往下执行；如果不为0，则超过设定的时间后会直接往下执行。
- type: 返回值类型。如果不设置，则RecBuf缓存格式为table形式；如果设置为“string”，则RecBuf缓存为字符串。

返回:

- err: 0表示接收数据成功，1表示接收数据失败。
- Recbuf: 接收数据缓存区。

示例:

```
err, RecBuf = UDPRead(socket, 0, "string") -- RecBuf数据类型为字符串
err, RecBuf = UDPRead(socket, 0) -- RecBuf数据类型为table
```

接收UDP数据，接收的数据分别保存为字符串和table形式。

UDPWrite

原型:

```
UDPWrite(socket, buf, timeout)
```

描述:

发送数据给UDP对端。

必选参数:

- socket: 已创建的socket对象。
- buf: 要发送的数据。

可选参数:

timeout: 等待超时时间，单位：秒。如果设置为0，会一直等待直到对端接收完数据再往下执行；如果不为0，则超过设定的时间后会直接往下执行。

返回:

发送结果。

- 0: 发送成功。
- 1: 发送失败

示例:

```
UDPWrite(socket, "test")
```

发送UDP数据，数据内容为“test”。

Modbus

Modbus函数用于进行Modbus通信。

ModbusCreate

原型:

```
ModbusCreate(IP,port,slave_id)
```

描述:

创建Modbus主站，并和从站建立连接。

可选参数:

- IP: 从站IP地址。不指定或指定为“127.0.0.1”或“0.0.0.1”时，表示连接本机的Modbus从站。
- port: 从站端口。
- slave_id: 从站ID，取值范围0~4。
- isRTU: 如果为空或0，表示建立ModbusTCP通信。如果为1，表示建立ModbusRTU通信。

注意:

此参数决定了连接建立后传输数据使用的协议格式，并不影响连接结果。因此，如果创建主站时该参数设置错误，依然可以创建成功，但后续通讯时会导致异常。

返回:

- err:
 - 0: 创建Modbus主站成功
 - 1: 创建的主站已经有5个，创建新主站失败
 - 2: 初始化主站失败，建议检查IP，端口，网络是否正常等
 - 3: 连接从站失败，建议排查从站是否正常建立，网络是否正常等
- id: 主站索引，取值范围0~4。

示例1:

```
local ip="192.168.1.6" -- 从站的IP地址
local port=503 -- 从站端口
local err=0
local id=1
```

```
err, id = ModbusCreate(ip, port, 1)
```

创建Modbus主站，并和指定从站建立连接。

示例1：

以下指令都表示连接本机的Modbus从站。

```
ModbusCreate()
```

```
ModbusCreate("127.0.0.1")
```

```
ModbusCreate("0.0.0.1")
```

```
ModbusCreate("127.0.0.1", xxx,xxx) -- xxx 任意值
```

```
ModbusCreate("0.0.0.1", xxx,xxx) -- xxx 任意值
```

GetInBits

原型：

```
GetInBits(id, addr, count)
```

描述：

读取Modbus从站触点寄存器地址的值。

必选参数：

- id：创建Modbus主站时返回的主站索引。
- addr：触点寄存器起始地址地址。取值范围：0 ~ 4095
- count：连续读取触点寄存器的值的数量。

返回：

触点寄存器地址的值，存储在table中。table中第一个值对应触点寄存器起始地址的值。

示例：

```
inBits = GetInBits(id,0,5)
```

从地址0开始连续读取5个地址的值。

GetInRegs

原型:

```
GetInRegs(id, addr, count, type)
```

描述:

按照指定的数据类型，读取Modbus从站输入寄存器地址的值。

必选参数:

- id: 创建Modbus主站时返回的主站索引。
- addr: 输入寄存器起始地址地址。取值范围: 0 ~ 4095
- count: 连续读取输入寄存器的值的数量。

可选参数:

type: 数据类型。

- 如果为空，默认为U16
- U16: 16位无符号整数（2个字节，占用1个寄存器）
- U32: 32位无符号整数（4个字节，占用2个寄存器）
- F32: 32位单精度浮点数（4个字节，占用2个寄存器）
- F64: 64位双精度浮点数（8个字节，占用4个寄存器）

返回:

输入寄存器地址的值，存储在table中。table中第一个值对应输入寄存器起始地址的值。

示例:

```
data = GetInRegs(id, 2048, 1, "U32")
```

从地址2048开始读取一个32位无符号整数。

GetCoils

原型:

```
GetCoils(id, addr, count)
```

描述:

读取Modbus从站线圈寄存器地址的值。

必选参数:

- id: 创建Modbus主站时返回的主站索引。
- addr: 线圈寄存器起始地址地址。取值范围: 0 ~ 4095
- count: 连续读取线圈寄存器的值的数量。

返回:

线圈寄存器地址的值, 存储在table中。table中第一个值对应线圈寄存器起始地址的值。

示例:

```
Coils = GetCoils(id,0,5)
```

从地址0开始连续读取5个地址的值。

GetHoldRegs

原型:

```
GetHoldRegs(id, addr, count, type)
```

描述:

按照指定的数据类型, 读取Modbus从站保持寄存器地址的值。

必选参数:

- id: 创建Modbus主站时返回的主站索引。
- addr: 保持寄存器起始地址地址。取值范围: 0 ~ 4095
- count: 连续读取保持寄存器的值的数量。

可选参数:

type: 数据类型。

- 如果为空, 默认为U16
- U16: 16位无符号整数 (2个字节, 占用1个寄存器)
- U32: 32位无符号整数 (4个字节, 占用2个寄存器)
- F32: 32位单精度浮点数 (4个字节, 占用2个寄存器)
- F64: 64位双精度浮点数 (8个字节, 占用4个寄存器)

返回:

保持寄存器地址的值, 存储在table中。table中第一个值对应保持寄存器起始地址的值。

示例：

```
data = GetHoldRegs(id, 2048, 1, "U32")
```

从地址2048开始读取一个32位无符号整数。

SetCoils

原型：

```
SetCoils(id, addr, count, table)
```

描述：

将指定的值写入线圈寄存器指定的地址。

必选参数：

- id：创建Modbus主站时返回的主站索引。
- addr：线圈寄存器起始地址地址。取值范围：6 ~ 4095
- count：连续写入线圈寄存器的值的数量。
- table：要写入线圈寄存器的值，存储在table中。table中第一个值对应线圈寄存器起始地址的值。

示例：

```
local Coils = {0,1,1,1,0}  
SetCoils(id, 1024, #coils, Coils)
```

从地址1024开始，连续写5个线圈。

SetHoldRegs

原型：

```
SetHoldRegs(id, addr, count, table, type)
```

描述：

按照指定的数据类型，写入指定的值到保持寄存器指定的地址。

必选参数：

- id：创建Modbus主站时返回的主站索引。
- addr：保持寄存器起始地址地址。取值范围：0 ~ 4095

- count: 连续写入保持寄存器的值的数量。
- table: 要写入保持寄存器的值，存储在table中。table中第一个值对应保持寄存器起始地址的值。

可选参数:

type: 数据类型。

- 如果为空，默认为U16
- U16: 16位无符号整数（2个字节，占用1个寄存器）
- U32: 32位无符号整数（4个字节，占用2个寄存器）
- F32: 32位单精度浮点数（4个字节，占用2个寄存器）
- F64: 64位双精度浮点数（8个字节，占用4个寄存器）

示例:

```
local data = {95.32105}
SetHoldRegs(id, 2048, #data, data, "F64")
```

从地址2048开始，写入一个64位双精度浮点数。

ModbusClose

原型:

```
ModbusClose(id)
```

描述:

和Modbus从站断开连接。

可选参数:

id: 创建Modbus主站时返回的主站索引。

返回:

操作结果

- 0: 断开连接成功。
- 1: 断开连接失败。

示例:

```
ModbusClose(id)
```

和Modbus从站断开连接。

程序控制

程序控制函数为程序运行控制相关的通用函数。其中while,if,for为lua的流程控制指令，请参考[Lua基础语法 - 流程控制](#)。print为打印指令，用于输出信息到控制台。

Sleep

原型：

```
Sleep(time)
```

描述：

脚本延时执行下一条指令。

必选参数：

time：延时时间。单位：毫秒

示例：

```
DO(1,ON)  
Sleep(100)  
DO(1,OFF)
```

设置DO1为ON后等待100ms再设置DO1为OFF。

Wait

原型：

```
Wait(time)
```

描述：

延时下发运动指令或运动完成后延时下发下一条指令。

必选参数：

time：延时时间。单位：毫秒

示例：

```
DO(1,ON)
```

```
Wait(100)
MovJ(P1)
Wait(100)
DO(1,OFF)
```

设置DO1为ON后等待100ms机械臂再运动至P1，运行至P1点后延时100ms，再设置DO1为OFF。

Pause

原型：

```
Pause()
```

描述：

暂停脚本运行。需要通过控制软件或者远程控制操作才可继续运行。

示例：

```
MovJ(P1)
Pause()
MovJ(P2)
```

机械臂运行至P1点后暂停运行，通过外部控制继续运行后才会运行至P2点。

SetCollisionLevel

原型：

```
SetCollisionLevel(level)
```

描述：

设置碰撞检测等级。通过该接口设置的碰撞检测等级仅在工程运行期间生效，工程停止后会恢复为修改前的值。

必选参数：

level：碰撞检测等级，取值范围0~5，0表示关闭碰撞检测，1~5等级越高碰撞检测越灵敏。

示例：

```
SetCollisionLevel(2)
```

设置碰撞检测等级为2级。

ResetElapsedTime

原型:

```
ResetElapsedTime()
```

描述:

待此指令前所有指令执行完成后开始计时，需配合ElapsedTime()一起使用，可用于计算运行时间。

示例:

请参考ElapsedTime的示例。

ElapsedTime

原型:

```
ElapsedTime()
```

描述:

结束计时，返回时间差，需配合ElapsedTime()一起使用。

返回:

从开始计时到结束计时的时间差。

示例:

```
MovJ(P2)
ResetElapsedTime()
for i=1,10 do
  MovL(P1)
  MovL(P2)
end
print (ElapsedTime())
```

计算机械臂在P1和P2间来回运动10次的时间，打印至控制台。

Systime

原型:

```
Systime()
```

描述:

获取当前系统时间。

返回:

当前系统时间的Unix时间戳。

示例:

```
local time = Systeime()
```

获取系统当前时间并保存在变量time里。

SetPayload

原型:

```
SetPayload(payload, {x, y}, index)
```

描述:

设置末端负载重量、偏心坐标以及伺服参数序号。

必选参数:

- payload: 末端负载重量。取值范围: 0~1000, 单位: g
- {x,y}: 偏心坐标

可选参数:

index: 伺服参数序号。请在技术支持的指导下进行设置。

示例:

```
SetPayload(100, {0, 0})
```

设置末端负载重量为100g, 无偏心。

SetTool485

原型:

```
SetTool485(baud,parity,stopbit)
```

描述:

设置末端工具的RS485接口对应的数据格式。

必选参数：

- baud：RS485接口的波特率
- parity：是否有奇偶校验位。"O"表示奇校验，"E"表示偶校验，"N"表示无奇偶校验位。
- stopbit：停止位长度。取值范围：1, 2。

示例：

```
SetTool485(115200, "N", 1)
```

将末端工具的RS485接口对应的波特率设置为115200Hz，无奇偶校验位，停止位长度为1。

SetUser

原型：

```
SetUser(index, table)
```

描述：

修改指定的用户坐标系。

必选参数：

- index：已标定的用户坐标系序号。
- table：修改后的用户坐标系矩阵，格式为{x, y, z, r}。

示例：

```
SetUser(1, {10, 10, 10, 0})
```

修改用户坐标系1为X=10, Y=10, Z=10, R=0。

CalcUser

原型：

```
CalcUser(index, matrix_direction, table)
```

描述：

计算用户坐标系。

必选参数：

- index：已标定的用户坐标系序号。
- matrix_direction：计算的方向。
 - 1：左乘，表示index指定的坐标系沿基坐标系偏转table指定的值。
 - 0：右乘，表示index指定的坐标系沿自己偏转table指定的值。
- table：用户坐标系偏移值，格式为{x, y, z, r}。

返回：

计算得出的用户坐标系，格式为{x, y, z, r}。

示例：

```
-- 下述计算过程可等价于：一个初始位姿与用户坐标系1相同的坐标系，沿基坐标系平移{x=10, y=10, z=10}并沿R轴旋转10度后，得到的新坐标系为newUser。  
newUser = CalcUser(1,1,{10,10,10,10})
```

```
-- 下述计算过程可等价于：一个初始位姿与用户坐标系1相同的坐标系，沿用户坐标系1平移{x=10, y=10, z=10}并沿R轴旋转10度后，得到的新坐标系为newUser。  
newUser = CalcUser(1,0,{10,10,10,10})
```

SetTool

原型：

```
SetTool(index,table)
```

描述：

修改指定的工具坐标系。

必选参数：

- index：已标定的工具坐标系序号。
- table：修改后的工具坐标系矩阵，格式为{x, y, z, r}。

示例：

```
SetTool(1,{10,10,10,0})
```

修改工具坐标系1为X=10, Y=10, Z=10, R=0。

CalcTool

原型：

```
CalcTool(index,matrix_direction,table)
```

描述:

计算工具坐标系。

必选参数:

- index: 已标定的工具坐标系序号。
- matrix_direction: 计算的方向。
 - 1: 左乘, 表示index指定的坐标系沿法兰坐标系 (TCP0) 偏转table指定的值。
 - 0: 右乘, 表示index指定的坐标系沿自己偏转table指定的值。
- table: 工具坐标系, 格式为{x, y, z, r}。

返回:

计算得出的工具坐标系, 格式为{x, y, z, r}。

示例:

```
-- 下述计算过程可等价于: 一个初始位姿与工具坐标系1相同的坐标系, 沿法兰坐标系 (TCP0) 平移{x=10, y=10, z=10}并沿R轴旋转10度后, 得到的新坐标系为newTool。  
newTool = CalcTool(1,1,{10,10,10,10})
```

```
-- 下述计算过程可等价于: 一个初始位姿与工具坐标系1相同的坐标系, 沿工具坐标系1平移{x=10, y=10, z=10}并沿R轴旋转10度后, 得到的新坐标系为newTool。  
newTool = CalcTool(1,0,{10,10,10,10})
```

视觉

视觉工艺包用于配置相机通讯相关配置。相机固定于机器人工作范围内，位置固定，拍照视野范围固定，相机充当机器人的眼睛，采用以太网通讯或I/O触发方式与机器人进行数据交互。

相机的安装与配置方法根据相机不同而有所差异，本文不进行详述。

配置视觉工艺

单击视觉函数组右侧的“视觉配置”按钮开始配置相机。如果您是第一次配置相机工艺，需要先单击“新建”按钮，输入相机名称后新建一个相机配置，然后软件显示如下界面。

触发方式

设定触发相机的方式。

- IO内触发：相机与机器人的DO口连接，需根据电气接线端口配置相应的输出端口号。

- 网络触发：相机与机器人的以太网口连接，需配置网络触发的字符串。需与相机侧事先约定好，机器人通过网络发送该字符串给相机会触发相机。

网络基本参数

基本参数用于设置相机和机器人的通讯模式，包含以下几种模式：

- TCP_Client: TCP通讯，机器人作为客户端，相机作为服务器。需要配置相机的IP，端口和连接超时时间。
- TCP_Server: TCP通讯，机器人作为服务器，相机作为客户端。需要配置端口和连接超时时间

接收方式包括堵塞和非堵塞两种，请根据工程脚本设计选择。

- 堵塞：发送触发信号后，程序会在堵塞时间内一直停留在接收数据的那一行函数，直到接收相机发送过来的数据或者超过堵塞时间后才会继续运行；若堵塞时间设置为 0，程序会一直停留在数据接收函数那一行直到接收到视觉发送的数据。
- 非堵塞：发送触发信号后，不管有没有接收到相机发送来的数据，程序会继续执行。

网络接受格式表示相机发送过来的数据的格式，用于解析。如果当前默认的数据位不够，还可以通过“增加数据位”的方式来增加接收数据的长度，最多到可增加到8位：

NO,D1,D2,D3,D4,D5,D6,STA，其中，NO代表起始位模板号；STA代表结束位（状态位）。

可设置多种视觉数据格式，例如：

- 没有起始位和结束位：XX,YY,CC;
- 有起始位，没有结束位：NO,XX,YY,CC;
- 没有起始位，有结束位:XX,YY,CC,STA;
- 有起始位和结束位：NO,XX,YY,CC,STA。

配置完成后，单击右上角的“保存”。

InitCam

原型：

```
InitCam(CAM)
```

描述：

与指定的相机建立连接并初始化。

必选参数：

CAM：相机的名称，需要与配置视觉工艺时配置的相机名称一致。

返回：

初始化结果。

- 0：初始化成功
- 1：初始化失败

示例：

```
InitCam("CAM0")
```

连接并初始化名称为CAM0的相机。

TriggerCam

原型：

```
TriggerCam(CAM)
```

描述：

触发已经初始化的相机拍照。

必选参数：

CAM：相机的名称，需要与配置视觉工艺时配置的相机名称一致。

返回：

触发结果。

- 0：触发成功
- 1：触发失败

示例：

```
TriggerCam("CAM0")
```

触发名称为CAM0的相机拍照。

SendCam

原型:

```
SendCam(CAM,data)
```

描述:

发送数据给已经初始化的相机。

必选参数:

- CAM: 相机的名称, 需要与配置视觉工艺时配置的相机名称一致。
- data: 发送的数据

返回:

发送结果。

- 0: 发送成功
- 1: 发送失败

示例:

```
SendCam("CAM0", "0,0,0,0")
```

给名称为CAM0的相机发送数据, 数据内容为"0,0,0,0"。

RecvCam

原型:

```
RecvCam(CAM,type)
```

描述:

接收已经初始化的相机发送的数据。

必选参数:

CAM: 相机的名称, 需要与配置视觉工艺时配置的相机名称一致。

可选参数:

type: 接收的数据的类型。取值范围: number或string, 不设置时默认为number。

返回:

- err: 错误码
 - 0: 正常接收

- 1: 接收超时
- 2: 数据格式配置有误, 无法解析
- 3: 网络断开
- n: 数据的组数。
- data: 接收到的数据, 保存在二维数组中。

示例:

```
local err,n,data = RecvCam("CAM0","number")
```

接收名称为CAM0的相机发送的数据, 数据类型为number。

DestroyCam

原型:

```
DestroyCam(CAM)
```

描述:

关闭与相机的连接。

必选参数:

CAM: 相机的名称, 需要与配置视觉工艺时配置的相机名称一致。。

返回:

关闭结果。

- 0: 关闭成功
- 1: 关闭失败

示例:

```
DestroyCam("CAM0")
```

关闭与名称为CAM0的相机的连接。

应用案例

配置完相机参数后, 可通过调用视觉API进行编程, 实现触发相机后接收相机数据。本文以脚本编程为例, 编写脚本获取CAM0的数据并赋值给示教点P2。

```

while true do
  ::create_camera::
  resultInit = InitCam("CAM0")
  if resultInit ~= 0 then
    print("Connect camera failed, code:", resultInit)
    Sleep(1000)
    goto create_camera
  end
  while true do
    TriggerCam("CAM0")
    SendCam("CAM0", "1,2,3,0;")
    err, visionNum, visionData = RecvCam("CAM0", "number")
    if err ~= 0 then
      print("Failed to read data")
      Sleep(1000)
      break
    end

    print("(visionNum):", (visionNum))
    print("(visionData[1][1]):", (visionData[1][1]))
    i = 1
    while not ((visionNum)<i) do
      print(type(P2.coordinate[1]))
      print(P2)
      P2.coordinate[1]=(visionData[i][1])
      P2.coordinate[2]=(visionData[i][2])
      Go(P2, "SYNC=1")
      i = i + 1
    end
    Sleep(10)
  end
  Sleep(10)
end

```

--Connect CAM0 camera

--Trigger CAM0 camera photo

--Send data to CAM0 camera

--Receive CAM0 camera data

--Print how many sets of CAM0 camera data received

--Print the first data of the first group received

--visionData[i][1] is assigned to P2.X

--visionData[i][2] is assigned to P2.Y